

前端面试宝典

第一章 内容提要

前端程序员目前就业形势良好，大前端概念深入人心，大前端最大的特点在于一次开发，同时适用于所有平台，开发者不用为一个APP需要做Android和iOS两种模式而担心。大前端是web统一的时代，利用web不仅能开发出网站，更可以开发手机端web应用和移动端应用程序。

本书是一本解析前端面试题的书，可以帮助求职者更好的准备面试。本书共包含八章，囊括了目前企业中常见的面试题类型和考点，包括Html、CSS、Javascript、Vue、计算机网络、数据结构与算法等最常见的面试题。本书通过技术点解析、代码辅佐的方式，让读者能深刻领会每个考点背后的技术。

本书紧扣面试精髓，对各种技术剖析一针见血，是想找工作的前端程序员和刚毕业学生的面试宝典。

第二章 HTML5面试题

1.H5的新特性有哪些

- 画布(Canvas) API
- 地理(Geolocation) API
- 音频、视频API(audio,video)
- localStorage和sessionStorage
- webworker和 websocket
- header,nav,footer,aside,article,section
- web worker是运行在浏览器后台的js程序，他不影响主程序的运行，是另开的一个js线程，可以用这个线程执行复杂的数据操作，然后把操作结果通过postMessage传递给主线程，这样在进行复杂且耗时的操作时就不会阻塞主线程了

2.Label的作用是什么？是怎么用的？

label标签来定义表单控制间的关系,当用户选择该标签时，浏览器会自动将焦点转到和标签相关的表单控件上。

```
<label for="Name">Number:</label>
<input type="text" name="Name" id="Name"/>
<label>Date:<input type="text" name="B"/></label>
```

3.HTML5的form如何关闭自动完成功能

给不想要提示的 form 或某个 input 设置为 autocomplete=off

4.dom如何实现浏览器内多个标签页之间的通信? (阿里)

- 1) WebSocket、SharedWorker;
- 2) 也可以调用localStorage、cookies等本地存储方式; localStorage另一个浏览上下文里被添加、修改或删除时, 它都会触发一个事件, 我们通过监听事件, 控制它的值来进行页面信息通信;
- 3) 注意quirks: Safari 在无痕模式下设置localStorage值时会抛出 QuotaExceededError 的异常;

5.实现不使用 border 画出1px高的线, 在不同浏览器的标准模式与怪异模式下都能保持一致的效果

```
<div style="height:1px;overflow:hidden;background:red"></div>
```

6.title与h1的区别、b与strong的区别、i与em的区别?

title属性没有明确意义只表示是个标题, H1则表示层次明确的标题, 对页面信息的抓取也有很大的影响;

strong是标明重点内容, 有语气加强的含义, 使用阅读设备阅读网络时: `<strong>` 会重读, 而 `<B>` 是展示强调内容。

i内容展示为斜体, em表示强调的文本;

Physical Style Elements -- 自然样式标签

b, i, u, s, pre

Semantic Style Elements -- 语义样式标签

strong, em, ins, del, code

应该准确使用语义样式标签, 但不能滥用, 如果不能确定时首选使用自然样式标签。

7.你做的页面在哪些浏览器测试过? 这些浏览器的内核分别是什么?

Trident内核:IE系列

Gecko内核:Firefox

Webkit内核:Safari

Blink内核: 是基于Webkit内核的子项目,使用的浏览器有:

Chrome/opera等除IE、Firefox、Safari之外的几乎所有浏览器

几乎所有国产双内核浏览器 (Trident/Blink) 如360、猎豹、qq、百度等

8.每个HTML文件里开头都有个很重要的东西, Doctype, 知道这是干什么的吗?

文档声明。

声明位于文档中的最前面的位置, 处于`<html>`标签之前。此标签可告知浏览器文档使用哪种 HTML 或 XHTML 规范。(重点: 告诉浏览器按照何种规范解析页面)

IE下如不书写文档声明会使用怪异模式解析网页导致一系列CSS兼容性问题

9.div+css的布局较table布局有什么优点

正常场景一般都适用div+CSS布局，

优点：

- 1) 结构与样式分离
- 2) 代码语义性好
- 3) 更符合HTML标准规范
- 4) SEO友好

Table布局的适用场景：

某种原因不方便加载外部CSS的场景，例如邮件正文，此时用table布局可以在无css情况下保持页面布局正常

10.img的alt与title有何异同？ strong与em的异同

alt(alt text):为不能显示图像、窗体或applets的用户代理（UA）， alt属性用来指定替换文字。替换文字的语言由lang属性指定。（在IE浏览器下会在没有title时把alt当成 tool tip显示）

title(tool tip):该属性为设置该属性的元素提供建议性的信息。

em:表现为斜体，语义表示强调

strong:表现为粗体，语义为强烈语气，强调程度超过em

11.简述一下src与href的区别

src用于替换当前元素， href用于在当前文档和引用资源之间确立联系。

src是source的缩写，指向外部资源的位置，指向的内容将会嵌入到文档中当前标签所在位置；在请求src资源时会将其指向的资源下载并应用到文档内，例如js脚本，img图片和frame等元素。

```
<script src ="js.js"></script>
```

当浏览器解析到该元素时，会暂停其他资源的下载和处理，直到将该资源加载、编译、执行完毕，图片和框架等元素也如此，类似于将所指向资源嵌入当前标签内。这也是为什么将js脚本放在底部而不是头部。

href是Hypertext Reference的缩写，指向网络资源所在位置，建立和当前元素（锚点）或当前文档（链接）之间的链接，如果我们在文档中添加

```
<link href="common.css" rel="stylesheet"/>
```

那么浏览器会识别该文档为css文件，就会并行下载资源并且不会停止对当前文档的处理。这也是为什么建议使用link方式来加载css，而不是使用@import方式。

12.知道的网页制作会用到的图片格式有哪些

png-8, png-24, jpeg, gif, svg。

但是上面的那些都不是面试官想要的最后答案。面试官希望听到是Webp。（是否有关注新技术，新鲜事物）

科普一下Webp：WebP格式，谷歌（google）开发的一种旨在加快图片加载速度的图片格式。图片压缩体积大约只有JPEG的2/3，并能节省大量的服务器带宽资源和数据空间。Facebook Ebay等知名网站已经开始测试并使用WebP格式。

在质量相同的情况下，WebP格式图像的体积要比JPEG格式图像小40%

13.在css/js代码上线之后开发人员经常会优化性能，从用户刷新网页开始，一次js请求一般情况下有哪些地方会有缓存处理

dns缓存，cdn缓存，浏览器缓存，服务器缓存

14.一个页面上有大量的图片（大型电商网站），加载很慢，你有哪些方法优化这些图片的加载，给用户更好的体验

- 1) 图片懒加载，在页面上的未可视区域可以添加一个滚动条事件，判断图片位置与浏览器顶端的距离与页面的距离，如果前者小于后者，优先加载。
- 2) 如果为幻灯片、相册等，可以使用图片预加载技术，将当前展示图片的前一张和后一张优先下载。
- 3) 如果图片为css图片，可以使用CSSsprite, SVGsprite, Iconfont、Base64等技术。
- 4) 如果图片过大，可以使用特殊编码的图片，加载时会先加载一张压缩的特别厉害的缩略图，以提高用户体验。
- 5) 如果图片展示区域小于图片的真实大小，则因在服务器端根据业务需要先行进行图片压缩，图片压缩后大小与展示一致

15.你如何理解HTML结构的语义化

- 1) 更符合W3C统一的规范标准，是技术趋势。
- 2) 没有样式时浏览器的默认样式也能让页面结构很清晰。
- 3) 对功能障碍用户友好。屏幕阅读器（如果访客有视障）会完全根据你的标记来“读”你的网页。
- 4) 对其他非主流终端设备友好。例如机顶盒、PDA、各种移动终端。
- 5) 对SEO友好。

16.谈谈以前端角度出发做好SEO需要考虑什么

搜索引擎主要以:

外链数量和质量,网页内容和结构等来决定某关键字下的网页搜索排名。

前端应该注意网页结构和内容方面的情况:

- 1) Meta标签优化: 主要包括主题 (Title), 网站描述(Description)。还有一些其它的隐藏文字比如 Author (作者), Category (目录), Language (编码语种) 等, 符合W3C规范的语义性标签的使用
- 2) 如何选取关键词并在网页中放置关键词: 搜索就得用关键词。关键词分析和选择是SEO最重要的工作之一。首先要给网站确定主关键词 (一般在5个上下), 然后针对这些关键词进行优化, 包括关键词密度 (Density), 相关度 (Relavancy), 突出性 (Prominency) 等等。

17.html5有哪些新特性、移除了那些元素

新特性:

- 1) 拖拽释放(Drag and drop) API
- 2) 语义化更好的内容标签 (header,nav,footer,aside,article,section)
- 3) 音频、视频API(audio,video)
- 4) 画布(Canvas) API
- 5) 地理(Geolocation) API
- 6) 本地离线存储 localStorage 长期存储数据, 浏览器关闭后数据不丢失;
- 7) sessionStorage 的数据在浏览器关闭后自动删除
- 8) 表单控件, calendar、date、time、email、url、search
- 9) 新的技术webworker, websocket, Geolocation

移除的元素:

- 1) 纯表现的元素: basefont, big, center, font,s, strike, tt, u;
- 2) 对可用性产生负面影响的元素: frame, frameset, noframes;

18.如何处理HTML5新标签的浏览器兼容问题

IE8/IE7/IE6支持通过 document.createElement 方法产生的标签, 可以利用这一特性让这些浏览器支持HTML5 新标签, 浏览器支持新标签后, 还需要添加标签默认的样式 (当然最好的方式是直接使用成熟的框架、使用最多的是html5shim框架) :

```
<!--[if lt IE 9]>
<script> src="http://html5shim.googlecode.com/svn/trunk/html5.js"</script>
<![endif]-->
```

19.如何区分HTML和HTML5?

DOCTYPE声明新增的结构元素、功能元素

20.HTML5 Canvas元素有什么用

Canvas 元素用于在网页上绘制图形，该元素标签强大之处在于可以直接在 HTML 上进行图形操作

21. 如何在 HTML5 页面中嵌入音频

HTML 5 包含嵌入音频文件的标准方式，支持的格式包括 MP3、Wav 和 Ogg：

```
<audio controls>
  <source src="jamshed.mp3" type="audio/mpeg">
  Your browser does'nt support audio embedding feature.
</audio>
```

22.如何在HTML5页面中嵌入视频

和音频一样，HTML5 定义了嵌入视频的标准方法，支持的格式包括：MP4、WebM 和 Ogg：

```
<video width="450" height="340" controls>
  <source src="jamshed.mp4" type="video/mp4">
  Your browser does'nt support video embedding feature.
</video>
```

23.HTML5引入什么新的表单属性

datalist datetime output keygen date month week time number range emailurl

24.语义化的理解

用正确的标签做正确的事情！

html语义化就是让页面的内容结构化，便于对浏览器、搜索引擎解析；

在没有样式CSS情况下也以一种文档格式显示，并且是容易阅读的。

搜索引擎的爬虫依赖于标记来确定上下文和各个关键字的权重，利于 SEO。

使阅读源代码的人对网站更容易将网站分块，便于阅读维护理解

25.介绍一下你对浏览器内核的理解

主要分成两部分：渲染引擎(layout engine或Rendering Engine)和JS引擎

渲染引擎：负责取得网页的内容（HTML、XML、图像等等）、整理讯息（例如加入CSS等），以及计算网页的显示方式，然后会输出至显示器或打印机。浏览器的内核的不同对于网页的语法解释会有不同，所以渲染的效果也不相同。所有网页浏览器、电子邮件客户端以及其它需要编辑、显示网络内容的应用程序都需要内核

JS引擎则：解析和执行javascript来实现网页的动态效果

最开始渲染引擎和JS引擎并没有区分的很明确，后来JS引擎越来越独立，内核就倾向于只指渲染引擎

26.浏览器是怎么对HTML5的离线储存资源进行管理和加载的呢

在线的情况下，浏览器发现html头部有manifest属性，它会请求manifest文件，如果是第一次访问app，那么浏览器就会根据manifest文件的内容下载相应的资源并且进行离线存储。如果已经访问过app并且资源已经离线存储了，那么浏览器就会使用离线的资源加载页面，然后浏览器会对比新的manifest文件与旧的manifest文件，如果文件没有发生改变，就不做任何操作，如果文件改变了，那么就会重新下载文件中的资源并进行离线存储。

离线的情况下，浏览器就直接使用离线存储的资源

27.请描述一下cookies, sessionStorage 和localStorage的区别

cookie是网站为了标示用户身份而储存在用户本地终端（Client Side）上的数据（通常经过加密）

cookie数据始终在同源的http请求中携带（即使不需要），记会在浏览器和服务器间来回传递

sessionStorage和localStorage不会自动把数据发给服务器，仅在本地保存

存储大小：

cookie数据大小不能超过4k

sessionStorage和localStorage虽然也有存储大小的限制，但比cookie大得多，可以达到5M或更大

有期时间：

localStorage 存储持久数据，浏览器关闭后数据不丢失除非主动删除数据

sessionStorage 数据在当前浏览器窗口关闭后自动删除

cookie 设置的cookie过期时间之前一直有效，即使窗口或浏览器关闭

28.css sprite是什么,有什么优缺点

概念：将多个小图片拼接到一个图片中。通过background-position和元素尺寸调节需要显示的背景图案。

优点：

减少HTTP请求数，极大地提高页面加载速度

增加图片信息重复度，提高压缩比，减少图片大小

更换风格方便，只需在一张或几张图片上修改颜色或样式即可实现

缺点：

图片合并麻烦

维护麻烦，修改一个图片可能需要从新布局整个图片，样式

29.canvas 如何绘制一个三角形|正方形

moveto 是移动到某个坐标， lineto 是从当前坐标连线到某个坐标。

这两个函数加起来就是画一条直线。画线要用“笔”，那么MoveTo()把笔要画的起始位置固定了(x,y) 然后要固定终止位置要用到LineTo函数确定终止位置 (xend,yend) ,这样一条线就画出来了。每次与前面一个坐标相连。

stroke() 方法会实际地绘制出通过 moveTo() 和 lineTo() 方法定义的路径。默认颜色是黑色。

```
<!DOCTYPE HTML5><html><head>
  <meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
  <title>画布</title>
</head> <body>
  <canvas id="myCanvas" width="200" height="100" style="border:1px solid
#c3c3c3;">
    Your browser does not support the canvas element.
  </canvas>
  <script type="text/javascript">
    var c=document.getElementById("myCanvas");//三角形
    var cxt=c.getContext("2d");
    cxt.moveTo(10,10);
    cxt.lineTo(50,50);
    cxt.lineTo(10,50);
    cxt.lineTo(10,10);
    cxt.stroke();//正方形
    var cxt2=c.getContext("2d");
    cxt2.moveTo(60,10);
    cxt2.lineTo(100,10);
    cxt2.lineTo(100,50);
    cxt2.lineTo(60,50);
    cxt2.lineTo(60,10);
    cxt2.stroke();
  </script></body></html>
```

30.弹性盒子模型? flex|box区别?

1) 引入弹性盒布局模型的目的是提供一种更加有效的方式来对一个容器中的条目进行排列、对齐和分配空白空间。

即便容器中条目的尺寸未知或是动态变化的，弹性盒布局模型也能正常的工作。在该布局模型中，容器会根据布局的需要，调整其中包含的条目的尺寸和顺序来最好地填充所有可用的空间。

当容器的尺寸由于屏幕大小或窗口尺寸发生变化时，其中包含的条目也会被动态地调整。比如当容器尺寸变大时，其中包含的条目会被拉伸以占满多余的空白空间；当容器尺寸变小时，条目会被缩小以防止超出容器的范围。弹性盒布局是与方向无关的。

在传统的布局方式中，block 布局是把块在垂直方向从上到下依次排列的；

而 inline 布局则是在水平方向来排列。弹性盒布局并没有这样内在的方向限制，可以由开发人员自由操作。

2) flex和box的区别: display: box 是老规范, 要兼顾古董机子就加上它; 父级元素有display:box;属性之后。他的子元素里面加上box-flex属性。可以让子元素按照父元素的宽度进行一定比例的分占空间。flex是最新的, 董机老机子不支持的;

父元素设置display:flex后, 子元素宽度会随父元素宽度的改变而改变, 而display:box不会。Android UC浏览器只支持display: box语法; 而iOS UC浏览器则支持两种方式。

31.解释在ie低版本下的怪异盒模型和c3的怪异盒模型 和 弹性盒模型?

IE当padding+border的值小于width或者height:

盒模型的宽度=margin(左右)+width (width已经包含了padding和border的值)

盒模型的高度=margin(上下)+height (height已经包含了padding和border的值)

当padding+border的值大于width或者height:

盒模型的宽度=margin(左右)+padding(左右)+border(左右)

盒模型的高度=margin(上下)+padding(上下)+border(上下)+19px (加一个默认行高 19px) 所以相当于是padding+border和width或者height比大小, 谁大取谁。

以上几种DOCTYPE都是标准的文档类型, 无论使用哪种模式完整定义DOCTYPE, 都会触发标准模式, 而如果DOCTYPE缺失则在ie6, ie7, ie8下将会触发怪异模式(quirks 模式) CSS3box-sizing有两个值一个是content-box, 另一个是border-box。

当设置为box-sizing:content-box时, 将采用标准模式解析计算, 也是默认模式;

当设置为box-sizing:border-box时, 将采用怪异模式解析计算;

Css3弹性盒模型引入了新的盒子模型—弹性盒模型, 该模型决定一个盒子在其他盒子中的分布方式以及如何处理可用的空间。使用该模型, 可以很轻松的创建自适应浏览器窗口的流动布局或自适应字体大小的弹性布局。

第三章 CSS面试题

1.盒模型

```
复制/* 红色区域的大小是多少? 200 - 20*2 - 20*2 = 120 */
```

```
.box {  
    width: 200px;  
    height: 200px;  
    padding: 20px;  
    margin: 20px;  
    background: red;  
    border: 20px solid black;  
    box-sizing: border-box;  
}
```

```
复制/* 标准模型 */
```

```
box-sizing:content-box;  
/*IE模型*/  
box-sizing:border-box;
```

[回到顶部](#)

2.如何实现一个最大的正方形

用 `padding-bottom` 撑开边距

```
复制 section {  
    width:100%;  
    padding-bottom: 100%;  
    background: #333;  
}
```

[回到顶部](#)

3.一行水平居中，多行居左

```
复制<div><span>我是多行文字。我是多行文字。我是多行文字。我是多行文字。我是多行文字。我是多行文  
字。我是多行文字。我是多行文字。我是多行文字。我是多行文字。</span></div>  
<div><span>我是一行文字</span></div>  
  
<style>  
div{text-align: center;}  
div span{display: inline-block;text-align: left;}  
</style>
```

[回到顶部](#)

4.水平垂直居中

水平居中

- 1) 若是行内元素, 给其父元素设置 `text-align:center`,即可实现行内元素水平居中.
- 2) 若是块级元素, 该元素设置 `margin:0 auto`即可.
- 3) 若子元素包含 `float:left` 属性, 为了让子元素水平居中, 则可以让父元素宽度设置为`fit-content`,并且配合 `margin`, 作如下设置:

```
.parent{  
    width: -moz-fit-content;  
    width: -webkit-fit-content;  
    width: fit-content;  
    margin:0 auto;  
}复制代码
```

`fit-content`是CSS3中给`width`属性新加的一个属性值,它配合`margin`可以轻松实现水平居中, 目前只支持 Chrome 和 Firefox浏览器.

- 4) 使用flex 2012年版本布局, 可以轻松的实现水平居中, 子元素设置如下:

```
.son{  
    display: flex;  
    justify-content: center;  
}复制代码
```

- 5) 使用flex 2009年版本, 父元素`display: box;box-pack: center`;如下设置:

```
.parent {
  display: -webkit-box;
  -webkit-box-orient: horizontal;
  -webkit-box-pack: center;
  display: -moz-box;
  -moz-box-orient: horizontal;
  -moz-box-pack: center;
  display: -o-box;
  -o-box-orient: horizontal;
  -o-box-pack: center;
  display: -ms-box;
  -ms-box-orient: horizontal;
  -ms-box-pack: center;
  display: box;
  box-orient: horizontal;
  box-pack: center;
}
```

复制代码

6) 使用CSS3中新增的transform属性, 子元素设置如下:

```
.son{
  position:absolute;
  left:50%;
  transform:translate(-50%,0);
}
```

复制代码

7) 使用绝对定位方式, 以及负值的margin-left, 子元素设置如下:

```
.son{
  position:absolute;
  width:固定;
  left:50%;
  margin-left:-0.5宽度;
}
```

复制代码

8) 使用绝对定位方式, 以及left:0;right:0;margin:0 auto; 子元素设置如下:

```
.son{
  position:absolute;
  width:固定;
  left:0;
  right:0;
  margin:0 auto;
}
```

复制代码

垂直居中

单行文本

1) 若元素是单行文本, 则可设置 line-height 等于父元素高度

行内块级元素

2) 若元素是行内块级元素, 基本思想是使用display: inline-block, vertical-align: middle和一个伪元素让内容块处于容器中央.

```
.parent::after, .son{
    display:inline-block;
    vertical-align:middle;
}
.parent::after{
    content:'';
    height:100%;
}复制代码
```

这是一种很流行的方法, 也适应IE7.

元素高度不定

3) 可用 **vertical-align** 属性, 而vertical-align只有在父层为 td 或者 th 时, 才会生效, 对于其他块级元素, 例如 div、p 等, 默认情况是不支持的. 为了使用vertical-align, 我们需要设置父元素display:table, 子元素display:table-cell;vertical-align:middle;

优点

元素高度可以动态改变, 不需再CSS中定义, 如果父元素没有足够空间时, 该元素内容也不会被截断.

缺点

IE6~7, 甚至IE8 beta中无效.

4) 可用 **Flex 2012版**, 这是CSS布局未来的趋势. Flexbox是CSS3新增属性, 设计初衷是为了解决像垂直居中这样的常见布局问题. 相关的文章如 [《弹性盒模型Flex指南》](#)

父元素做如下设置即可保证子元素垂直居中:

```
.parent {
    display: flex;
    align-items: center;
}复制代码
```

优点

- 内容块的宽高任意, 优雅的溢出.
- 可用于更复杂高级的布局技术中.

缺点

- IE8/IE9不支持
- 需要浏览器厂商前缀
- 渲染上可能会有一些问题

5) 使用flex 2009版.

```
.parent {
    display: box;
    box-orient: vertical;
    box-pack: center;
}复制代码
```

优点

实现简单, 扩展性强

缺点

兼容性差, 不支持IE

6) 可用 **transform** , 设置父元素相对定位(position:relative), 子元素如下css样式:

```
.son{
    position:absolute;
    top:50%;
    -webkit-transform: translate(-50%,-50%);
    -ms-transform: translate(-50%,-50%);
    transform: translate(-50%,-50%);
}复制代码
```

优点

代码量少

缺点

IE8不支持, 属性需要追加浏览器厂商前缀, 可能干扰其他 transform 效果, 某些情形下会出现文本或元素边界渲染模糊的现象.

元素高度固定

7) 设置父元素相对定位(position:relative), 子元素如下css样式:

```
.son{
    position:absolute;
    top:50%;
    height:固定;
    margin-top:-0.5高度;
}复制代码
```

优点

适用于所有浏览器.

缺点

父元素空间不够时, 子元素可能不可见(当浏览器窗口缩小时,滚动条不出现时).如果子元素设置了 overflow:auto, 则高度不够时, 会出现滚动条.

8) 设置父元素相对定位(position:relative), 子元素如下css样式:

```
.son{
  position:absolute;
  height:固定;
  top:0;
  bottom:0;
  margin:auto 0;
}复制代码
```

优点

简单

缺点

没有足够空间时, 子元素会被截断, 但不会有滚动条.

总结

水平居中较为简单, 共提供了8种方法, 一般情况下 text-align:center,margin:0 auto; 足矣

- ① text-align:center;
- ② margin:0 auto;
- ③ width:fit-content;
- ④ flex
- ⑤ 盒模型
- ⑥ transform
- ⑦ ⑧ 两种不同的绝对定位方法

垂直居中, 共提供了8种方法.

- ① 单行文本, line-height
- ② 行内块级元素, 使用 display: inline-block, vertical-align: middle; 加上伪元素辅助实现
- ③ vertical-align
- ④ flex
- ⑤ 盒模型
- ⑥ transform
- ⑦ ⑧ 两种不同的绝对定位方法

我们发现, flex, 盒模型, transform, 绝对定位, 这几种方法同时适用于水平居中和垂直居中.

希望对大家有所帮助.

5.两栏布局, 左边固定, 右边自适应, 左右不重叠

flex做自适应布局很容易, 但兼容性不好, 这里统一不用flex布局

```
复制.left{
  float:left;
  width:300px;
  margin-right: 10px;
  background: red;
}
.right{
  overflow: hidden; /* 创建BFC */
  background: yellow;
}
```

[回到顶部](#)

6.如何实现左右等高布局

table 布局兼容性最好，当然 flex 布局的 align-items: stretch; 也行

```
复制<div class="layout">
  <div class="layout left">left</div>
  <div class="layout right">center</div>
</div>

<style>
.layout{
  display: table;
  width: 100%;
}
.layout div{
  display: table-cell;
}
.layout .left{
  width: 50%;
  height: 200px;
  background: red;
}
.layout .right{
  width: 50%;
  background: yellow;
}
</style>
```

[回到顶部](#)

7.画三角形

```
复制.shape {
  width: 0;
  height: 0;
  border-left: 50px solid transparent;
  border-right: 50px solid transparent;
  border-top: 50px solid transparent;
  border-bottom: 50px solid blue;
  background: white;
}
```

[回到顶部](#)

8.link @import导入css

1. link是XHTML标签，除了加载CSS外，还可以定义RSS等其他事务；@import属于CSS范畴，只能加载CSS
2. link引用CSS时，在页面载入时同时加载；@import需要页面网页完全载入以后加载
3. link无兼容问题；@import是在CSS2.1提出的，低版本的浏览器不支持
4. link支持使用JavaScript控制DOM去改变样式；而@import不支持

[回到顶部](#)

9.BFC理解

BFC触发条件：

1. 根元素，即html
2. float的值不为none（默认）
3. position的值为absolute或fixed
4. overflow的值不为visible（默认）
5. display的值为inline-block、table-cell、table-caption

BFC特性：

1. 内部的Box会在垂直方向上一个接一个放置。
2. Box垂直方向的距离由margin决定，属于同一个BFC的两个相邻Box的margin会发生重叠。
3. 每个元素的margin box 的左边，与包含块border box的左边相接触。
4. BFC的区域不会与float box重叠。（可用于清浮动）
5. BFC是页面上的一个隔离的独立容器，容器里面的子元素不会影响到外面的元素。
6. 计算BFC的高度时，浮动元素也会参与计算。

第四章 JS面试题

1.判断 js 类型的方式

1. typeof

可以判断出'string','number','boolean','undefined','symbol'
但判断 typeof(null) 时值为 'object'; 判断数组和对象时值均为 'object'

2. instanceof

原理是 构造函数的 prototype 属性是否出现在对象的原型链中的任何位置

```
复制function A() {}  
let a = new A();  
a instanceof A    //true,因为 Object.getPrototypeOf(a) === A.prototype;
```

3. Object.prototype.toString.call()

常用于判断浏览器内置对象,对于所有基本的数据类型都能进行判断，即使是 null 和 undefined

4. Array.isArray()

用于判断是否为数组

[回到顶部](#)

2.ES5 和 ES6 分别几种方式声明变量

ES5 有两种：`var` 和 `function`

ES6 有六种：增加四种，`let`、`const`、`class` 和 `import`

注意：`let`、`const`、`class` 声明的全局变量再也不会和全局对象的属性挂钩

[回到顶部](#)

3.闭包的概念？优缺点？

闭包的概念：闭包就是能读取其他函数内部变量的函数。

优点：

1. 避免全局变量的污染
2. 希望一个变量长期存储在内存中（缓存变量）

缺点：

1. 内存泄露（消耗）
2. 常驻内存，增加内存使用量

[回到顶部](#)

4.浅拷贝和深拷贝

• 浅拷贝

```
复制// 第一层为深拷贝
Object.assign()
Array.prototype.slice()
扩展运算符 ...
```

• 深拷贝

```
复制JSON.parse(JSON.stringify())
```

递归函数

```
复制function cloneObject(obj) {
  var newObj = {} //如果不是引用类型，直接返回
  if (typeof obj !== 'object') {
    return obj
  }
  //如果是引用类型，遍历属性
  else {
    for (var attr in obj) {
      //如果某个属性还是引用类型，递归调用
      newObj[attr] = cloneObject(obj[attr])
    }
  }
}
```

```
    return newObj
  }
```

[如何实现一个深拷贝](#)

[详细解析赋值、浅拷贝和深拷贝的区别](#)

[回到顶部](#)

5.数组去重的方法

1.ES6 的 Set

```
复制let arr = [1,1,2,3,4,5,5,6]
let arr2 = [...new Set(arr)]
```

2.reduce()

```
复制let arr = [1,1,2,3,4,5,5,6]
let arr2 = arr.reduce(function(ar,cur) {
  if(!ar.includes(cur)) {
    ar.push(cur)
  }

  return ar
},[])
```

3.filter()

```
复制// 这种方法会有一个问题: [1, '1']会被当做相同元素, 最终输入[1]
let arr = [1,1,2,3,4,5,5,6]
let arr2 = arr.filter(function(item,index) {
  // indexOf() 方法可返回某个指定的 字符串值 在字符串中首次出现的位置
  return arr.indexOf(item) === index
})
```

[回到顶部](#)

6.DOM 事件有哪些阶段? 谈谈对事件代理的理解

分为三大阶段: 捕获阶段--目标阶段--冒泡阶段

事件代理简单说就是: 事件不直接绑定到某元素上, 而是绑定到该元素的父元素上, 进行触发事件操作时(例如'click'), 再通过条件判断, 执行事件触发后的语句(例如'alert(e.target.innerHTML)')

好处: (1)使代码更简洁; (2)节省内存开销

[回到顶部](#)

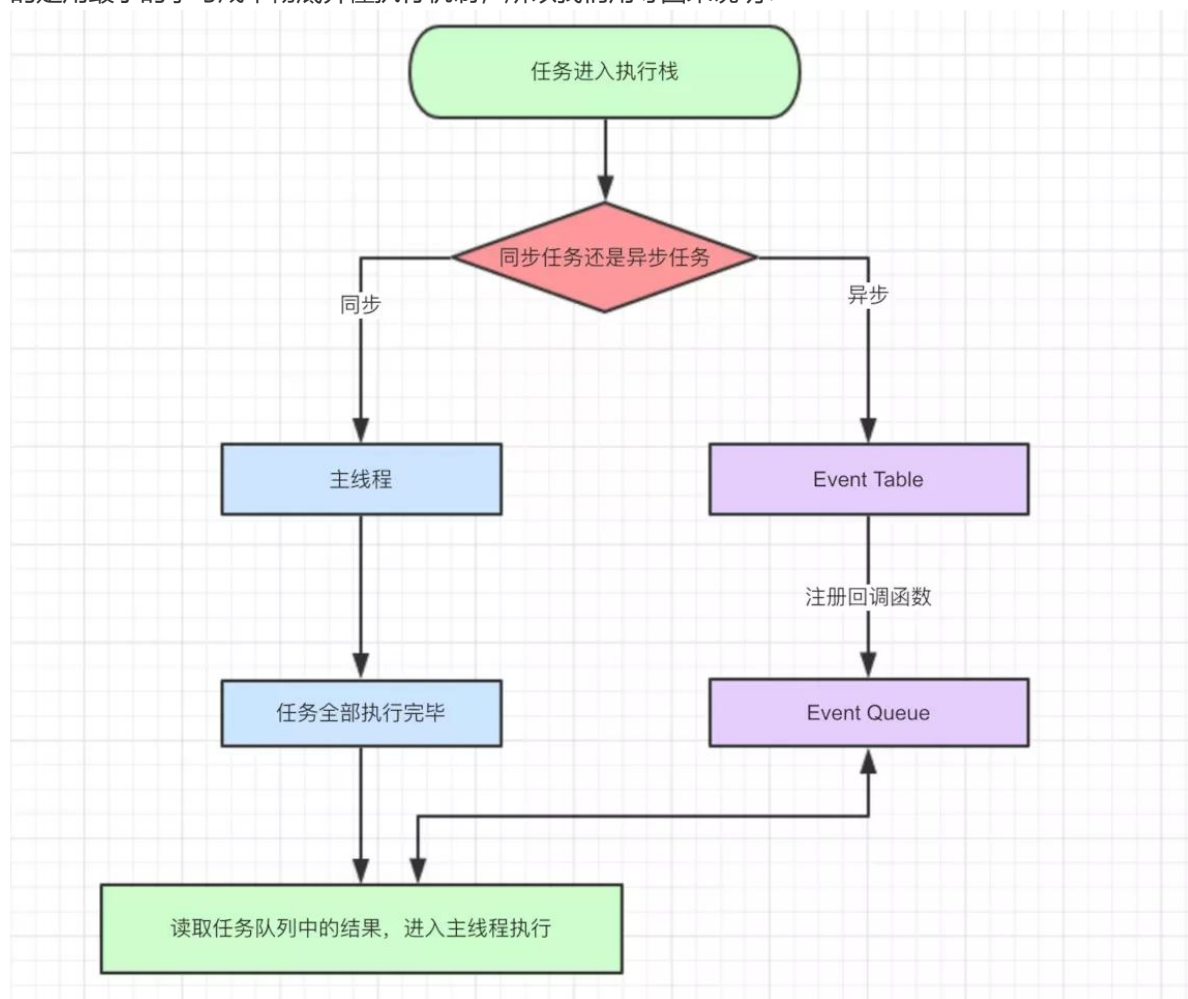
7.js 执行机制、事件循环

JavaScript 语言的一大特点就是单线程, 同一个时间只能做一件事。单线程就意味着, 所有任务需要排队, 前一个任务结束, 才会执行后一个任务。如果前一个任务耗时很长, 后一个任务就不得不一直等着。JavaScript 语言的设计者意识到这个问题, 将所有任务分成两种, 一种是**同步任务**

(synchronous), 另一种是**异步任务 (asynchronous)**, 在所有同步任务执行完之前, 任何的异步任务是不会执行的。

当我们打开网站时, 网页的渲染过程就是一大堆同步任务, 比如页面骨架和页面元素的渲染。而像加

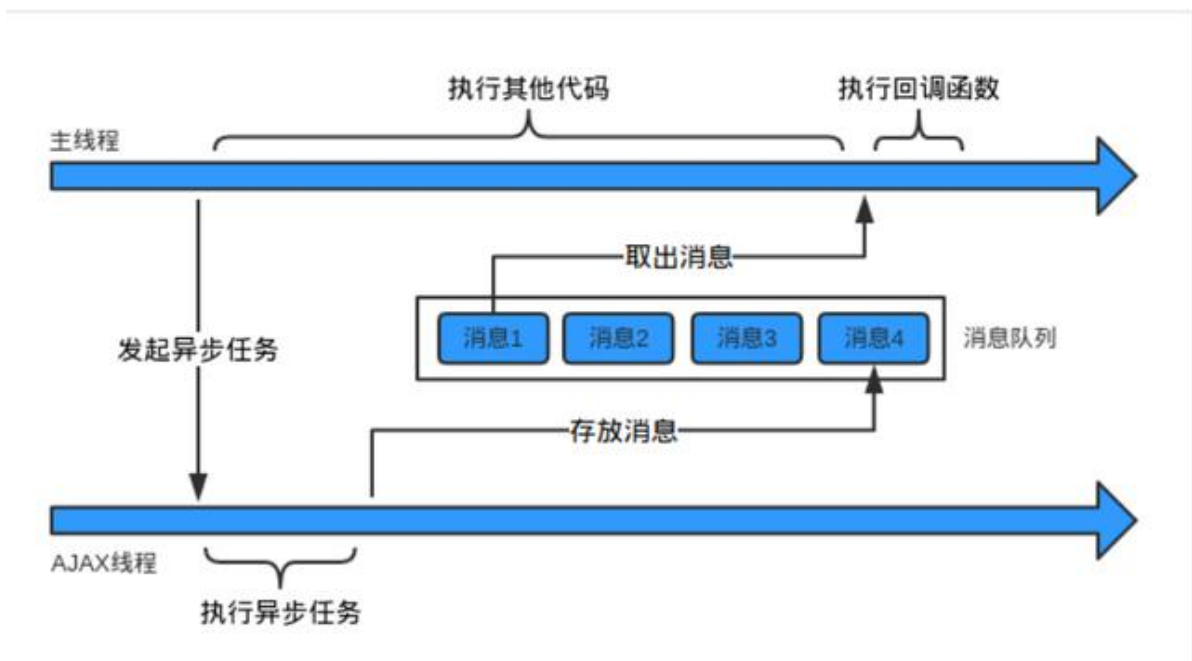
载图片音乐之类占用资源大耗时久的任务，就是异步任务。关于这部分有严格的文字定义，但本文的目的是用最小的学习成本彻底弄懂执行机制，所以我们用导图来说明：



导图要表达的内容用文字来表述的话：

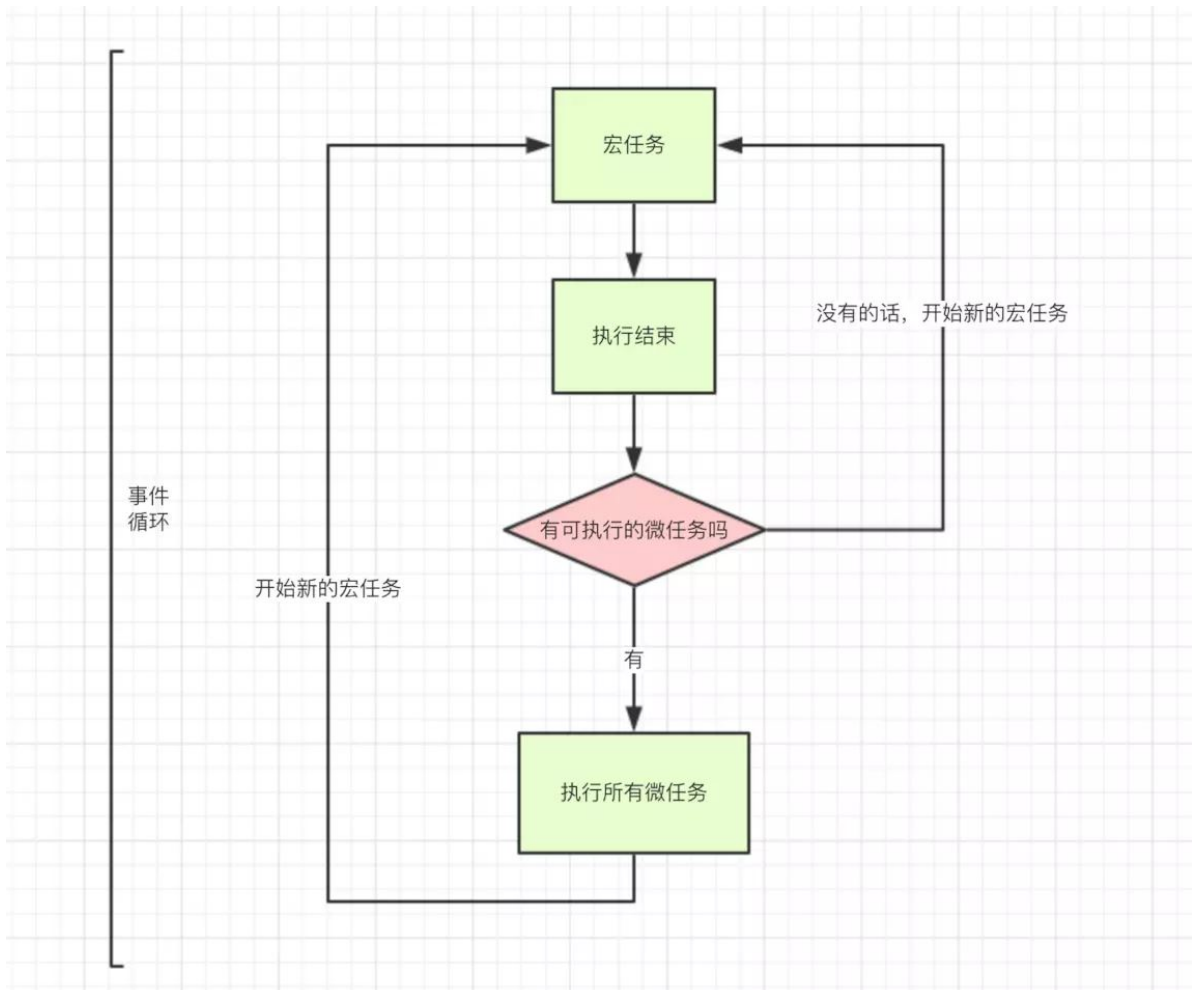
同步和异步任务分别进入不同的执行"场所"，同步的进入主线程，异步的进入 Event Table 并注册函数。当**指定的事情完成时**，Event Table 会将这个函数移入 Event Queue。主线程内的任务执行完毕为空，会去 Event Queue 读取对应的函数，进入主线程执行。上述过程会不断重复，也就是常说的 Event Loop(事件循环)。

我们不禁要问了，那怎么知道主线程执行栈为空啊？js 引擎存在 monitoring process 进程，会持续不断的检查主线程执行栈是否为空，一旦为空，就会去 Event Queue 那里检查是否有等待被调用的函数。换一张图片也许更好理解主线程的执行过程：



上图用文字表述就是：主线程从"任务队列"中读取事件，这个过程是循环不断的，所以整个的这种运行机制又称为 Event Loop（事件循环）。只要主线程空了，就会去读取"任务队列"，这就是 JavaScript 的运行机制。

说完 JS 主线程的执行机制，下面说说经常被问到的 JS 异步中 宏任务（macrotasks）、微任务（microtasks）执行顺序。**JS 异步有一个机制，就是遇到宏任务，先执行宏任务，将宏任务放入 Event Queue，然后再执行微任务，将微任务放入 Event Queue，但是，这两个 Queue 不是一个 Queue。当你往外拿的时候先从微任务里拿这个回调函数，然后再从宏任务的 Queue 拿宏任务的回调函数。**如下图：



宏任务：整体代码 script, setTimeout, setInterval

微任务：Promise, process.nextTick

参考链接：[这一次，彻底弄懂JavaScript执行机制](#)

[回到顶部](#)

8.介绍下 promise.all

Promise.all()方法将多个Promise实例包装成一个Promise对象（p），接受一个数组（p1,p2,p3）作为参数，数组中不一定需要都是Promise对象，但是一定具有Iterator接口，如果不是的话，就会调用Promise.resolve将其转化为Promise对象之后再进行处理。

使用Promise.all()生成的Promise对象（p）的状态是由数组中的Promise对象（p1,p2,p3）决定的。

1. 如果所有的Promise对象（p1,p2,p3）都变成fulfilled状态的话，生成的Promise对象（p）也会变成fulfilled状态，
p1,p2,p3三个Promise对象产生的结果会组成一个数组返回给传递给p的回调函数。
2. 如果p1,p2,p3中有一个Promise对象变为rejected状态的话，p也会变成rejected状态，第一个被rejected的对象的返回值会传递给p的回调函数。
Promise.all()方法生成的Promise对象也会有一个catch方法来捕获错误处理，但是如果数组中的Promise对象变成rejected状态时，
并且这个对象还定义了catch的方法，那么rejected的对象会执行自己的catch方法。
并且返回一个状态为fulfilled的Promise对象，Promise.all()生成的对象会接受这个Promise对象，不会返回rejected状态。

[回到顶部](#)

9.async 和 await

主要考察宏任务和微任务，搭配promise，询问一些输出的顺序

原理：**async** 和 **await** 用了同步的方式去做异步，**async** 定义的函数的返回值都是 **promise**，**await** 后面的函数会先执行一遍，然后就会跳出整个 **async** 函数来执行后面js栈的代码

[回到顶部](#)

10.ES6 的 class 和构造函数的区别

class 的写法只是语法糖，和之前 prototype 差不多，但还是有细微差别的，下面看看：

1. 严格模式

类和模块的内部，默认就是严格模式，所以不需要使用 `use strict` 指定运行模式。只要你的代码写在类或模块之中，就只有严格模式可用。考虑到未来所有的代码，其实都是运行在模块之中，所以 ES6 实际上把整个语言升级到了严格模式。

2. 不存在提升

类不存在变量提升（hoist），这一点与 ES5 完全不同。

```
复制new Foo(); // ReferenceError
class Foo {}
```

3. 方法默认是不可枚举的

ES6 中的 class，它的方法（包括静态方法和实例方法）默认是不可枚举的，而构造函数默认是可枚举的。细想一下，这其实是个优化，让你在遍历时候，不需要再判断 hasOwnProperty 了

4. class 的所有方法（包括静态方法和实例方法）都没有原型对象 prototype，所以也没有 [[construct]]，不能使用 new 来调用。

5. class 必须使用 new 调用，否则会报错。这是它跟普通构造函数的一个主要区别，后者不用 new 也可以执行。

6. ES5 和 ES6 子类 this 生成顺序不同

ES5 的继承先生成了子类实例，再调用父类的构造函数修饰子类实例。ES6 的继承先生成父类实例，再调用子类的构造函数修饰父类实例。这个差别使得 ES6 可以继承内置对象。

7. ES6可以继承静态方法，而构造函数不能

[回到顶部](#)

11.transform、translate、transition 分别是什么属性？CSS 中常用的实现动画方式

三者属性说明

transform 是指变换、变形，是 css3 的一个属性，和 width, height 属性一样；
translate 是 transform 的属性值，是指元素进行 2D(3D)维度上位移或范围变换；
transition 是指过渡效果，往往理解成简单的动画，需要有触发条件。

这里可以补充下 transition 和 animation 的比较，前者一般定义开始结束两个状态，需要有触发条件；而后者引入了关键帧、速度曲线、播放次数等概念，更符合动画的定义，且无需触发条件

[回到顶部](#)

12.介绍一下rAF(requestAnimationFrame)

专门用来做动画，不卡顿，用法和setTimeout一样。对 rAF 的阐述 [MDN 资料](#)

定时器一直是 js 动画的核心技术，但它们不够精准，因为定时器时间参数是指将执行代码放入 UI 线程队列中等待的时间，如果前面有其他任务队列执行时间过长，则会导致动画延迟，效果不精确等问题。

所以处理动画循环的关键是知道延迟多长时间合适：时间要足够短，才能让动画看起来比较柔滑平顺，避免多余性能损耗；时间要足够长，才能让浏览器准备好变化渲染。这个时候 rAF 就出现了，采用系统时间间隔(大多浏览器刷新频率是 60Hz，相当于 1000ms/60≈16.6ms)，保持最佳绘制效率，不会因为间隔时间过短，造成过度绘制，增加开销；也不会因为间隔时间太长，使用动画卡顿不流畅，让各种网页动画效果能够有一个统一的刷新机制。并且 rAF 会把每一帧中的所有 DOM 操作集中起来，在一次重绘或回流中就完成。

详情：[CSS3动画那么强，requestAnimationFrame还有毛线用？](#)

[回到顶部](#)

13.javascript 的垃圾回收机制讲一下

定义：指一块被分配的内存既不能使用，又不能回收，直到浏览器进程结束。

像 C 这样的编程语言，具有低级内存管理原语，如 malloc()和 free()。开发人员使用这些原语显式地对操作系统的内存进行分配和释放。

而 JavaScript 在创建对象(对象、字符串等)时会为它们分配内存，不再使用时会“自动”释放内存，这个过程称为垃圾收集。

内存生命周期中的每一个阶段:

分配内存 — 内存是由操作系统分配的, 它允许您的程序使用它。在低级语言(例如 C 语言)中, 这是一个开发人员需要自己处理的显式执行的操作。然而, 在高级语言中, 系统会自动为你分配内存。

使用内存 — 这是程序实际使用之前分配的内存, 在代码中使用分配的变量时, 就会发生读和写操作。

释放内存 — 释放所有不再使用的内存,使之成为自由内存,并可以被重利用。与分配内存操作一样,这一操作在低级语言中也是需要显式地执行。

四种常见的内存泄漏: 全局变量, 未清除的定时器, 闭包, 以及 dom 的引用

1. 全局变量 不用 var 声明的变量, 相当于挂载到 window 对象上。如: b=1; 解决: 使用严格模式
2. 被遗忘的定时器和回调函数
3. 闭包
4. 没有清理的 DOM 元素引用

[回到顶部](#)

14.对前端性能优化有什么了解? 一般都通过那几个方面去优化的?

前端性能优化的七大手段

1. 减少请求数量
2. 减小资源大小
3. 优化网络连接
4. 优化资源加载
5. 减少重绘回流
6. 性能更好的API
7. webpack优化

第五章 JS高频手写代码题

1.实现 new 方法

```
复制/*
 * 1.创建一个空对象
 * 2.链接到原型
 * 3.绑定this值
 * 4.返回新对象
 */
// 第一种实现
function createNew() {
  let obj = {} // 1.创建一个空对象

  let constructor = [].shift.call(arguments)
  // let [constructor,...args] = [...arguments]

  obj.__proto__ = constructor.prototype // 2.链接到原型

  let result = constructor.apply(obj, arguments) // 3.绑定this值
  // let result = constructor.apply(obj, args)

  return typeof result === 'object' ? result : obj // 4.返回新对象
}

function People(name,age) {
  this.name = name
```

```
    this.age = age
  }

  let peo = createNew(People, 'Bob', 22)
  console.log(peo.name)
  console.log(peo.age)
```

[回到顶部](#)

2.实现 Promise

```
复制// 未添加异步处理等其他边界情况
// ①自动执行函数, ②三个状态, ③then
class Promise {
  constructor (fn) {
    // 三个状态
    this.state = 'pending'
    this.value = undefined
    this.reason = undefined
    let resolve = value => {
      if (this.state === 'pending') {
        this.state = 'fulfilled'
        this.value = value
      }
    }
    let reject = value => {
      if (this.state === 'pending') {
        this.state = 'rejected'
        this.reason = value
      }
    }
    // 自动执行函数
    try {
      fn(resolve, reject)
    } catch (e) {
      reject(e)
    }
  }
  // then
  then(onFulfilled, onRejected) {
    switch (this.state) {
      case 'fulfilled':
        onFulfilled(this.value)
        break
      case 'rejected':
        onRejected(this.value)
        break
      default:
    }
  }
}
```

[回到顶部](#)

3.实现一个 call 函数

复制// 思路：将要改变this指向的方法挂到目标this上执行并返回

```
Function.prototype.mycall = function (context) {  
  if (typeof this !== 'function') {  
    throw new TypeError('not function')  
  }  
  context = context || window  
  context.fn = this  
  let arg = [...arguments].slice(1)  
  let result = context.fn(...arg)  
  delete context.fn  
  return result  
}
```

[回到顶部](#)

4.实现一个 apply 函数

复制// 思路：将要改变this指向的方法挂到目标this上执行并返回

```
Function.prototype.myapply = function (context) {  
  if (typeof this !== 'function') {  
    throw new TypeError('not function')  
  }  
  context = context || window  
  context.fn = this  
  let result  
  if (arguments[1]) {  
    result = context.fn(...arguments[1])  
  } else {  
    result = context.fn()  
  }  
  delete context.fn  
  return result  
}
```

[回到顶部](#)

5.实现一个 bind 函数

复制// 思路：类似call，但返回的是函数

```
Function.prototype.mybind = function (context) {  
  if (typeof this !== 'function') {  
    throw new TypeError('Error')  
  }  
  let _this = this  
  let arg = [...arguments].slice(1)  
  return function F() {  
    // 处理函数使用new的情况  
    if (this instanceof F) {  
      return new _this(...arg, ...arguments)  
    } else {  
      return _this.apply(context, arg.concat(...arguments))  
    }  
  }  
}
```

```
}
```

更多介绍: [bind方法的实现](#)

[回到顶部](#)

6.浅拷贝、深拷贝的实现

浅拷贝:

```
复制// 1. ...实现
let copy1 = {...{x:1}}

// 2. Object.assign实现
let copy2 = Object.assign({}, {x:1})
```

深拷贝:

```
复制// 1. JSON.stringify()/JSON.parse()
// 缺点: 拷贝对象包含 正则表达式, 函数, 或者undefined等值会失败
let obj = {a: 1, b: {x: 3}}
JSON.parse(JSON.stringify(obj))

// 2. 递归拷贝
function deepClone(obj) {
  let copy = obj instanceof Array ? [] : {}
  for (let i in obj) {
    if (obj.hasOwnProperty(i)) {
      copy[i] = typeof obj[i] === 'object' ? deepClone(obj[i]) : obj[i]
    }
  }
  return copy
}
```

[回到顶部](#)

7.实现一个节流函数

```
复制// 思路: 在规定时间内只触发一次
function throttle (fn, delay) {
  // 利用闭包保存时间
  let prev = Date.now()
  return function () {
    let context = this
    let arg = arguments
    let now = Date.now()
    if (now - prev >= delay) {
      fn.apply(context, arg)
      prev = Date.now()
    }
  }
}

function fn () {
  console.log('节流')
}
```

```
addEventListener('scroll', throttle(fn, 1000))
```

[回到顶部](#)

8.实现一个防抖函数

```
复制// 思路:在规定时间内未触发第二次, 则执行
function debounce (fn, delay) {
  // 利用闭包保存定时器
  let timer = null
  return function () {
    let context = this
    let arg = arguments
    // 在规定时间内再次触发会先清除定时器后再重设定时器
    clearTimeout(timer)
    timer = setTimeout(function () {
      fn.apply(context, arg)
    }, delay)
  }
}

function fn () {
  console.log('防抖')
}
addEventListener('scroll', debounce(fn, 1000))
```

[回到顶部](#)

9.instanceof 的原理

```
复制// 思路: 右边变量的原型存在于左边变量的原型链上
function instanceof(left, right) {
  let leftValue = left.__proto__
  let rightValue = right.prototype
  while (true) {
    if (leftValue === null) {
      return false
    }
    if (leftValue === rightValue) {
      return true
    }
    leftValue = leftValue.__proto__
  }
}
```

[回到顶部](#)

10.柯里化函数的实现

柯里化函数的定义：**将多参数的函数转换成单参数的形式。**

柯里化函数实现的原理：利用闭包原理在执行可以形成一个不销毁的作用域，然后把需要预先处理的内容都储存在这个不销毁的作用域中，并且返回一个最少参数函数。

第一种：固定传入参数，参数够了才执行

复制/**

* 实现要点：柯里化函数接收到足够参数后，就会执行原函数，那么我们如何去确定何时达到足够的参数呢？

* 柯里化函数需要记住你已经给过他的参数，如果没给的话，则默认为一个空数组。

* 接下来每次调用的时候，需要检查参数是否给够，如果够了，则执行**fn**，没有的话则返回一个新的 **curry** 函数，将现有的参数塞给他。

*

*/

// 待柯里化处理的函数

```
let sum = (a, b, c, d) => {  
  return a + b + c + d  
}
```

// 柯里化函数，返回一个被处理过的函数

```
let curry = (fn, ...arr) => { // arr 记录已有参数  
  return (...args) => { // args 接收新参数  
    if (fn.length <= (...arr,...args)) { // 参数够时，触发执行  
      return fn(...arr, ...args)  
    } else { // 继续添加参数  
      return curry(fn, [...arr, ...args])  
    }  
  }  
}
```

```
var sumPlus = curry(sum)  
sumPlus(1)(2)(3)(4)  
sumPlus(1, 2)(3)(4)  
sumPlus(1, 2, 3)(4)
```

第二种：不固定传入参数，随时执行

复制/**

* 当然了，柯里化函数的主要作用还是延迟执行，执行的触发条件不一定是参数个数相等，也可以是其他的条件。

* 例如参数个为0的情况，那么我们需要对上面**curry**函数稍微做修改

*/

// 待柯里化处理的函数

```
let sum = arr => {  
  return arr.reduce((a, b) => {  
    return a + b  
  })  
}
```

```
let curry = (fn, ...arr) => { // arr 记录已有参数  
  return (...args) => { // args 接收新参数  
    if (args.length === 0) { // 参数为空时，触发执行  
      return fn(...arr, ...args)  
    } else { // 继续添加参数  
      return curry(fn, ...arr, ...args)  
    }  
  }  
}
```

```
var sumPlus = curry(sum)  
sumPlus(1)(2)(3)(4)()  
sumPlus(1, 2)(3)(4)()  
sumPlus(1, 2, 3)(4)()
```

参考链接: [js如何用一句代码实现函数的柯里化 \(ES6\)](#)

[回到顶部](#)

11.Object.create 的基本实现原理

```
复制// 思路: 将传入的对象作为原型
function create(obj) {
  function F() {}
  F.prototype = obj
  return new F()
}
```

[回到顶部](#)

12.实现一个基本的 Event Bus

```
复制// 组件通信, 一个触发与监听的过程
class EventEmitter {
  constructor () {
    // 存储事件
    this.events = this.events || new Map()
  }
  // 监听事件
  addListener (type, fn) {
    if (!this.events.get(type)) {
      this.events.set(type, fn)
    }
  }
  // 触发事件
  emit (type) {
    let handle = this.events.get(type)
    handle.apply(this, [...arguments].slice(1))
  }
}

// 测试
let emitter = new EventEmitter()
// 监听事件
emitter.addListener('ages', age => {
  console.log(age)
})
// 触发事件
emitter.emit('ages', 18) // 18
```

[回到顶部](#)

13.实现一个双向数据绑定

```
复制let obj = {}
let input = document.getElementById('input')
let span = document.getElementById('span')
// 数据劫持
Object.defineProperty(obj, 'text', {
  configurable: true,
```

```

enumerable: true,
get() {
  console.log('获取数据了')
},
set(newVal) {
  console.log('数据更新了')
  input.value = newVal
  span.innerHTML = newVal
}
})
// 输入监听
input.addEventListener('keyup', function(e) {
  obj.text = e.target.value
})

```

详细的实现见: [这应该是最详细的响应式系统讲解了](#)

[回到顶部](#)

14.实现一个简单路由

```

复制// hash路由
class Route{
  constructor(){
    // 路由存储对象
    this.routes = {}
    // 当前hash
    this.currentHash = ''
    // 绑定this, 避免监听时this指向改变
    this.freshRoute = this.freshRoute.bind(this)
    // 监听
    window.addEventListener('load', this.freshRoute, false)
    window.addEventListener('hashchange', this.freshRoute, false)
  }
  // 存储
  storeRoute (path, cb) {
    this.routes[path] = cb || function () {}
  }
  // 更新
  freshRoute () {
    this.currentHash = location.hash.slice(1) || '/'
    this.routes[this.currentHash]()
  }
}

```

[回到顶部](#)

15.实现懒加载

```

复制<ul>
  <li></li>
  <li></li>
  <li></li>
  <li></li>
  <li></li>
  <li></li>

```

```

<li></li>
<li></li>
<li></li>
<li></li>
</ul>
复制let imgs = document.querySelectorAll('img')
// 可视区高度
let clientHeight = window.innerHeight || document.documentElement.clientHeight ||
document.body.clientHeight
function lazyLoad () {
  // 滚动卷去的高度
  let scrollTop = window.pageYOffset || document.documentElement.scrollTop ||
document.body.scrollTop
  for (let i = 0; i < imgs.length; i++) {
    // 图片在可视区冒出的高度
    let x = clientHeight + scrollTop - imgs[i].offsetTop
    // 图片在可视区内
    if (x > 0 && x < clientHeight+imgs[i].height) {
      imgs[i].src = imgs[i].getAttribute('data')
    }
  }
}
// addEventListener('scroll', lazyLoad) or setInterval(lazyLoad, 1000)

```

[回到顶部](#)

16.rem 基本设置

```

复制// 提前执行，初始化 resize 事件不会执行
setRem()
// 原始配置
function setRem () {
  let doc = document.documentElement
  let width = doc.getBoundingClientRect().width
  let rem = width / 75
  doc.style.fontSize = rem + 'px'
}
// 监听窗口变化
addEventListener("resize", setRem)

```

[回到顶部](#)

17.手写实现 AJAX

```

复制// 1. 简单流程

// 实例化
let xhr = new XMLHttpRequest()
// 初始化
xhr.open(method, url, async)
// 发送请求
xhr.send(data)
// 设置状态变化回调处理请求结果
xhr.onreadystatechange = () => {
  if (xhr.readyState === 4 && xhr.status === 200) {
    console.log(xhr.responseText)
  }
}

```

```

    }
  }

// 2. 基于promise实现

function ajax (options) {
  // 请求地址
  const url = options.url
  // 请求方法
  const method = options.method.toLocaleLowerCase() || 'get'
  // 默认为异步true
  const async = options.async
  // 请求参数
  const data = options.data
  // 实例化
  const xhr = new XMLHttpRequest()
  // 请求超时
  if (options.timeout && options.timeout > 0) {
    xhr.timeout = options.timeout
  }
  // 返回一个Promise实例
  return new Promise ((resolve, reject) => {
    xhr.ontimeout = () => reject && reject('请求超时')
    // 监听状态变化回调
    xhr.onreadystatechange = () => {
      if (xhr.readyState == 4) {
        // 200-300 之间表示请求成功, 304资源未变, 取缓存
        if (xhr.status >= 200 && xhr.status < 300 || xhr.status == 304) {
          resolve && resolve(xhr.responseText)
        } else {
          reject && reject()
        }
      }
    }
    // 错误回调
    xhr.onerror = err => reject && reject(err)
    let paramArr = []
    let encodeData
    // 处理请求参数
    if (data instanceof Object) {
      for (let key in data) {
        // 参数拼接需要通过 encodeURIComponent 进行编码
        paramArr.push(encodeURIComponent(key) + '=' +
encodeURIComponent(data[key]))
      }
      encodeData = paramArr.join('&')
    }
    // get请求拼接参数
    if (method === 'get') {
      // 检测url中是否已存在 ? 及其位置
      const index = url.indexOf('?')
      if (index === -1) url += '?'
      else if (index !== url.length -1) url += '&'
      // 拼接url
      url += encodeData
    }
    // 初始化
    xhr.open(method, url, async)
  })
}

```



```

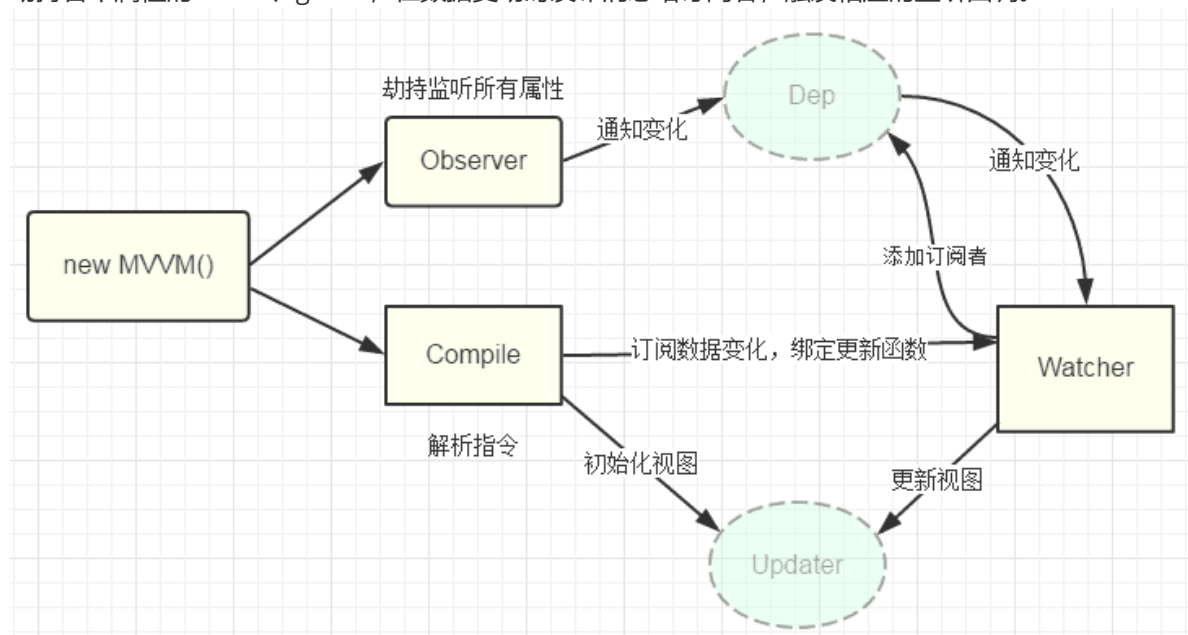
// 发送请求
if (method === 'get') xhr.send(null)
else {
  // post 方式需要设置请求头
  xhr.setRequestHeader('Content-Type','application/x-www-form-
urlencoded;charset=UTF-8')
  xhr.send(encodeData)
}
})
}

```

第六章 Vue面试题

1.Vue 双向绑定原理

mvvm 双向绑定，采用数据劫持结合发布者-订阅者模式的方式，通过 `Object.defineProperty()` 来劫持各个属性的 setter、getter，在数据变动时发布消息给订阅者，触发相应的监听回调。



几个要点:

- 1、实现一个数据监听器 Observer，能够对数据对象的所有属性进行监听，如有变动可拿到最新值并通知订阅者
- 2、实现一个指令解析器 Compile，对每个元素节点的指令进行扫描和解析，根据指令模板替换数据，以及绑定相应的更新函数
- 3、实现一个 Watcher，作为连接 Observer 和 Compile 的桥梁，能够订阅并收到每个属性变动的通知，执行指令绑定的相应回调函数，从而更新视图
- 4、mvvm 入口函数，整合以上三者

具体步骤:

1. 需要 observe 的数据对象进行递归遍历，包括子属性对象的属性，都加上 setter 和 getter
这样的话，给这个对象的某个值赋值，就会触发 setter，那么就能监听到了数据变化
2. compile 解析模板指令，将模板中的变量替换成数据，然后初始化渲染页面视图，并将每个指令对应的节点绑定更新函数，添加监听数据的订阅者，一旦数据有变动，收到通知，更新视图
3. Watcher 订阅者是 Observer 和 Compile 之间通信的桥梁，主要做的事情是：
 - 在自身实例化时往属性订阅器(dep)里面添加自己
 - 自身必须有一个 update() 方法

- 待属性变动 dep.notice() 通知时，能调用自身的 update() 方法，并触发 Compile 中绑定的回调，则功成身退。
4. MVVM 作为数据绑定的入口，整合 Observer、Compile 和 Watcher 三者，通过 Observer 来监听自己的 model 数据变化，通过 Compile 来解析编译模板指令，最终利用 Watcher 搭起 Observer 和 Compile 之间的通信桥梁，达到数据变化 -> 视图更新；视图交互变化(input) -> 数据 model 变更的双向绑定效果。

[回到顶部](#)

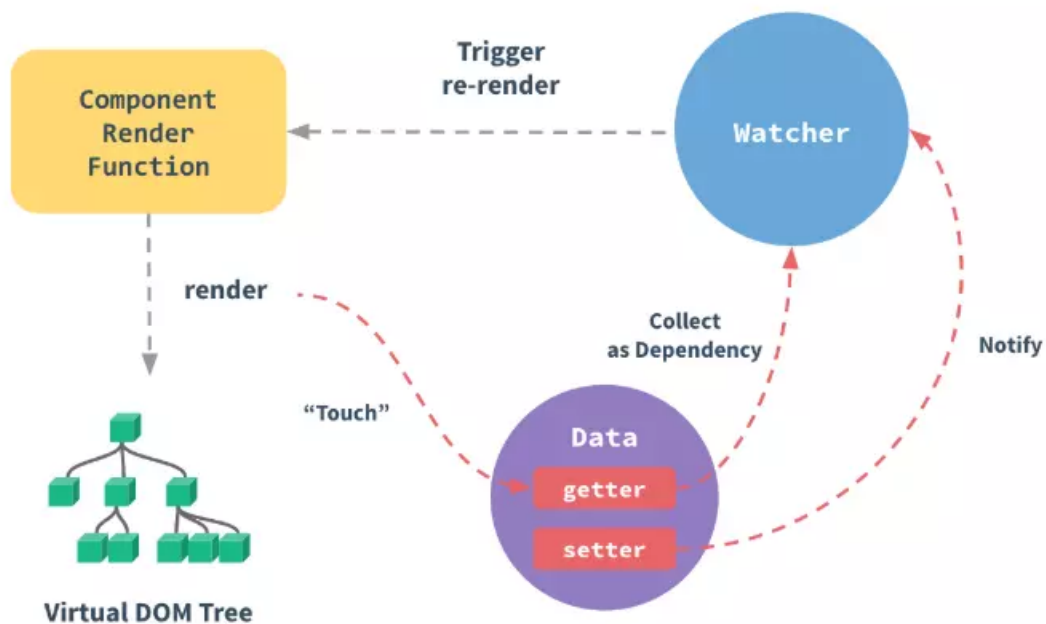
2.描述下 vue 从初始化页面--修改数据--刷新页面 UI 的过程？

当 Vue 进入初始化阶段时，一方面 Vue 会遍历 data 中的属性，并用 Object.defineProperty 将它转化成 getter/setter 的形式，实现数据劫持(暂不谈 Vue3.0 的 Proxy)；另一方面，Vue 的指令编译器 Compiler 对元素节点的各个指令进行解析，初始化视图，并订阅 Watcher 来更新视图，此时 Watcher 会将自己添加到消息订阅器 Dep 中，此时初始化完毕。

当数据发生变化时，触发 Observer 中 setter 方法，立即调用 Dep.notify(), Dep 这个数组开始遍历所有的订阅者，并调用其 update 方法，Vue 内部再通过 diff 算法，patch 相应的更新完成对订阅者视图的改变。

[回到顶部](#)

3.你是如何理解 Vue 的响应式系统的？



响应式系统简述:

- 任何一个 Vue Component 都有一个与之对应的 Watcher 实例
- Vue 的 data 上的属性会被添加 getter 和 setter 属性
- 当 Vue Component render 函数被执行的时候, data 上会被 触碰(touch), 即被读, getter 方法会被调用, 此时 Vue 会去记录此 Vue component 所依赖的所有 data。(这一过程被称为依赖收集)
- data 被改动时 (主要是用户操作), 即被写, setter 方法会被调用, 此时 Vue 会去通知所有依赖于此 data 的组件去调用他们的 render 函数进行更新

[回到顶部](#)

4.虚拟 DOM 实现原理

- 虚拟DOM本质上是JavaScript对象,是对真实DOM的抽象
- 状态变更时, 记录新树和旧树的差异
- 最后把差异更新到真正的dom中

详细实现见 [面试官: 你对虚拟DOM原理的理解?](#)

[回到顶部](#)

5.既然 Vue 通过数据劫持可以精准探测数据变化,为什么还需要虚拟 DOM 进行 diff 检测差异?

考点: Vue 的变化侦测原理

前置知识: 依赖收集、虚拟 DOM、响应式系统

现代前端框架有两种方式侦测变化,一种是pull,一种是push

pull: 其代表为React, 我们可以回忆一下React是如何侦测到变化的,我们通常会用setStateAPI显式更新, 然后React会进行一层层的Virtual Dom Diff操作找出差异, 然后Patch到DOM上, React从一开始就不知道到底是哪发生了变化, 只是知道「有变化了」, 然后再进行比较暴力的Diff操作查找「哪发生了变化了」, 另外一个代表就是Angular的脏检查操作。

push: Vue的响应式系统则是push的代表, 当Vue程序初始化的时候就会对数据data进行依赖的收集, 一旦数据发生变化,响应式系统就会立刻得知。因此Vue是一开始就知道是「在哪发生了变化了」, 但是这又会产生一个问题, 如果你熟悉Vue的响应式系统就知道, 通常一个绑定一个数据就需要一个Watcher, 一旦我们的绑定细粒度过高就会产生大量的Watcher, 这会带来内存以及依赖追踪的开销, 而细粒度过低会无法精准侦测变化,因此Vue的设计是选择中等细粒度的方案,在组件级别进行push侦测的方式,也就是那套响应式系统,通常我们会第一时间侦测到发生变化的组件,然后在组件内部进行Virtual Dom Diff获取更加具体的差异, 而Virtual Dom Diff则是pull操作, Vue是push+pull结合的方式进行变化侦测的。

[Vue和React的视图更新机制对比](#)

[回到顶部](#)

6.Vue 中 key 值的作用?

当 Vue.js 用 v-for 正在更新已渲染过的元素列表时, 它默认用“**就地复用**”策略。如果数据项的顺序被改变, Vue 将不会移动 DOM 元素来匹配数据项的顺序, 而是简单复用此处每个元素, 并且确保它在特定索引下显示已被渲染过的每个元素。**key** 的作用主要是为了高效的更新虚拟DOM。

[回到顶部](#)

7.Vue 的生命周期

[vue生命周期详解](#)

1. beforeCreate 和 created
2. beforeMount 和 mounted
3. beforeUpdate 和 updated
4. beforeDestory 和 destoryed
5. activated 和 deactivated

[回到顶部](#)

8.Vue 组件间通信有哪些方式?

[Vue 组件间通信六种方式](#)

1. props/\$emit
2. \$emit/\$on
3. vuex
4. \$attrs/\$listeners
5. provide/inject
6. \$parent/\$children 与 ref

[回到顶部](#)

9.watch、methods 和 computed 的区别?

- watch 为了监听某个响应数据的变化。computed 是自动**监听依赖值**的变化，从而动态返回内容，主要目的是简化模板内的复杂运算。所以区别来源于用法，只是需要动态值，那就用 computed；需要知道值的改变后执行业务逻辑，才用 watch。
- methods是一个方法，它可以接受参数，而computed 不能，computed 是可以缓存的，methods 不会。computed 可以依赖其他 computed，甚至是其他组件的 data。

[回到顶部](#)

10.vue 中怎么重置 data?

使用Object.assign(), vm.\$data可以获取当前状态下的data, vm.\$options.data(this)可以获取到组件初始化状态下的data。

```
复制Object.assign(this.$data, this.$options.data(this)) // 注意加this，不然取不到
data() { a: this.methodA } 中的this.methodA。
```

原因参考: [Vue中的this.\\$options.data\(\)和this.\\$data](#)

[回到顶部](#)

11.组件中写 name 选项有什么作用?

1. 项目使用 keep-alive 时，可搭配组件 name 进行缓存过滤
2. DOM 做递归组件时需要调用自身 name
3. vue-devtools 调试工具里显示的组见名称是由vue中组件name决定的

[回到顶部](#)

12.vue-router 有哪些钩子函数?

官方文档: [vue-router钩子函数](#)

- 全局前置守卫 router.beforeEach
- 全局解析守卫 router.beforeResolve
- 全局后置钩子 router.afterEach
- 路由独享的守卫 beforeEnter
- 组件内的守卫 beforeRouteEnter、beforeRouteUpdate、beforeRouteLeave

[前端路由简介以及vue-router实现原理](#)

[回到顶部](#)

13.route 和 router 的区别是什么？

`route` 是“路由信息对象”，包括 `path`, `params`, `hash`, `query`, `fullPath`, `matched`, `name` 等路由信息参数。

`router` 是“路由实例对象”，包括了路由的跳转方法(`push`、`replace`)，钩子函数等。

[回到顶部](#)

14.说一下 Vue 和 React 的认识，做一个简单的对比

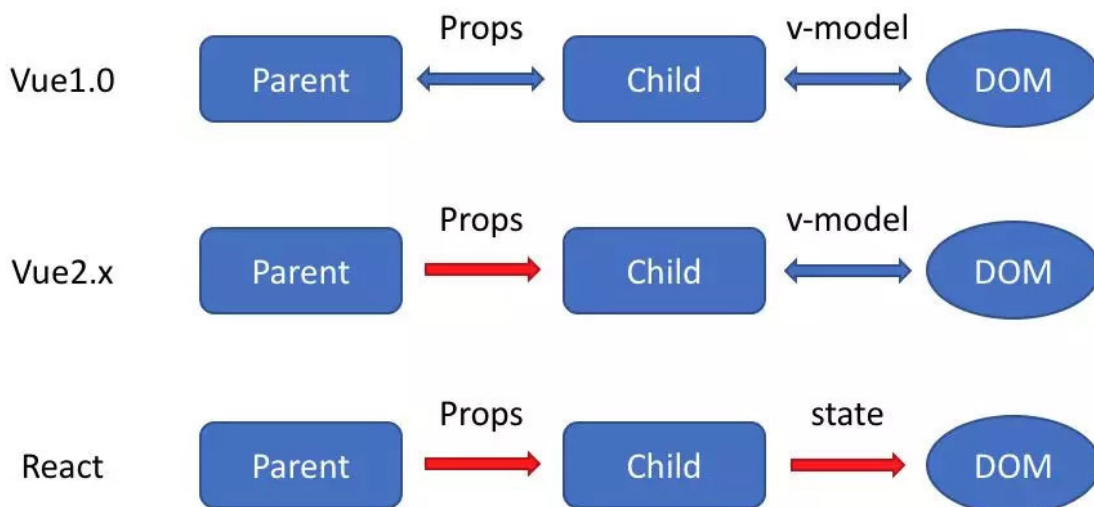
1.监听数据变化的实现原理不同

- Vue 通过 `getter/setter` 以及一些函数的劫持，能精确快速的计算出 Virtual DOM 的差异。这是由于它在渲染过程中，会跟踪每一个组件的依赖关系，不需要重新渲染整个组件树。
- React 默认是通过比较引用的方式进行，如果不优化，每当应用的状态被改变时，全部子组件都会重新渲染，可能导致大量不必要的 VDOM 的重新渲染。

Vue 不需要特别的优化就能达到很好的性能，而对于 React 而言，需要通过 `PureComponent/shouldComponentUpdate` 这个生命周期方法来进行控制。如果你的应用中，交互复杂，需要处理大量的 UI 变化，那么使用 Virtual DOM 是一个好主意。如果你更新元素并不频繁，那么 Virtual DOM 并不一定适用，性能很可能还不如直接操控 DOM。

为什么 React 不精确监听数据变化呢？这是因为 Vue 和 React 设计理念上的区别，Vue 使用的是可变数据，而 React 更强调数据的不可变。

2.数据流的不同



https://blog.csdn.net/Tokki_

- Vue 中默认支持双向绑定，组件与 DOM 之间可以通过 `v-model` 双向绑定。但是，父子组件之间，`props` 在 2.x 版本是单向数据流
- React 一直提倡的是单向数据流，他称之为 `onChange/setState()` 模式。

不过由于我们一般都会用 `Vuex` 以及 `Redux` 等单向数据流的状态管理框架，因此很多时候我们感受不到这一点的区别了。

3.模板渲染方式的不同

在表层上，模板的语法不同

- React 是通过 `JSX` 渲染模板
- 而 Vue 是通过一种拓展的 `HTML` 语法进行渲染

在深层上，模板的原理不同，这才是他们的本质区别：

- React 是在组件 JS 代码中，通过原生 JS 实现模板中的常见语法，比如插值，条件，循环等，都是通过 JS 语法实现的
- Vue 是在和组件 JS 代码分离的单独的模板中，通过指令来实现的，比如条件语句就需要 v-if 来实现

对这一点，我个人比较喜欢 React 的做法，因为他更加纯粹更加原生，而 Vue 的做法显得有些独特，会把 HTML 弄得很乱。举个例子，说明 React 的好处：react 中 render 函数是支持闭包特性的，所以我们 import 的组件在 render 中可以直接调用。但是在 Vue 中，由于模板中使用的数据都必须挂在 this 上进行一次中转，所以我们 import 一个组件完了之后，还需要在 components 中再声明下，这样显然是很奇怪但又不得不这样的做法。

[回到顶部](#)

15.Vue 的 nextTick 的原理是什么？

1. 为什么需要 nextTick

Vue 是异步修改 DOM 的并且不鼓励开发者直接接触 DOM，但有时候业务需要必须对数据更改--刷新后的 DOM 做相应的处理，这时候就可以使用 Vue.nextTick(callback)这个 api 了。

2. 理解原理前的准备

首先需要知道事件循环中宏任务和微任务这两个概念(这其实也是面试常考点)。请阅大佬文章--[彻底搞懂浏览器 Event-loop](#)

常见的宏任务有 script, setTimeout, setInterval, setImmediate, I/O, UI rendering

常见的微任务有 process.nextTick(Nodejs), Promise.then(), MutationObserver;

3. 理解 nextTick

而 nextTick 的原理正是 vue 通过异步队列控制 DOM 更新和 nextTick 回调函数先后执行的方式。如果大家看过这部分的源码，会发现其中做了很多 isNative()的判断，因为这里还存在兼容性优雅降级的问题。可见 Vue 开发团队的深思熟虑，对性能的良苦用心。

如果你比较了解了前面的事件循环原理，推荐你看看这篇文章 请阅大佬文章--[全面解析 Vue.nextTick 实现原理](#)

[回到顶部](#)

16.Vuex 有哪几种属性？

有五种，分别是 State、Getter、Mutation、Action、Module

[回到顶部](#)

17.vue 首屏加载优化

1. 把不常改变的库放到 index.html 中，通过 cdn 引入

```
<link href="https://cdn.bootcss.com/font-awesome/4.7.0/css/font-awesome.css" rel="stylesheet">
<link rel="stylesheet" href="https://cdn.bootcss.com/element-ui/2.4.0/theme-chalk/index.css">

<body>
  <div id="app"> Loading... </div>

  <!-- built files will be auto injected -->
  <script src="https://cdn.bootcss.com/vue/2.5.16/vue.min.js"></script>
  <script src="https://cdn.bootcss.com/vue-router/3.0.1/vue-router.min.js"></script>
  <script src="https://cdn.bootcss.com/element-ui/2.4.0/index.js"></script>
</body>
```

然后找到 build/webpack.base.conf.js 文件，在 module.exports = { } 中添加以下代码


```
复制externals: {  
  'vue': 'vue',  
  'vue-router': 'VueRouter',  
  'element-ui': 'ELEMENT',  
},
```

这样 webpack 就不会把 vue.js, vue-router, element-ui 库打包了。声明一下，我把 main.js 中对 element 的引入删掉了，不然我发现打包后的 app.css 还是会把 element 的 css 打包进去，删掉后就没了。

然后你打包就会发现 vendor 文件小了很多~

2. vue 路由的懒加载

`import` 或者 `require` 懒加载。你打包就会发现，多了很多 1.xxxxx.js; 2.xxxxx.js 等等，而 vendor.xxx.js 没了，剩下 app.js 和 manifest.js，而且 app.js 还很小，我这里是 100k 多一点。

3. 不生成 map 文件

找到 config/index.js，修改为 `productionSourceMap: false`

4. vue 组件尽量不要全局引入

5. 使用更轻量级的工具库

6. 开启gzip压缩

这个优化是两方面的，前端将文件打包成.gz文件，然后通过nginx的配置，让浏览器直接解析.gz文件。

7. 首页单独做服务端渲染

如果首页真的有瓶颈，可以考虑用 node 单独做服务端渲染，而下面的子页面仍用 spa 单页的方式交互。

这里不推荐直接用 nuxt.js 服务端渲染方案，因为这样一来增加了学习成本，二来服务端的维护成本也会上升，有时在本机测试没问题，在服务端跑就有问题，为了省心，还是最大限度的使用静态页面较好。

参考链接：

[vue首屏加载优化](#)

[vue项目首屏加载优化实战](#)

[回到顶部](#)

18.Vue 3.0 有没有过了了解？

关于Vue 3.0有幸看过尤大的关于3.0版本的[RFC Vue Function-based API RFC](#)。大致说了三个点，第一个是关于提出的新API `setup()` 函数，第二个说了对于Typescript的支持，最后说了关于替换 `Object.defineProperty` 为 Proxy 的支持。

详细说了下关于Proxy代替带来的性能上的提升，因为传统的原型链拦截的方法，无法检测对象及数组的一些更新操作，但使用Proxy又带来了浏览器兼容问题。

[回到顶部](#)

19.vue-cli 替我们做了哪些工作？

首先需要知道 vue-cli 是什么？它是基于 Vue.js 进行快速开发的完整系统，也可以理解成是很多 npm 包的集合。其次，vue-cli 完成的功能有哪些？

- .vue 文件 --> .js 文件
- ES6 语法 --> ES5 语法
- Sass, Less, Stylus --> CSS
- 对 jpg, png, font 等静态资源的处理
- 热更新
- 定义环境变量，区分 dev 和 production 模式
- ...

如果开发者需要补充或修改默认设置，需要在 package.json 同级下新建一个 vue.config.js 文件

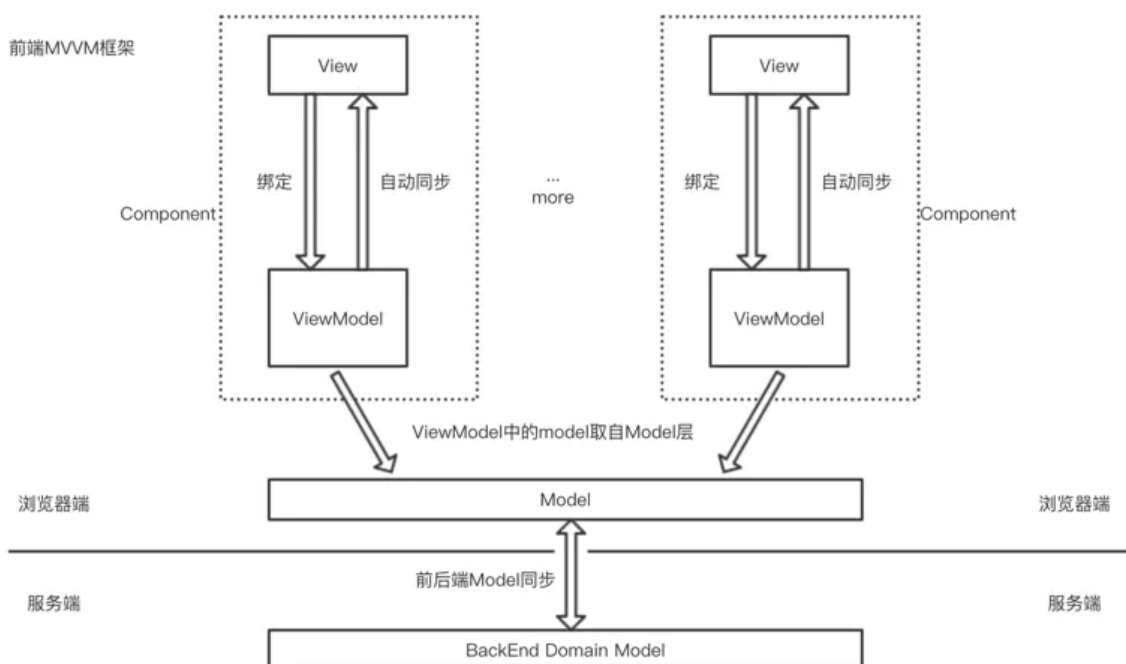
20.MVVM是什么？

MVVM 模式，顾名思义即 Model-View-ViewModel 模式。它萌芽于2005年微软推出的基于 Windows 的用户界面框架 WPF，前端最早的 MVVM 框架 knockout 在2010年发布。

Model 层: 对应数据层的域模型，它主要做域模型的同步。通过 Ajax/fetch 等 API 完成客户端和服务端业务 Model 的同步。在层间关系里，它主要用于抽象出 ViewModel 中视图的 Model。

View 层: 作为视图模板存在，在 MVVM 里，整个 View 是一个动态模板。除了定义结构、布局外，它展示的是 ViewModel 层的数据和状态。View 层不负责处理状态，View 层做的是数据绑定的声明、指令的声明、事件绑定的声明。

ViewModel 层: 把 View 需要的层数据暴露，并对 View 层的数据绑定声明、指令声明、事件绑定声明负责，也就是处理 View 层的具体业务逻辑。ViewModel 底层会做好绑定属性的监听。当 ViewModel 中数据变化，View 层会得到更新；而当 View 中声明了数据的双向绑定（通常是表单元素），框架也会监听 View 层（表单）值的变化。一旦值变化，View 层绑定的 ViewModel 中的数据也会得到自动更新。



21.MVVM的优缺点？

优点:

1. 分离视图 (View) 和模型 (Model) ,降低代码耦合, 提高视图或者逻辑的重用性: 比如视图 (View) 可以独立于Model变化和修改, 一个ViewModel可以绑定不同的"View"上, 当View变化的时候Model不可以不变, 当Model变化的时候View也可以不变。你可以把一些视图逻辑放在一个ViewModel里面, 让很多view重用这段视图逻辑
2. 提高可测试性: ViewModel的存在可以帮助开发者更好地编写测试代码
3. 自动更新dom: 利用双向绑定,数据更新后视图自动更新,让开发者从繁琐的手动dom中解放

缺点:

1. Bug很难被调试: 因为使用双向绑定的模式, 当你看到界面异常了, 有可能是你View的代码有Bug, 也可能是Model的代码有问题。数据绑定使得一个位置的Bug被快速传递到别的位置, 要定位原始出问题的地方就变得不那么容易了。另外, 数据绑定的声明是指令式地写在View的模版当中的, 这些内容是没办法去打断点debug的
2. 一个大的模块中model也会很大, 虽然使用方便了也很容易保证了数据的一致性, 当时长期持有, 不释放内存就造成了花费更多的内存
3. 对于大型的图形应用程序, 视图状态较多, ViewModel的构建和维护的成本都会比较高

22.你对Vue生命周期的理解？

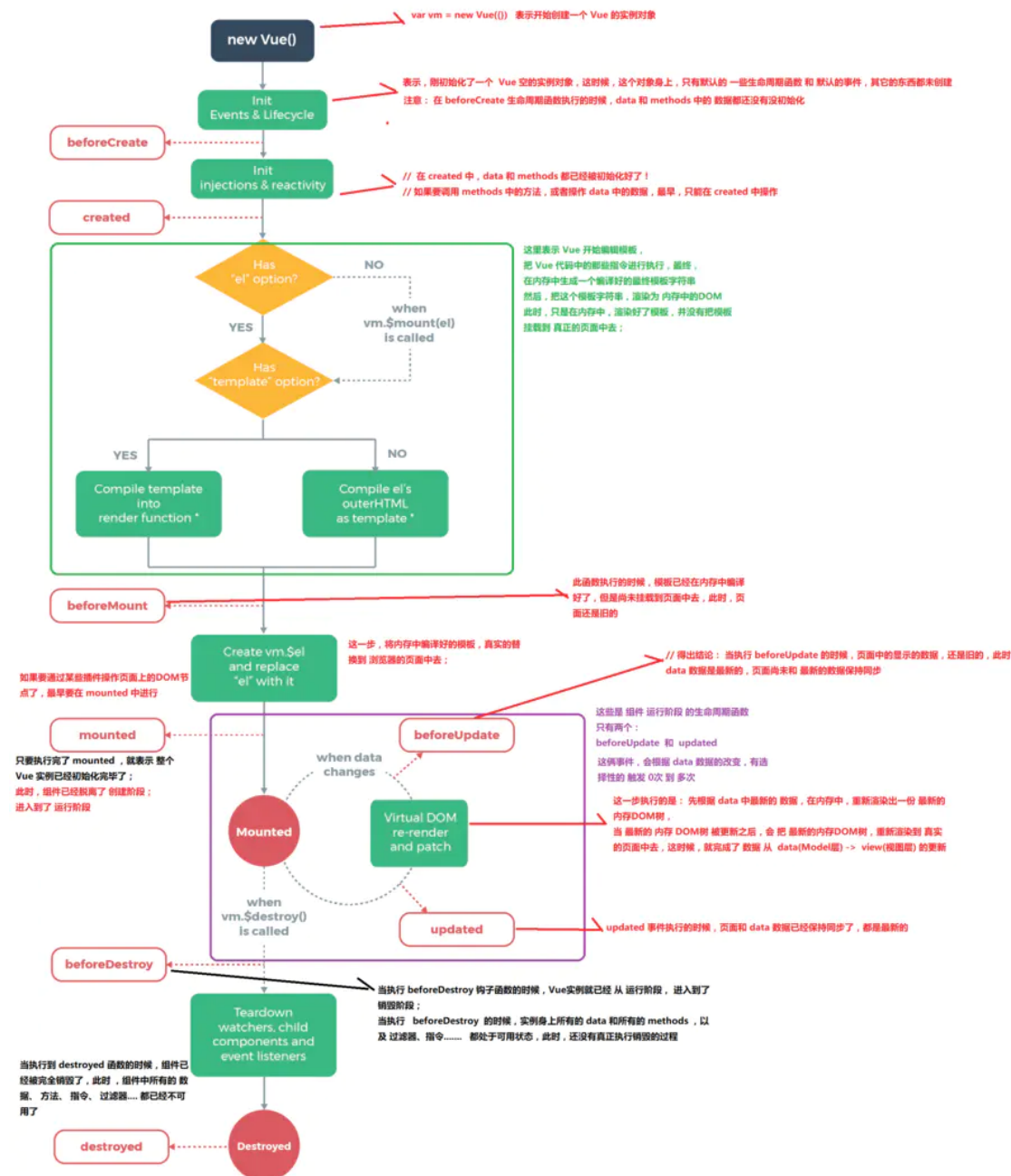
生命周期是什么

Vue 实例有一个完整的生命周期, 也就是从开始创建、初始化数据、编译模版、挂载Dom -> 渲染、更新 -> 渲染、卸载等一系列过程, 我们称这是Vue的生命周期。

各个生命周期的作用

生命周期	描述
beforeCreate	组件实例被创建之初, 组件的属性生效之前
created	组件实例已经完全创建, 属性也绑定, 但真实dom还没有生成, <code>\$el</code> 还不可用
beforeMount	在挂载开始之前被调用: 相关的 <code>render</code> 函数首次被调用
mounted	<code>el</code> 被新创建的 <code>vm.\$el</code> 替换, 并挂载到实例上去之后调用该钩子
beforeUpdate	组件数据更新之前调用, 发生在虚拟 DOM 打补丁之前
update	组件数据更新之后
activated	keep-alive专属, 组件被激活时调用
deactivated	keep-alive专属, 组件被销毁时调用
beforeDestory	组件销毁前调用
destoryed	组件销毁后调用

生命周期示意图



23.异步请求适合在哪个生命周期调用？

官方实例的异步请求是在 mounted 生命周期中调用的，而实际上也可以在 created 生命周期中调用。

24.Vue组件如何通信？

Vue组件通信的方法如下：

- **props/\$emit+v-on**：通过 props 将数据自上而下传递，而通过 \$emit 和 v-on 来向上传递信息。
- **EventBus**：通过 EventBus 进行信息的发布与订阅
- **vuex**：是全局数据管理库，可以通过 vuex 管理全局的数据流
- **\$attrs/\$listeners**：Vue 2.4 中加入的 \$attrs/\$listeners 可以进行跨级的组件通信
- **provide/inject**：以允许一个祖先组件向其所有子孙后代注入一个依赖，不论组件层次有多深，并在起上下游关系成立的时间里始终生效，这成为了跨组件通信的基础

还有一些用solt插槽或者ref实例进行通信的，使用场景过于有限就不赘述了。

25.computed和watch有什么区别？

computed:

1. `computed` 是计算属性,也就是计算值,它更多用于计算值的场景
2. `computed` 具有缓存性,computed的值在getter执行后是会缓存的,只有在它依赖的属性值改变之后,下一次获取computed的值时才会重新调用对应的getter来计算
3. `computed` 适用于计算比较消耗性能的计算场景

watch:

1. 更多的是「观察」的作用,类似于某些数据的监听回调,用于观察 `props` `$emit` 或者本组件的值,当数据变化时来执行回调进行后续操作
2. 无缓存性, 页面重新渲染时值不变化也会执行

小结:

1. 当我们要进行数值计算,而且依赖于其他数据, 那么把这个数据设计为computed
2. 如果你需要在某个数据变化时做一些事情, 使用watch来观察这个数据变化

26.Vue是如何实现双向绑定的？

利用 `Object.defineProperty` 劫持对象的访问器,在属性值发生变化时我们可以获取变化,然后根据变化进行后续响应,在vue3.0中通过Proxy代理对象进行类似的操作。

```
// 这是将要被劫持的对象
const data = {
  name: '',
};

function say(name) {
  if (name === '古天乐') {
    console.log('给大家推荐一款超好玩的游戏');
  } else if (name === '渣渣辉') {
    console.log('戏我演过很多,可游戏我只玩贪玩懒月');
  } else {
    console.log('来做我的兄弟');
  }
}

// 遍历对象,对其属性值进行劫持
Object.keys(data).forEach(function(key) {
  Object.defineProperty(data, key, {
    enumerable: true,
    configurable: true,
    get: function() {
      console.log('get');
    },
    set: function(newVal) {
      // 当属性值发生变化时我们可以进行额外操作
      console.log(`大家好,我系${newVal}`);
      say(newVal);
    },
  });
});
```

```
data.name = '渣渣辉';  
//大家好,我系渣渣辉  
//戏我演过很多,可游戏我只玩贪玩懒月  
复制代码
```

详细实现见[Proxy比defineproperty优劣对比?](#)

27.Proxy与Object.defineProperty的优劣对比?

Proxy的优势如下:

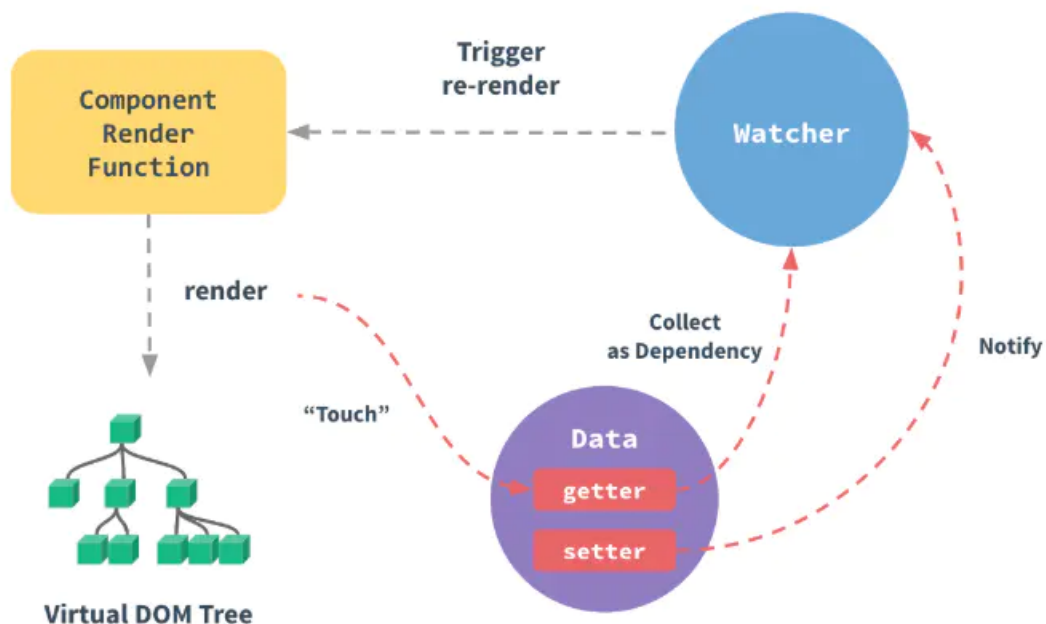
- Proxy可以直接监听对象而非属性
- Proxy可以直接监听数组的变化
- Proxy有多达13种拦截方法,不限于apply、ownKeys、deleteProperty、has等等是 `Object.defineProperty` 不具备的
- Proxy返回的是一个新对象,我们可以只操作新的对象达到目的,而 `Object.defineProperty` 只能遍历对象属性直接修改
- Proxy作为新标准将受到浏览器厂商重点持续的性能优化, 也就是传说中的新标准的性能红利

Object.defineProperty的优势如下:

- 兼容性好,支持IE9

详细实现见[Proxy比defineproperty优劣对比?](#)

28.你是如何理解Vue的响应式系统的?



响应式系统简述:

- 任何一个 Vue Component 都有一个与之对应的 Watcher 实例。
- Vue 的 data 上的属性会被添加 getter 和 setter 属性。
- 当 Vue Component render 函数被执行的时候, data 上会被 触碰(touch), 即被读, getter 方法会被调用, 此时 Vue 会去记录此 Vue component 所依赖的所有 data。(这一过程被称为依赖收集)

- data 被改动时（主要是用户操作），即被写，setter 方法会被调用，此时 Vue 会去通知所有依赖于此 data 的组件去调用他们的 render 函数进行更新。

29.虚拟DOM的优劣如何？

优点：

- 保证性能下限：虚拟DOM可以经过diff找出最小差异，然后批量进行patch，这种操作虽然比不上手动优化，但是比起粗暴的DOM操作性能要好很多，因此虚拟DOM可以保证性能下限
- 无需手动操作DOM：虚拟DOM的diff和patch都是在一次更新中自动进行的，我们无需手动操作DOM，极大提高开发效率
- 跨平台：虚拟DOM本质上是JavaScript对象，而DOM与平台强相关，相比之下虚拟DOM可以进行更方便地跨平台操作，例如服务器渲染、移动端开发等等

缺点：

- 无法进行极致优化：在一些性能要求极高的应用中虚拟DOM无法进行针对性的极致优化，比如VScode采用直接手动操作DOM的方式进行极端的性能优化

30.虚拟DOM实现原理？

- 虚拟DOM本质上是JavaScript对象，是对真实DOM的抽象
- 状态变更时，记录新树和旧树的差异
- 最后把差异更新到真正的dom中

详细实现见[虚拟DOM原理？](#)

31.既然Vue通过数据劫持可以精准探测数据变化,为什么还需要虚拟DOM进行diff检测差异？

考点：Vue的变化侦测原理

前置知识：依赖收集、虚拟DOM、响应式系统

现代前端框架有两种方式侦测变化，一种是pull一种是push

pull：其代表为React，我们可以回忆一下React是如何侦测到变化的，我们通常会用 `setState` API显式更新，然后React会进行一层层的Virtual Dom Diff操作找出差异，然后Patch到DOM上，React从一开始就不知道到底是哪发生了变化，只是知道「有变化了」，然后再进行比较暴力的Diff操作查找「哪发生了变化了」，另外一个代表就是Angular的脏检查操作。

push：Vue的响应式系统则是push的代表，当Vue程序初始化的时候就会对数据data进行依赖的收集，一旦数据发生变化，响应式系统就会立刻得知，因此Vue是一开始就知道是「在哪发生了变化了」，但是这又会产生一个问题，如果你熟悉Vue的响应式系统就知道，通常一个绑定一个数据就需要一个Watcher，但我们的绑定细粒度过高就会产生大量的Watcher，这会带来内存以及依赖追踪的开销，而细粒度过低会无法精准侦测变化，因此Vue的设计是选择中等细粒度的方案，在组件级别进行push侦测的方式，也就是那套响应式系统，通常我们会第一时间侦测到发生变化的组件，然后在组件内部进行Virtual Dom Diff获取更加具体的差异，而Virtual Dom Diff则是pull操作，Vue是push+pull结合的方式进行变化侦测的。

32.Vue为什么没有类似于React中shouldComponentUpdate的生命周期？

考点：Vue的变化侦测原理

前置知识：依赖收集、虚拟DOM、响应式系统

根本原因是Vue与React的变化侦测方式有所不同

React是pull的方式侦测变化,当React知道发生变化后,会使用Virtual Dom Diff进行差异检测,但是很多组件实际上是肯定不会发生变化的,这个时候需要用shouldComponentUpdate进行手动操作来减少diff,从而提高程序整体的性能.

Vue是pull+push的方式侦测变化的,在一开始就知道那个组件发生了变化,因此在push的阶段并不需要手动控制diff,而组件内部采用的diff方式实际上是可以引入类似于shouldComponentUpdate相关生命周期的,但是通常合理大小的组件不会有过量的diff,手动优化的价值有限,因此目前Vue并没有考虑引入shouldComponentUpdate这种手动优化的生命周期.

33.Vue中的key到底有什么用?

key 是为Vue中的vnode标记的唯一id,通过这个key,我们的diff操作可以更准确、更快速

diff算法的过程中,先会进行新旧节点的首尾交叉对比,当无法匹配的时候会用新节点的 key 与旧节点进行比对,然后超出差异.

diff程可以概括为: oldCh和newCh各有两个头尾的变量StartIdx和EndIdx, 它们的2个变量相互比较, 一共有4种比较方式. 如果4种比较都没匹配, 如果设置了key, 就会用key进行比较, 在比较的过程中, 变量会往中间靠, 一旦StartIdx>EndIdx表明oldCh和newCh至少有一个已经遍历完了, 就会结束比较,这四种比较方式就是首、尾、旧尾新头、旧头新尾.

- 准确: 如果不加 key,那么vue会选择复用节点(Vue的就地更新策略),导致之前节点的状态被保留下来,会产生一系列的bug.
- 快速: key的唯一性可以被Map数据结构充分利用,相比于遍历查找的时间复杂度O(n),Map的时间复杂度仅仅为O(1).

```
60 function createKeyToOldIdx (children, beginIdx, endIdx) {
61   let i, key
62   const map = {}
63   for (i = beginIdx; i <= endIdx; ++i) {
64     key = children[i].key
65     if (isDef(key)) map[key] = i
66   }
67   return map
68 }
```

第七章 HTTP、浏览器面试题

1.浏览器存储的方式有哪些

特性	cookie	localStorage	sessionStorage	indexedDB
数据生命周期	一般由服务器生成, 可以设置过期时间	除非被清理, 否则一直存在	页面关闭就清理	除非被清理, 否则一直存在
数据存储空间	4K	5M	5M	无限
与服务端通信	每次都会携带在 header, 中, 对于请求性能影响	不参与	不参与	不参与

补充：cookie 原本并不是用来储存的，而是用来与服务端通信的，需要存取请自行封装 api。而 localStorage 则自带 getItem 和 setItem 方法，使用很方便。

localStorage 注意点：

1. localStorage 只能存字符串，存取 JSON 数据需配合 JSON.stringify() 和 JSON.parse()
2. 遇上禁用 setItem 的浏览器，需要使用 try...catch 捕获异常

[回到顶部](#)

2.对前后端跨域可以说一下吗？如何解决跨域的？

[九种跨域方式实现原理](#)

如何跨域我们已经明白了，如果这样问：浏览器没有同源策略会有什么危险？是不是有点瞬间懵逼？下面是摘选的事例，参考：[AJAX跨域访问被禁止的原因](#)

假设有一个黑客叫做小黑，他从网上抓取了一堆美女图做了一个网站，每日访问量爆表。为了维护网站运行，小黑挂了一张收款码，觉得网站不错的可以适当资助一点，可是无奈伸手党太多，小黑的网站入不敷出。于是他非常生气的在网页中写了一段js代码，使用ajax向淘宝发起登陆请求，因为很多数人都访问过淘宝，所以电脑中存有淘宝的cookie，不需要输入账号密码直接就自动登录了，然后小黑在ajax回调函数中解析了淘宝返回的数据，得到了很多人的隐私信息，转手一卖，小黑的网站终于盈利了。如果跨域也可以发送AJAX请求的话，小黑就真的获取到了用户的隐私并成功获利了！！

[回到顶部](#)

3.浏览器 cookie 和 session 的认识。

session 是基于 cookie 实现的。cookie 保存在客户端浏览器中，而 session 保存在服务器上。cookie 机制是通过检查客户身上的“通行证”来确定客户身份的话，那么 session 机制就是通过检查服务器上的“客户明细表”来确认客户身份。session 相当于程序在服务器上建立的一份客户档案，客户来访的时候只需要查询客户档案表就可以了。

cookie 和 session 的区别：

1. 存在的位置：
cookie 存在于客户端，临时文件夹中；session 存在于服务器的内存中，一个 session 域对象为一个用户浏览器服务
2. 安全性
cookie 是以明文的方式存放在客户端的，安全性低，可以通过一个加密算法进行加密后存放；session 存放于服务器的内存中，所以安全性好
3. 生命周期(以 20 分钟为例)
cookie 的生命周期是累计的，从创建时，就开始计时，20 分钟后 cookie 生命周期结束；session 的生命周期是间隔的，从创建时，开始计时如在 20 分钟，没有访问 session，那么 session 生命周期被销毁。但是，如果在 20 分钟内（如在第 19 分钟时）访问过 session，那么，将重新计算 session 的生命周期。关机造成 session 生命周期的结束，但是对 cookie 没有影响
4. 访问范围
cookie 为多个用户浏览器共享；session 为一个用户浏览器独享

好文推荐：[彻底理解 cookie、session、token](#)

[回到顶部](#)

4.输入URL发生什么？

1. DNS 域名解析（域名解析成ip地址，走UDP协议，因此不会有握手过程）：浏览器将 URL 解析出相对应的服务器的 IP 地址（1. 本地浏览器的 DNS 缓存中查找 2. 再向系统DNS缓存发送查询请求 3. 再向路由器DNS缓存 4. 网络运营商DNS缓存 5. 递归搜索），并从 url 中解析出端口号
2. 浏览器与目标服务器建立一条 TCP 连接（三次握手）
3. 浏览器向服务器发送一条 HTTP 请求报文
4. 服务器返回给浏览器一条 HTTP 响应报文
5. 浏览器进行渲染
6. 关闭 TCP 连接（四次挥手）

[回到顶部](#)

5.浏览器渲染的步骤

1. HTML 解析出 DOM Tree
2. CSS 解析出 Style Rules
3. 两者关联生成 Render Tree
4. Layout（布局）根据 Render Tree 计算每个节点的信息
5. Painting 根据计算好的信息进行渲染整个页面

浏览器解析文档的过程中，如果遇到 script 标签，会立即解析脚本，停止解析文档（因为 JS 可能会改变 DOM 和 CSS,如果继续解析会造成浪费）。

如果是外部 script, 会等待脚本下载完成之后再继续解析文档。现在 script 标签增加了 defer 和 async 属性，脚本解析会将脚本中改变 DOM 和 css 的地方> 解析出来，追加到 DOM Tree 和 Style Rules 上

[回到顶部](#)

6.页面渲染优化

基于对渲染过程的了解，推荐一下优化：

1. HTML 文档结构层次尽量少，最好不深于 6 层
2. 脚本尽量放后边，避免组织页面加载
3. 少量首屏样式可以放在便签内
4. 样式结构层次尽量简单
5. 脚本减少 DOM 操作，减少回流，尽量缓存访问 DOM 的样式信息
6. 尽量减少 JS 修改样式，可以通过修改 class 名的方式解决
7. 减少 DOM 查找，缓存 DOM 查找结果
8. 动画在屏幕外或页面滚动时，尽量停止

[回到顶部](#)

7.强制缓存和协商缓存

- 强制缓存是我们在第一次请求资源时在 http 响应头设置一个过期时间，在时效内都将直接从浏览器进行获取，常见的 http 响应头字段如 Cache-Control 和 Expires
- 协商缓存是我们通过 http 响应头字段 etag 或者 Last-Modified 等判断服务器上资源是否修改，如果修改则从服务器重新获取，如果未修改则 304 指向浏览器缓存中进行获取

[回到顶部](#)

8.GET 和 POST 请求的区别

- GET 参数通过 url 传递, POST 放在 body 中。(http 协议规定, url 在请求头中, 所以大小限制很小)
- GET 请求在 url 中传递的参数是有长度限制的, 而 POST 没有。原因见上↑↑↑
- GET 在浏览器回退时是无害的, 而 POST 会再次提交请求
- GET 请求会被浏览器主动 cache, 而 POST 不会, 除非手动设置
- GET 比 POST 更不安全, 因为参数直接暴露在 url 中, 所以不能用来传递敏感信息
- 对参数的数据类型, GET 只接受 ASCII 字符, 而 POST 没有限制
- GET 请求只能进行 url(x-www-form-urlencoded)编码, 而 POST 支持多种编码方式
- **GET 产生一个 TCP 数据包; POST 产生两个 TCP 数据包。**对于 GET 方式的请求, 浏览器会把 http 的 header 和 data 一并发送出去, 服务器响应 200 (返回数据)。而对于 POST, 浏览器先发送 header, 服务器响应 100 continue, 浏览器再发送 data, 服务器响应 200 ok (返回数据)

[回到顶部](#)

9.HTTP1.0 / 1.1 / 2.0 及HTTPS

[你需要知道的HTTP常识](#)

[可能是全网最全的http面试答案](#)

[如何优雅的谈论HTTP / 1.0 / 1.1 / 2.0](#)

HTTP1.1 是当前使用最为广泛的HTTP协议

- HTTP1.0 和 HTTP1.1 相比
 - HTTP1.0 定义了三种请求方法: GET, POST 和 HEAD 方法。HTTP1.1 新增了六种请求方法: OPTIONS、PUT、PATCH、DELETE、TRACE 和 CONNECT 方法。
 - 缓存处理: 在HTTP1.0中主要使用header里的If-Modified-Since, Expires来做为缓存判断的标准, HTTP1.1则引入了更多的缓存控制策略例如Entity tag, If-Unmodified-Since, If-Match, If-None-Match等更多可供选择的缓存头来控制缓存策略。
 - 带宽优化及网络连接的使用: HTTP1.0中, 存在一些浪费带宽的现象, 例如客户端只是需要某个对象的一部分, 而服务器却将整个对象送过来了, 并且不支持断点续传功能, HTTP1.1则在请求头引入了range头域, 它允许只请求资源的某个部分, 即返回码是206 (Partial Content), 这样就方便了开发者自由的选择以便于充分利用带宽和连接。
 - 错误通知的管理: 在HTTP1.1中新增了24个错误状态响应码, 如409 (Conflict) 表示请求的资源与资源的当前状态发生冲突; 410 (Gone) 表示服务器上的某个资源被永久性的删除。
 - Host头处理: 在HTTP1.0中认为每台服务器都绑定一个唯一的IP地址, 因此, 请求消息中的URL并没有传递主机名 (hostname)。但随着虚拟主机技术的发展, 在一台物理服务器上可以存在多个虚拟主机 (Multi-homed Web Servers), 并且它们共享一个IP地址。HTTP1.1的请求消息和响应消息都应支持Host头域, 且请求消息中如果没有Host头域会报告一个错误 (400 Bad Request)。
 - 长连接: HTTP 1.1支持长连接 (Persistent Connection) 和请求的流水线 (Pipelining) 处理, 在一个TCP连接上可以传送多个HTTP请求和响应, 减少了建立和关闭连接的消耗和延迟, 在HTTP1.1中默认开启Connection: keep-alive, 一定程度上弥补了HTTP1.0每次请求都要创建连接的缺点。通过设置http的请求头部和应答头部, 保证本次数据请求结束之后, 下一次请求仍可以重用这一通道, 避免重新握手。
- HTTP2.0 和 HTTP1.X 相比
 - 新的二进制格式 (Binary Format): HTTP1.x的解析是基于文本。基于文本协议的格式解析存在天然缺陷, 文本的表现形式有多样性, 要做到健壮性考虑的场景必然很多, 二进制则不同, 只认0和1的组合。基于这种考虑HTTP2.0的协议解析决定采用二进制格式, 实现方便且健壮。
 - 多路复用 (MultiPlexing): 即连接共享, 即每一个request都是是用作连接共享机制的。一个request对应一个id, 这样一个连接上可以有多个request, 每个连接的request可以随机的

混杂在一起，接收方可以根据request的 id将request再归属到各自不同的服务端请求里面。

- header压缩：如上文中所言，对前面提到过HTTP1.x的header带有大量信息，而且每次都要重复发送，HTTP2.0使用了专门为首部压缩而设计的 HPACK 算法，使用encoder来减少需要传输的header大小，通讯双方各自cache一份header fields表，既避免了重复header的传输，又减小了需要传输的大小。
- 服务端推送（server push）：服务端推送能把客户端所需要的资源伴随着index.html一起发送到客户端，省去了客户端重复请求的步骤。正因为没有发起请求，建立连接等操作，所以静态资源通过服务端推送的方式可以极大地提升速度。例如我的网页有一个style.css的请求，在客户端收到style.css数据的同时，服务端会将style.js的文件推送给客户端，当客户端再次尝试获取style.js时就可以直接从缓存中获取到，不用再发请求了。
- HTTPS 与 HTTP 相比
 - HTTPS协议需要到CA申请证书，一般免费证书很少，需要交费。
 - HTTP协议运行在TCP之上，所有传输的内容都是明文，HTTPS运行在SSL/TLS之上，SSL/TLS运行在TCP之上，所有传输的内容都经过加密的。
 - HTTP和HTTPS使用的是完全不同的连接方式，用的端口也不一样，前者是80，后者是443。
 - HTTPS可以有效的防止运营商劫持，解决了防劫持的一个大问题。

HTTPS 介绍：HTTPS在传输数据之前需要客户端（浏览器）与服务端（网站）之间进行一次握手，在握手过程中将确立双方加密传输数据的密码信息。TLS/SSL协议不仅仅是一套加密传输的协议，TLS/SSL中使用了非对称加密，对称加密以及HASH算法。

握手过程的简单描述如下：

- 1.浏览器将自己支持的一套加密规则发送给网站。
- 2.网站从中选出一组加密算法与HASH算法，并将自己的身份信息以证书的形式发回给浏览器。证书里面包含了网站地址，加密公钥，以及证书的颁发机构等信息。
- 3.获得网站证书之后浏览器要做以下工作：
 - a. 验证证书的合法性（颁发证书的机构是否合法，证书中包含的网站地址是否与正在访问的地址一致等），如果证书受信任，则浏览器栏里面会显示一个小锁头，否则会给出证书不受信的提示。
 - b. 如果证书受信任，或者是用户接受了不受信的证书，浏览器会生成一串随机数的密码，并用证书中提供的公钥加密。
 - c. 使用约定好的HASH计算握手消息，并使用生成的随机数对消息进行加密，最后将之前生成的所有信息发送给网站。
- 4.网站接收浏览器发来的数据之后要做以下的操作：
 - a. 使用自己的私钥将信息解密取出密码，使用密码解密浏览器发来的握手消息，并验证HASH是否与浏览器发来的一致。
 - b. 使用密码加密一段握手消息，发送给浏览器。
- 5.浏览器解密并计算握手消息的HASH，如果与服务端发来的HASH一致，此时握手过程结束，之后所有的通信数据将由之前浏览器生成的随机密码并利用对称加密算法进行加密。

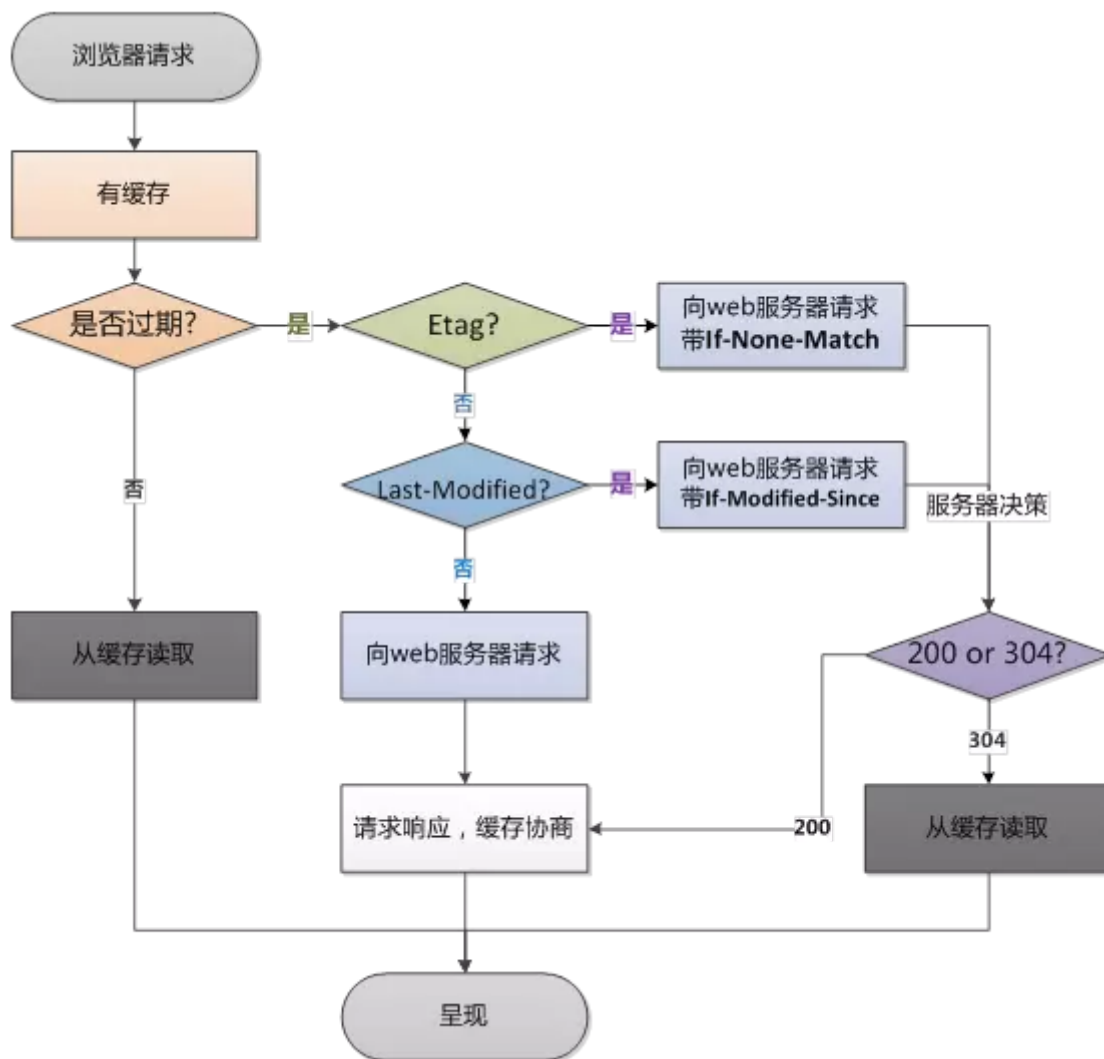
这里浏览器与网站互相发送加密的握手消息并验证，目的是为了保证双方都获得了一致的密码，并且可以正常的加密解密数据。其中非对称加密算法用于在握手过程中加密生成的密码，对称加密算法用于对真正传输的数据进行加密，而HASH算法用于验证数据的完整性。由于浏览器生成的密码是整个数据加密的关键，因此在传输的时候使用了非对称加密算法对其加密。非对称加密算法会生成公钥和私钥，公钥只能用于加密数据，因此可以随意传输，而网站的私钥用于对数据进行解密，所以网站都会非常小心的保管自己的私钥，防止泄漏。TLS握手过程中如果有任何错误，都会使加密连接断开，从而阻止了隐私信息的传输。正是由于HTTPS非常的安全，攻击者无法从中找到下手的地方，于是更多的是采用了假证书的手法来欺骗客户端，从而获取明文的信息。



[回到顶部](#)

10.介绍下304过程

- 浏览器请求资源时首先命中资源的Expires 和 Cache-Control, Expires 受限与本地时间, 如果修改了本地时间, 可能会造成缓存失效, 可以通过Cache-control: max-age指定最大生命周期, 状态仍然返回200, 但不会请求数据, 在浏览器中能明显看到from cache字样。
- 强缓存失效, 进入协商缓存阶段, 首先验证ETagETag可以保证每一个资源是唯一的, 资源变化都会导致ETag变化。服务器根据客户端上送的If-None-Match值来判断是否命中缓存。
- 协商缓存Last-Modify/If-Modify-Since阶段, 客户端第一次请求资源时, 服务端返回的header中会加上Last-Modify, Last-modify是一个时间标识该资源的最后修改时间。再次请求该资源时, request的请求头中会包含If-Modify-Since, 该值为缓存之前返回的Last-Modify。服务器收到If-Modify-Since后, 根据资源的最后修改时间判断是否命中缓存。



[回到顶部](#)

11.HTTP 状态码

- 1xx (临时响应) 表示临时响应并需要请求者继续执行操作的状态码
 - 100 - 继续 请求者应当继续提出请求。服务器返回此代码表示已收到请求的第一部分，正在等待其余部分
 - 101 - 切换协议 请求者已要求服务器切换协议，服务器已确认并准备切换
- 2xx (成功) 表示成功处理了请求的状态码
 - 200 - 成功 服务器已经成功处理了请求。通常，这表示服务器提供了请求的网页
 - 201 - 已创建 请求成功并且服务器创建了新的资源
 - 202 - 已接受 服务器已接受请求，但尚未处理
 - 203 - 非授权信息 服务器已经成功处理了请求，但返回的信息可能来自另一来源
 - 204 - 无内容 服务器成功处理了请求，但没有返回任何内容
 - 205 - 重置内容 服务器成功处理了请求，但没有返回任何内容
 - 206 - 部分内容 服务器成功处理了部分GET请求
- 3xx (重定向) 表示要完成请求，需要进一步操作；通常，这些状态代码用来重定向
 - 300 - 多种选择 针对请求，服务器可执行多种操作。服务器可根据请求者（user agent）选择一项操作，或提供操作列表供请求者选择
 - 301 - 永久移动 请求的网页已永久移动到新位置。服务器返回此响应（对GET或HEAD请求的响应）时，会自动将请求者转到新位置
 - 302 - 临时移动 服务器目前从不同位置的网页响应请求，但请求者应继续使用原有位置来进行以后的请求

- 303 - 查看其它位置 请求者应当对不同的位置使用单独的GET请求来检索响应时，服务器返回此代码
- 304 - 未修改 自上次请求后，请求的网页未修改过。服务器返回此响应，不会返回网页的内容
- 305 - 使用代理 请求者只能使用代理访问请求的网页。如果服务器返回此响应，还表示请求者应使用代理
- 307 - 临时性重定向 服务器目前从不同位置的网页响应请求，但请求者应继续使用原有的位置来进行以后的请求
- 4xx (请求错误) 这些状态码表示请求可能出错，妨碍了服务器的处理
 - 400 - 错误请求 服务器不理解请求的语法
 - 401 - 未授权 请求要求身份验证。对于需要登录的网页，服务器可能返回此响应
 - 403 - 禁止 服务器拒绝请求
 - 404 - 未找到 服务器找不到请求的网页
 - 405 - 方法禁用 禁用请求中指定的方法
 - 406 - 不接受 无法使用请求的内容特性响应请求的网页
 - 407 - 需要代理授权 此状态码与401（未授权）类似，但指定请求者应当授权使用代理
 - 408 - 请求超时 服务器等候请求时发生超时
 - 409 - 冲突 服务器在完成请求时发生冲突。服务器必须在响应中包含有关冲突的信息
 - 410 - 已删除 如果请求的资源已永久删除，服务器就会返回此响应
 - 411 - 需要有效长度 服务器不接受不含有效内容长度标头字段的请求
 - 412 - 未满足前提条件 服务器未满足请求者在请求中设置的其中一个前提条件
 - 413 - 请求实体过大 服务器无法处理请求，因为请求实体过大，超出了服务器的处理能力
 - 414 - 请求的URI过长 请求的URI（通常为网址）过长，服务器无法处理
 - 415 - 不支持媒体类型 请求的格式不受请求页面的支持
 - 416 - 请求范围不符合要求 如果页面无法提供请求的范围，则服务器会返回此状态码
 - 417 - 未满足期望值 服务器未满足“期望”请求标头字段的要求
- 5xx (服务器错误) 这些状态码表示服务器在尝试处理请求时发生内部错误。这些错误可能是服务器本身的错误，而不是请求出错
 - 500 - 服务器内部错误 服务器遇到错误，无法完成请求
 - 501 - 尚未实施 服务器不具备完成请求的功能。例如，服务器无法识别请求方法时可能会返回此代码
 - 502 - 错误网关 服务器作为网关或代理，从上游服务器无法收到无效响应
 - 503 - 服务器不可用 服务器目前无法使用（由于超载或者停机维护）。通常，这只是暂时状态
 - 504 - 网关超时 服务器作为网关代理，但是没有及时从上游服务器收到请求
 - 505 - HTTP版本不受支持 服务器不支持请求中所用的HTTP协议版本

[回到顶部](#)

12.Web性能优化



第八章 数据结构和算法

一.链表

1.简单的反转链表

反转一个单链表。

示例:

输入: 1->2->3->4->5->NULL
输出: 5->4->3->2->1->NULL

&循环解决方案

这道题是链表中的经典题目，充分体现链表这种数据结构 操作思路简单，但是 实现上 并没有那么简单的特点。

那在实现上应该注意一些什么问题呢？

保存后续节点。作为新手来说，很容易将当前节点的 `next` 指针直接指向前一个节点，但其实当前节点 `下一个节点` 的指针也就丢失了。因此，需要在遍历的过程当中，先将下一个节点保存，然后再操作 `next` 指向。

链表结构声定义如下:

```
function ListNode(val) {  
  this.val = val;  
  this.next = null;  
}
```

实现如下:


```

/**
 * @param {ListNode} head
 * @return {ListNode}
 */
let reverseList = (head) => {
  if (!head)
    return null;
  let pre = null, cur = head;
  while (cur) {
    // 关键：保存下一个节点的值
    let next = cur.next;
    cur.next = pre;
    pre = cur;
    cur = next;
  }
  return pre;
};

```

由于逻辑比较简单，代码直接一气呵成。不过仅仅写完还不够，对于链表问题，[边界检查](#)的习惯能帮助我们进一步保证代码的质量。具体来说：

- 当 head 节点为空时，我们已经处理，通过 ☒
- 当链表只包含 [一个节点](#) 时，此时我们希望直接返回这个节点，对上述代码而言，进入循环后 pre 被赋值为 cur，也就是 head，没毛病，通过 ☒

运行在 LeetCode, 成功 AC 

但作为系统性的训练而言，单单让程序通过未免太草率了，我们后续会尽可能地用不同的方式去解决相同的问题，达到[融会贯通](#)的效果，也是对自己思路的开拓，有时候或许能达到[更优解](#)。

递归解决方案

由于之前的思路已经介绍得非常清楚了，因此在这我们贴上代码，大家好好体会：

```

let reverseList = (head) => {
  let reverse = (pre, cur) => {
    if (!cur) return pre;
    // 保存 next 节点
    let next = cur.next;
    cur.next = pre;
    return reverse(cur, next);
  }
  return reverse(null, head);
}

```

2. 区间反转

反转从位置 m 到 n 的链表。请使用一趟扫描完成反转。

说明: $1 \leq m \leq n \leq$ 链表长度。

示例:

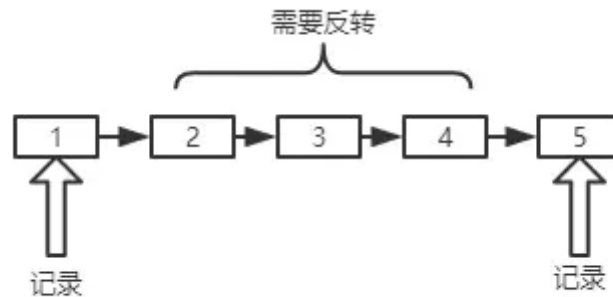
输入: 1->2->3->4->5->NULL, m = 2, n = 4

输出: 1->4->3->2->5->NULL

思路

这一题相比上一个整个链表反转的题, 其实是 **换汤不换药**。我们依然有两种类型的解法: **循环解法**和**递归解法**。

需要注意的问题就是 **前后节点** 的保存(或者记录), 什么意思呢? 看这张图你就明白了。



关于前节点和后节点的定义, 大家在图上应该能看的比较清楚了, 后面会经常用到。

反转操作上一题已经拆解过, 这里不再赘述。值得注意的是反转后的工作, 那么对于整个区间反转后的工作, 其实就是一个 **移花接木** 的过程, 首先将**前节点**的 next 指向区间终点, 然后将区间起点的 next 指向**后节点**。因此这一题中有四个需要重视的节点: **前节点**、**后节点**、**区间起点** 和 **区间终点**。接下来我们开始实际的编码操作。

循环解法

```
/**
 * @param {ListNode} head
 * @param {number} m
 * @param {number} n
 * @return {ListNode}
 */
var reverseBetween = function(head, m, n) {
    let count = n - m;
    let p = dummyHead = new ListNode();
    let pre, cur, start, tail;
    p.next = head;
    for(let i = 0; i < m - 1; i++) {
        p = p.next;
    }
    // 保存前节点
    front = p;
    // 同时保存区间首节点
    pre = tail = p.next;
    cur = pre.next;
    // 区间反转
    for(let i = 0; i < count; i++) {
        let next = cur.next;
        cur.next = pre;
        pre = cur;
    }
}
```



```

        cur = next;
    }
    // 前节点的 next 指向区间末尾
    front.next = pre;
    // 区间首节点的 next 指向后节点(循环完后的cur就是区间后面第一个节点，即后节点)
    tail.next = cur;
    return dummyHead.next;
};

```

递归解法

对于递归解法，唯一的不同就在于对于区间的处理，采用递归程序进行处理，大家也可以趁着复习一下递归反转的实现。

```

var reverseBetween = function(head, m, n) {
    // 递归反转函数
    let reverse = (pre, cur) => {
        if(!cur) return pre;
        // 保存 next 节点
        let next = cur.next;
        cur.next = pre;
        return reverse(cur, next);
    }
    let p = dummyHead = new ListNode();
    dummyHead.next = head;
    let start, end; //区间首尾节点
    let front, tail; //前节点和后节点
    for(let i = 0; i < m - 1; i++) {
        p = p.next;
    }
    front = p; //保存前节点
    start = front.next;
    for(let i = m - 1; i < n; i++) {
        p = p.next;
    }
    end = p;
    tail = end.next; //保存后节点
    end.next = null;
    // 开始穿针引线啦，前节点指向区间首，区间首指向后节点
    front.next = reverse(null, start);
    start.next = tail;
    return dummyHead.next;
}

```

3.两个一组翻转链表

给定一个链表，两两交换其中相邻的节点，并返回交换后的链表。

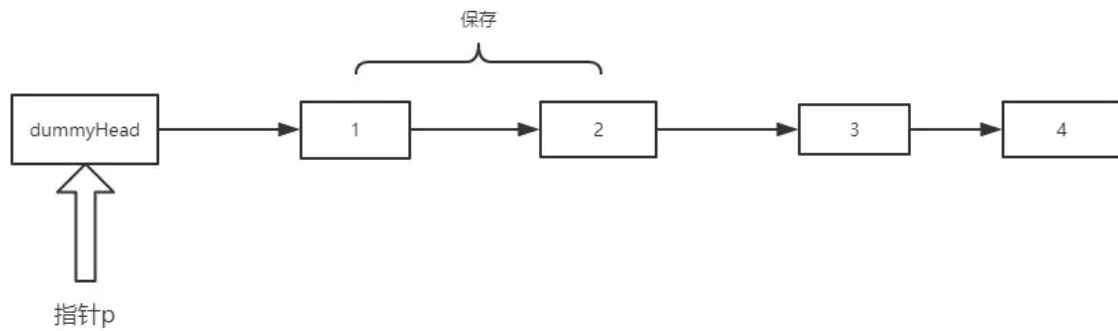
你不能只是单纯的改变节点内部的值，而是需要实际的进行节点交换。

示例:

给定 1->2->3->4，你应该返回 2->1->4->3。

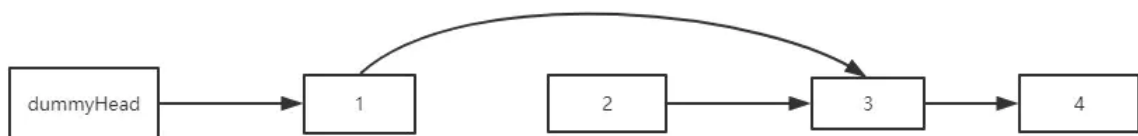
思路

如图所示，我们首先建立一个虚拟头节点(dummyHead)，辅助我们分析。

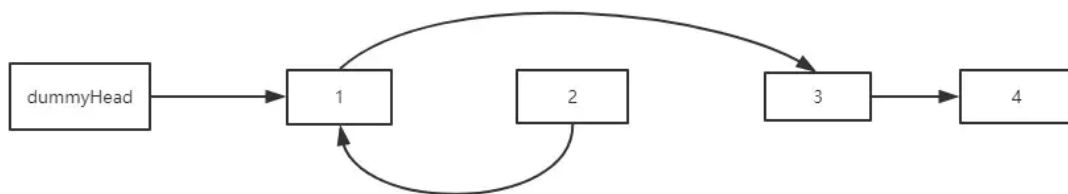


首先让 p 处在 dummyHead 的位置，记录下 p.next 和 p.next.next 的节点，也就是 node1 和 node2。

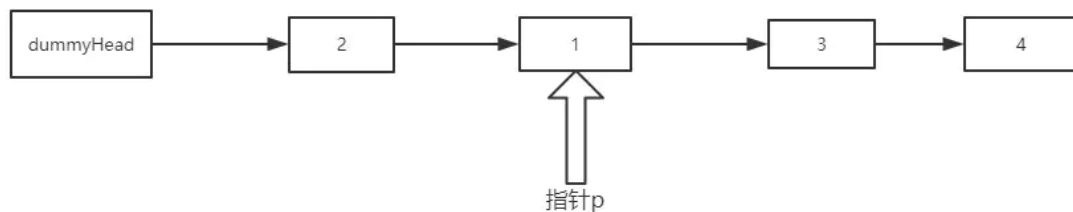
随后让 **node1.next = node2.next**, 效果:



然后让 **node2.next = node1**, 效果:



最后，**dummyHead.next = node2**，本次翻转完成。同时 p 指针指向node1, 效果如下:



依此循环，如果 **p.next** 或者 **p.next.next** 为空，也就是找不到新的一组节点了，循环结束。

循环解决

思路清楚了，其实实现还是比较容易的，代码如下:

```
var swapPairs = function(head) {  
  if(head == null || head.next == null)  
    return head;  
  let dummyHead = p = new ListNode();  
  let node1, node2;
```

```

dummyHead.next = head;
while((node1 = p.next) && (node2 = p.next.next)) {
    node1.next = node2.next;
    node2.next = node1;
    p.next = node2;
    p = node1;
}
return dummyHead.next;
};

```

递归方式

```

var swapPairs = function(head) {
    if(head == null || head.next == null)
        return head;
    let node1 = head, node2 = head.next;
    node1.next = swapPairs(node2.next);
    node2.next = node1;
    return node2;
};

```

利用递归方式之后，是不是感觉代码特别简洁？😊😊😊

希望你能好好体会一下递归调用的过程，相信理解之后对自己是一个很大的提升。

4.K个一组翻转链表

给你一个链表，每 k 个节点一组进行翻转，请你返回翻转后的链表。

k 是一个正整数，它的值小于或等于链表的长度。

如果节点总数不是 k 的整数倍，那么请将最后剩余的节点保持原有顺序。

示例：

```

给定这个链表：1->2->3->4->5
当 k = 2 时，应当返回：2->1->4->3->5
当 k = 3 时，应当返回：3->2->1->4->5

```

说明：

- 你的算法只能使用常数的额外空间。
- 你不能只是单纯的改变节点内部的值，而是需要实际的进行节点交换。

思路

思路类似**No.3**中的两个一组翻转。唯一的不同在于两个一组的情况下每一组只需要反转两个节点，而在 K 个一组的情况下对应的操作是将 **K** 个元素的链表进行反转。

递归解法

这一题我觉得递归的解法更容易理解，因此，先贴上递归方法的代码。

以下代码的注释中首节点、尾结点等概念都是针对反转前的链表而言的。

```
/**
 * @param {ListNode} head
 * @param {number} k
 * @return {ListNode}
 */
var reverseKGroup = function(head, k) {
    let pre = null, cur = head;
    let p = head;
    // 下面的循环用来检查后面的元素是否能组成一组
    for(let i = 0; i < k; i++) {
        if(p == null) return head;
        p = p.next;
    }
    for(let i = 0; i < k; i++){
        let next = cur.next;
        cur.next = pre;
        pre = cur;
        cur = next;
    }
    // pre为本组最后一个节点，cur为下一组的起点
    head.next = reverseKGroup(cur, k);
    return pre;
};
```

循环解法

重点都放在注释里面了。

```
var reverseKGroup = function(head, k) {
    let count = 0;
    // 看是否能构成一组，同时统计链表元素个数
    for(let p = head; p != null; p = p.next) {
        if(p == null && i < k) return head;
        count++;
    }
    let loopCount = Math.floor(count / k);
    let p = dummyHead = new ListNode();
    dummyHead.next = head;
    // 分成了 loopCount 组，对每一个组进行反转
    for(let i = 0; i < loopCount; i++) {
        let pre = null, cur = p.next;
        for(let j = 0; j < k; j++) {
            let next = cur.next;
            cur.next = pre;
            pre = cur;
            cur = next;
        }
        // 当前 pre 为该组的尾结点，cur 为下一组首节点
        let start = p.next; // start 是该组首节点
        // 开始穿针引线！思路和2个一组的情况一模一样
        p.next = pre;
        start.next = cur;
        p = start;
    }
}
```

```
    }  
    return dummyHead.next;  
};
```

5.如何检测链表形成环?

给定一个链表，判断链表中是否形成环。

思路

思路一: 循环一遍，用 Set 数据结构保存节点，利用节点的内存地址来进行判重，如果同样的节点走过两次，则表明已经形成了环。

思路二: 利用快慢指针，快指针一次走两步，慢指针一次走一步，如果 **两者相遇**，则表明已经形成了环。

可能你会纳闷，为什么思路二用两个指针在环中一定会相遇呢？

其实很简单，如果有环，两者一定同时走到环中，那么在环中，**选慢指针为参考系**，快指针每次 **相对参考系** 向前走一步，终究会绕回原点，也就是回到慢指针的位置，从而让两者相遇。如果没有环，则两者的相对距离越来越远，永远不会相遇。

接下来我们来编程实现。

方法一: Set 判重

```
/**  
 * @param {ListNode} head  
 * @return {boolean}  
 */  
var hasCycle = (head) => {  
    let set = new Set();  
    let p = head;  
    while(p) {  
        // 同一个节点再次碰到，表示有环  
        if(set.has(p)) return true;  
        set.add(p);  
        p = p.next;  
    }  
    return false;  
}
```

方法二: 快慢指针

```
var hasCycle = function(head) {  
    let dummyHead = new ListNode();  
    dummyHead.next = head;  
    let fast = slow = dummyHead;  
    // 零个结点或者一个结点，肯定无环  
    if(fast.next == null || fast.next.next == null)  
        return false;  
    while(fast && fast.next) {  
        fast = fast.next.next;  
        slow = slow.next;  
    }
```

```
// 两者相遇了
if(fast == slow) {
    return true;
}
return false;
};
```

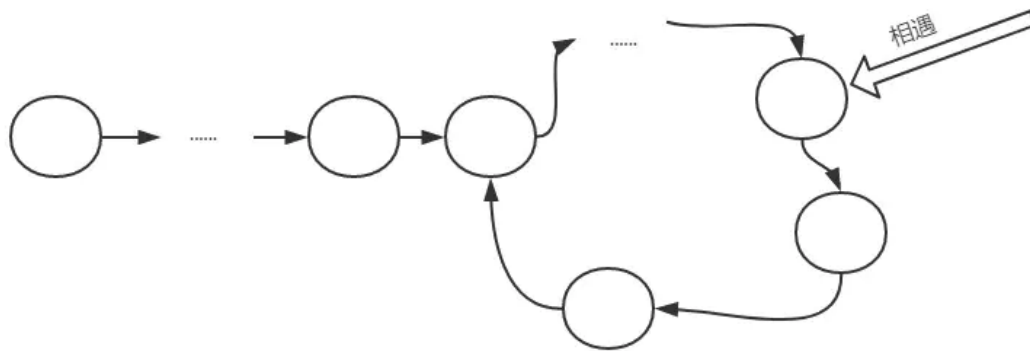
6.如何找到环的起点

给定一个链表，返回链表开始入环的第一个节点。如果链表无环，则返回 null。

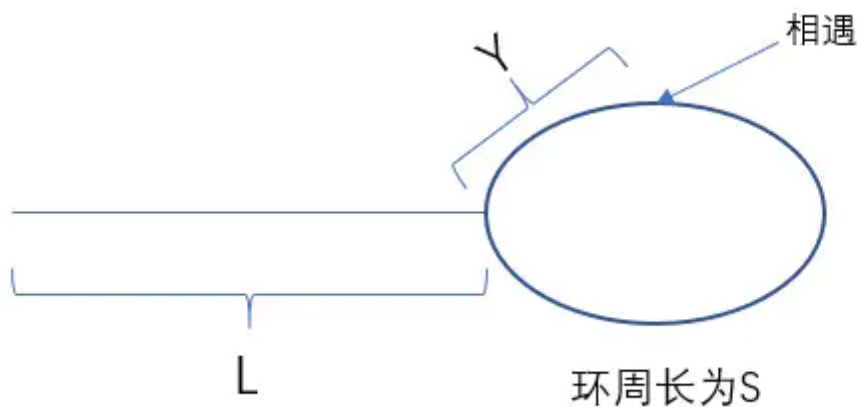
说明：不允许修改给定的链表。

思路分析

刚刚已经判断了如何判断出现环，那如何找到环的节点呢？我们来分析一波。



看上去比较繁琐，我们把它做进一步的抽象：



设快慢指针走了 x 秒，慢指针一秒走一次。

对快指针，有： $2x - L = m * S + Y$ -----①

对慢指针，有： $x - L = n * S + Y$ -----②

其中， m 、 n 均为自然数。

① - ② * 2 得：

$L = (m - n) * S - Y$ -----③

好，这是一个非常重要的等式。我们现在假设有一个新的指针在 L 段的最左端，慢指针现在还在相遇处。

让 **新指针** 和 **慢指针** 都每次走一步，那么，当 **新指针** 走了 L 步之后**到达环起点**，而与此同时，我们看看慢指针情况如何。

由③式，慢指针走了 $(m - n) * s - Y$ 个单位，以环起点为参照物，相遇时的位置为 Y，而现在由 $Y + (m - n) * s - Y$ 即 $(m - n) * s$ ，得知慢指针实际上参照环起点，走了整整 $(m - n)$ 圈。也就是说，**慢指针此时也到达了环起点**。:::tip 结论 现在的解法就很清晰了，当快慢指针相遇之后，让新指针从头出发，和慢指针同时前进，且每次前进一步，两者相遇的地方，就是**环起点**。:::

编程实现

懂得原理之后，实现起来就容易很多了。

```
/**
 * @param {ListNode} head
 * @return {ListNode}
 */
var detectCycle = function(head) {
    let dummyHead = new ListNode();
    dummyHead.next = head;
    let fast = slow = dummyHead;
    // 零个结点或者一个结点，肯定无环
    if(fast.next == null || fast.next.next == null)
        return null;
    while(fast && fast.next) {
        fast = fast.next.next;
        slow = slow.next;
        // 两者相遇了
        if(fast == slow) {
            let p = dummyHead;
            while(p != slow) {
                p = p.next;
                slow = slow.next;
            }
            return p;
        }
    }
    return null;
};
```

7.合并两个有序链表

将两个有序链表合并为一个新的有序链表并返回。新链表是通过拼接给定的两个链表的所有节点组成的。

示例:

```
输入: 1->2->4, 1->3->4
输出: 1->1->2->3->4->4
```

递归解法

递归解法更容易理解，我们先用递归来做一下：

```
/**
 * @param {ListNode} l1
 * @param {ListNode} l2
 * @return {ListNode}
 */
var mergeTwoLists = function(l1, l2) {
  const merge = (l1, l2) => {
    if(l1 == null) return l2;
    if(l2 == null) return l1;
    if(l1.val > l2.val) {
      l2.next = merge(l1, l2.next);
      return l2;
    } else {
      l1.next = merge(l1.next, l2);
      return l1;
    }
  }
  return merge(l1, l2);
};
```

循环解法

```
var mergeTwoLists = function(l1, l2) {
  if(l1 == null) return l2;
  if(l2 == null) return l1;
  let p = dummyHead = new ListNode();
  let p1 = l1, p2 = l2;
  while(p1 && p2) {
    if(p1.val > p2.val) {
      p.next = p2;
      p = p.next;
      p2 = p2.next;
    } else {
      p.next = p1;
      p = p.next;
      p1 = p1.next;
    }
  }
  // 循环完成后务必检查剩下的部分
  if(p1) p.next = p1;
  else p.next = p2;
  return dummyHead.next;
};
```


8.合并 K 个有序链表

合并 k 个排序链表，返回合并后的排序链表。请分析和描述算法的复杂度。

示例:

```
输入:
[
  1->4->5,
  1->3->4,
  2->6
]
输出: 1->1->2->3->4->4->5->6
```

自上而下(递归)实现

```
/**
 * @param {ListNode[]} lists
 * @return {ListNode}
 */
var mergeKLists = function(lists) {
  // 上面已经实现
  var mergeTwoLists = function(l1, l2) { /*上面已经实现*/ };
  const _mergeLists = (lists, start, end) => {
    if(end - start < 0) return null;
    if(end - start == 0) return lists[end];
    let mid = Math.floor(start + (end - start) / 2);
    return mergeTwoList(_mergeLists(lists, start, mid), _mergeLists(lists,
mid + 1, end));
  }
  return _mergeLists(lists, 0, lists.length - 1);
};
```

自下而上实现

在这里需要提醒大家的是，在自下而上的实现方式中，我为每一个链表绑定了一个虚拟头指针(dummyHead)，为什么这么做？

这是为了方便链表的合并，比如 l1 和 l2 合并之后，合并后链表的头指针就直接是 l1 的 dummyHead.next 值，等于说两个链表都合并到了 l1 当中，方便了后续的合并操作。

```
var mergeKLists = function(lists) {
  var mergeTwoLists = function(l1, l2) { /*上面已经实现*/ };
  // 边界情况
  if(!lists || !lists.length) return null;
  // 虚拟头指针集合
  let dummyHeads = [];
  // 初始化虚拟头指针
  for(let i = 0; i < lists.length; i++) {
    let node = new ListNode();
    node.next = lists[i];
    dummyHeads[i] = node;
  }
}
```

```
// 自底向上进行merge
for(let size = 1; size < lists.length; size += size){
    for(let i = 0; i + size < lists.length; i += 2 * size) {
        dummyHeads[i].next = mergeTwoLists(dummyHeads[i].next, dummyHeads[i +
size].next);
    }
}
return dummyHeads[0].next;
};
```

多个链表的合并到这里就实现完成了，在这里顺便告诉你这种归并的方式同时也是**对链表进行归并排序的核心代码**。希望你能好好体会**自上而下**和**自下而上**两种不同的实现细节，相信对你的编程内功是一个不错的提升。

9.判断回文链表

请判断一个单链表是否为回文链表。

示例1:

输入: 1->2
输出: false

示例2:

输入: 1->2->2->1
输出: true

你能否用 $O(n)$ 时间复杂度和 $O(1)$ 空间复杂度解决此题？

思路分析

这一题如果不考虑性能的限制，其实是非常简单的。但考虑到 $O(n)$ 时间复杂度和 $O(1)$ 空间复杂度，恐怕就值得停下来好好想想了。

题目的要求是单链表，没有办法访问前面的节点，那我们只得另辟蹊径：

找到链表中点，然后将后半部分反转，就可以依次比较得出结论了。下面我们来实现一波。

代码实现

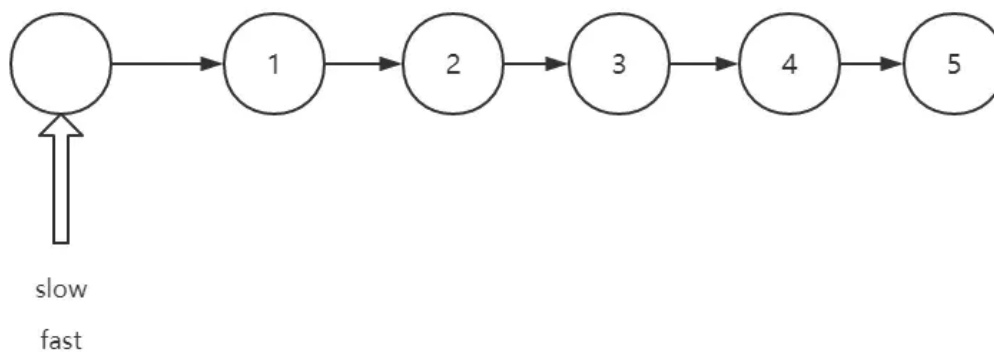
其实关键部分的代码就是找中点了。先亮剑：

```
let dummyHead = slow = fast = new ListNode();
dummyHead.next = head;
// 注意注意，来找中点了
while(fast && fast.next) {
    slow = slow.next;
    fast = fast.next.next;
}
```

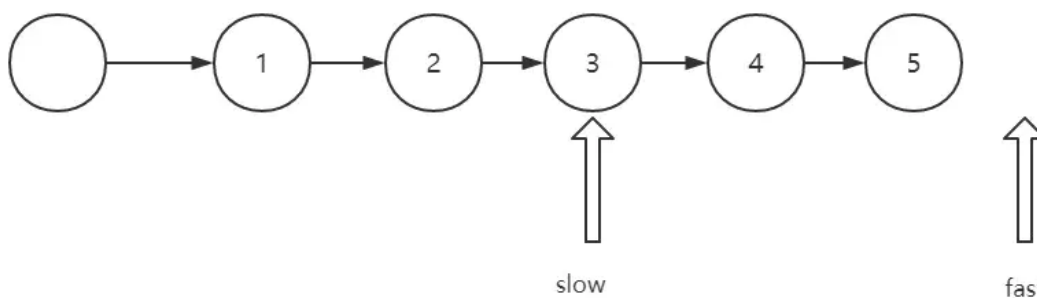
你可能会纳闷了，为什么边界要设成这样？

我们不妨来分析一下，分链表节点个数为 奇数 和 偶数 的时候分别讨论。

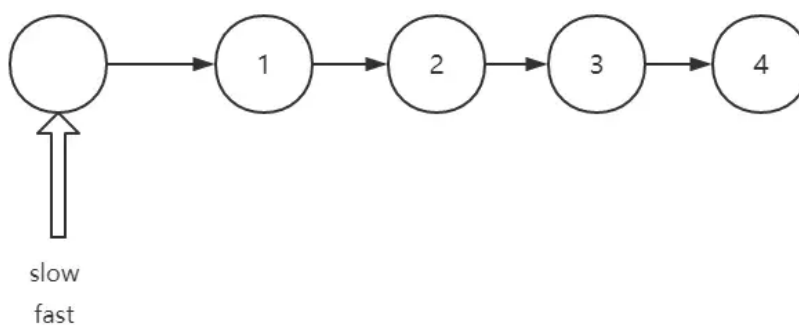
- 当链表节点个数为奇数



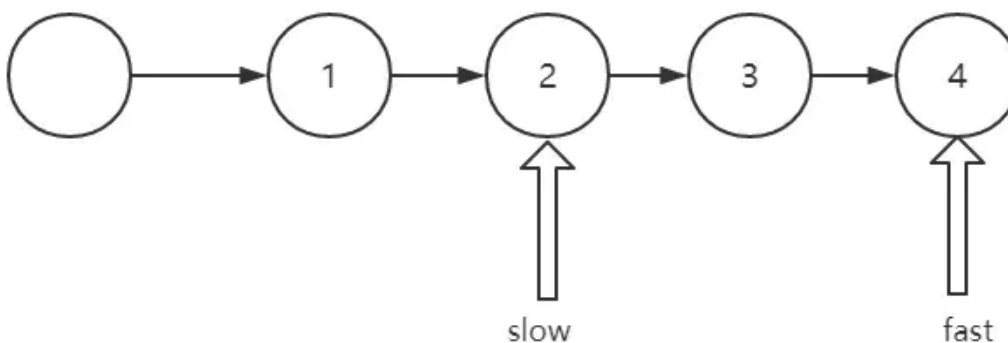
试着模拟一下，fast 为空的时候，停止循环，状态如下：



- 当链表节点个数为偶数



模拟走一遍，当 fast.next 为空的时候，停止循环，状态如下：



对于 fast 为空 和 fast.next 为空 两个条件，在奇数的情况下，总是 fast 为空 先出现，偶数的情况下，总是 fast.next 先出现。

也就是说：一旦 fast 为空，链表节点个数一定为奇数，否则为偶数。因此两种情况可以合并来讨论，当 fast 为空或者 fast.next 为空，循环就可以终止了。

完整实现如下:

```
/**
 * @param {ListNode} head
 * @return {boolean}
 */
var isPalindrome = function(head) {
    let reverse = (pre, cur) => {
        if(!cur) return pre;
        let next = cur.next;
        cur.next = pre;
        return reverse(cur, next);
    }
    let dummyHead = slow = fast = new ListNode();
    dummyHead.next = head;
    // 注意注意，来找中点了，黄金模板
    while(fast && fast.next) {
        slow = slow.next;
        fast = fast.next.next;
    }
    let next = slow.next;
    slow.next = null;
    let newStart = reverse(null, next);
    for(let p = head, newP = newStart; newP != null; p = p.next, newP = newP.next) {
        if(p.val != newP.val) return false;
    }
    return true;
};
```

二.栈和队列

1.有效括号

给定一个只包括 '(' , ')' , '{' , '}' , '[' , ']' 的字符串，判断字符串是否有效。

有效字符串需满足：

左括号必须用相同类型的右括号闭合。左括号必须以正确的顺序闭合。注意空字符串可被认为是有效字符串。

示例:

```
输入: "()"
输出: true
```

代码实现

```
/**
 * @param {string} s
 * @return {boolean}
 */
var isValid = function(s) {
```

```

let stack = [];
for(let i = 0; i < s.length; i++) {
    let ch = s.charAt(i);
    if(ch == '(' || ch == '[' || ch == '{')
        stack.push(ch);
    if(!stack.length) return false;
    if(ch == ')' && stack.pop() !== '(') return false;
    if(ch == ']' && stack.pop() !== '[') return false;
    if(ch == '}' && stack.pop() !== '{') return false;
}
return stack.length === 0;
};

```

2.多维数组 flatten

将多维数组转化为一维数组。

示例:

```
[1, [2, [3, [4, 5]]], 6] -> [1, 2, 3, 4, 5, 6]
```

代码实现

```

/**
 * @constructor
 * @param {NestedInteger[]} nestedList
 * @return {Integer[]}
 */
let flatten = (nestedList) => {
    let result = [];
    let fn = function (target, ary) {
        for (let i = 0; i < ary.length; i++) {
            let item = ary[i];
            if (Array.isArray(ary[i])) {
                fn(target, item);
            } else {
                target.push(item);
            }
        }
    }
    fn(result, nestedList)
    return result;
}

```

同时可采用 reduce 的方式, 一行就可以解决, 非常简洁。

```

let flatten = (nestedList) => nestedList.reduce((pre, cur) =>
pre.concat(Array.isArray(cur) ? flatten(cur): cur), [])

```

3.普通的层次遍历

给定一个二叉树，返回其按层次遍历的节点值。（即逐层地，从左到右访问所有节点）。

示例:

```
    3
   / \
  9  20
 /  \
15   7
```

结果应输出:

```
[
  [3],
  [9,20],
  [15,7]
]
```

实现

```
/**
 * @param {TreeNode} root
 * @return {number[][]}
 */
var levelOrder = function(root) {
  if(!root) return [];
  let queue = [];
  let res = [];
  let level = 0;
  queue.push(root);
  let temp;
  while(queue.length) {
    res.push([]);
    let size = queue.length;
    // 注意一下：size -- 在层次遍历中是一个非常重要的技巧
    while(size --) {
      // 出队
      let front = queue.shift();
      res[level].push(front.val);
      // 入队
      if(front.left) queue.push(front.left);
      if(front.right) queue.push(front.right);
    }
    level++;
  }
  return res;
};
```

4.二叉树的锯齿形层次遍历

给定一个二叉树，返回其节点值的锯齿形层次遍历。（即先从左往右，再从右往左进行下一层遍历，以此类推，层与层之间交替进行）。

例如：

给定二叉树 [3,9,20,null,null,15,7], 输出应如下：

```
    3
   / \
  9  20
   / \
  15  7
```

返回锯齿形层次遍历如下：

```
[
  [3],
  [20,9],
  [15,7]
]
```

思路

这一题思路稍微不同，但如果把握住层次遍历的思路，就会非常简单。

代码实现

```
var zigzagLevelOrder = function(root) {
    if(!root) return [];
    let queue = [];
    let res = [];
    let level = 0;
    queue.push(root);
    let temp;
    while(queue.length) {
        res.push([]);
        let size = queue.length;
        while(size --) {
            // 出队
            let front = queue.shift();
            res[level].push(front.val);
            if(front.left) queue.push(front.left);
            if(front.right) queue.push(front.right);
        }
        // 仅仅增加下面一行代码即可
        if(level % 2) res[level].reverse();
        level++;
    }
    return res;
};
```

5.二叉树的右视图

给定一棵二叉树，想象自己站在它的右侧，按照从顶部到底部的顺序，返回从右侧所能看到的节点值。

示例:

输入: [1,2,3,null,5,null,4]

输出: [1, 3, 4]

解释:



思路

右视图? 如果你以 DFS 即深度优先搜索的思路来想, 你会感觉异常的痛苦。没错, 当初我就是这样被坑的:)

但如果用广度优先搜索的思想, 即用层序遍历的方式, 求解这道题目也变得轻而易举。

代码实现

```
/**
 * @param {TreeNode} root
 * @return {number[]}
 */
var rightSideView = function(root) {
    if(!root) return [];
    let queue = [];
    let res = [];
    queue.push(root);
    while(queue.length) {
        res.push(queue[0].val);
        let size = queue.length;
        while(size --) {
            // 一个size的循环就是一层的遍历, 在这一层只拿最右边的结点
            let front = queue.shift();
            if(front.right) queue.push(front.right);
            if(front.left) queue.push(front.left);
        }
    }
    return res;
};
```


6.完全平方数

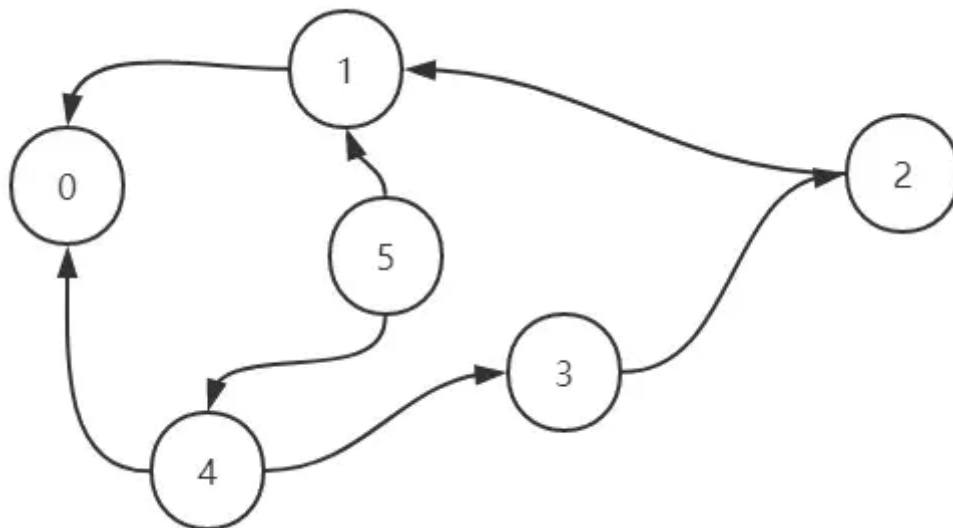
给定正整数 n ，找到若干个完全平方数（比如 1, 4, 9, 16, ...）使得它们的和等于 n 。你需要让组成和的完全平方数的个数最少。

示例:

输入: $n = 12$
输出: 3
解释: $12 = 4 + 4 + 4$.

思路

这一题其实最容易想到的思路是动态规划，我们放到后面专门来拆解。实际上用队列进行图的建模，也是可以顺利地用广度优先遍历的方式解决的。



看到这个图，你可能会有点懵，我稍微解释一下你就明白了。

在这个无权图中，每一个点指向的都是它可能经过的上一个节点。举例来说，对 5 而言，可能是 4 加上了 1 的平方 转换而来，也可能是 1 加上了 2 的平方 转换而来，因此跟 1 和 2 都有联系，依次类推。

那么我们现在要做了就是寻找到从 n 转换到 0 最短的连线数。

举个例子， $n = 8$ 时，我们需要找到它的邻居节点 4 和 7，此时到达 4 和到达 7 的步数都为 1，然后分别从 4 和 7 出发，4 找到邻居节点 3 和 0，达到 3 和 0 的步数都为 2，考虑到此时已经到达 0，遍历终止，返回到达 0 的步数 2 即可。

Talk is cheap, show me your code. 我们接下来一步步实现这个寻找的过程。

实现

接下来我们来实现第一版的代码。

```
/**
 * @param {number} n
 * @return {number}
```

```

*/
var numSquares = function(n) {
  let queue = [];
  queue.push([n, 0]);
  while(queue.length) {
    let [num, step] = queue.shift();
    for(let i = 1; ; i++) {
      let nextNum = num - i * i;
      if(nextNum < 0) break;
      // 还差最后一步就到了，直接返回 step + 1
      if(nextNum == 0) return step + 1;
      queue.push([nextNum, step + 1]);
    }
  }
  // 最后是不需要返回另外的值的，因为 1 也是完全平方数，所有的数都能用 1 来组合
};

```

这个解法从功能上来讲是没有问题的，但是其中隐藏了巨大的性能问题

那为什么会出现这样的问题？

出就出在这样一行代码：

```
queue.push([nextNum, step + 1]);
```

只要是大于 0 的数，统统塞进队列。要知道 $2 - 1 = 1$ ， $5 - 4 = 1$ ， $9 - 8 = 1$ 这样会重复非常多的 1，依次类推，也会重复非常多的 2, 3 等等等等。

这样大量的重复数字不仅仅消耗了更多的循环次数，同时也造成更加巨大的内存空间压力。

因此，我们需要对已经推入队列的数字进行标记，避免重复推入。改善代码如下：

```

var numSquares = function(n) {
  let map = new Map();
  let queue = [];
  queue.push([n, 0]);
  map.set(n, true);
  while(queue.length) {
    let [num, step] = queue.shift();
    for(let i = 1; ; i++) {
      let nextNum = num - i * i;
      if(nextNum < 0) break;
      if(nextNum == 0) return step + 1;
      // nextNum 未被访问过
      if(!map.get(nextNum)){
        queue.push([nextNum, step + 1]);
        // 标记已经访问过
        map.set(nextNum, true);
      }
    }
  }
};

```

7.单词接龙

给定两个单词 (beginWord 和 endWord) 和一个字典，找到从 beginWord 到 endWord 的最短转换序列的长度。转换需遵循如下规则：

- 每次转换只能改变一个字母。
- 转换过程中的中间单词必须是字典中的单词。

说明:

- 1) 如果不存在这样的转换序列，返回 0。
- 2) 所有单词具有相同的长度。
- 3) 所有单词只由小写字母组成。
- 4) 字典中不存在重复的单词。
- 5) 你可以假设 beginWord 和 endWord 是非空的，且二者不相同。

示例:

输入:

```
beginword = "hit",  
endword = "cog",  
wordList = ["hot","dot","dog","lot","log","cog"]
```

输出: 5

解释: 一个最短转换序列是 "hit" -> "hot" -> "dot" -> "dog" -> "cog",
返回它的长度 5。

思路

这一题是一个更加典型的用图建模的问题。如果每一个单词都是一个节点，那么只要和这个单词仅有一个字母不同，那么就是它的相邻节点。

这里我们可以通过 BFS 的方式来进行遍历。实现如下:

代码实现

```
/**  
 * @param {string} beginword  
 * @param {string} endword  
 * @param {string[]} wordList  
 * @return {number}  
 */  
var ladderLength = function(beginword, endword, wordList) {  
    // 两个单词在图中是否相邻  
    const isSimilar = (a, b) => {  
        let diff = 0  
        for(let i = 0; i < a.length; i++) {  
            if(a.charAt(i) !== b.charAt(i)) diff++;  
            if(diff > 1) return false;  
        }  
        return true;  
    }  
}
```

```

    }
    let queue = [beginword];
    let index = wordList.indexOf(beginword);
    if(index !== -1) wordList.splice(index, 1);
    let res = 2;
    while(queue.length) {
        let size = queue.length;
        while(size --) {
            let front = queue.shift();
            for(let i = 0; i < wordList.length; i++) {
                if(!isSimilar(front, wordList[i]))continue;
                // 找到了
                if(wordList[i] === endword) {
                    return res;
                }
                else {
                    queue.push(wordList[i]);
                }
                // wordList[i]已经成功推入，现在不需要了，删除即可
                // 这一步性能优化，相当关键，不然100%超时
                wordList.splice(i, 1);
                i --;
            }
        }
        // 步数 +1
        res += 1;
    }
    return 0;
};

```

8.优先队列

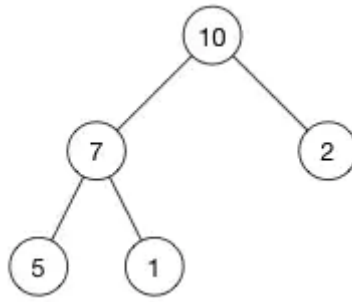
所谓优先队列，就是一种特殊的**队列**，其底层使用**堆**的结构，使得每次添加或者删除，让队首元素始终是优先级最高的。关于优先级通过什么字段、按照什么样的比较方式来设定，可以由我们自己来决定。

要实现优先队列，首先来实现一个堆的结构。

9.关于堆的说明

可能你以前没有接触过**堆**这种数据结构，但是其实是很简单的一种结构，其本质就是一棵二叉树。但是这棵二叉树比较特殊，除了用数组来依次存储各个节点(节点对应的数组下标和**层序遍历的序号**一致)之外，它需要保证**任何一个父节点的优先级大于子节点**，这也是它最关键的性质，因为保证了根元素一定是优先级最高的。

举一个例子：



现在这个堆的数组就是: **[10, 7, 2, 5, 1]**

因此也会产生两个非常关键的操作——**siftUp** 和 **siftDown**。

对于**siftUp**操作，我们试想一下现在有一个正常的堆，满足任何父元素优先级大于子元素，这时候向这个堆的数组末尾又添加了一个元素，那现在可能就不符合堆的结构特点了。那么现在我将新增的节点和其父节点进行比较，如果父节点优先级小于它，则两者交换，不断向上比较直到根节点为止，这样就保证了堆的正确结构。而这样的操作就是**siftUp**。

siftDown是与其相反方向的操作，从上到下比较，原理相同，也是为了保证堆的正确结构。

10.实现一个最大堆

最大堆，即堆顶元素为优先级最高的元素。

```
// 以最大堆为例来实现一波
/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number[]}
 */
class MaxHeap {
  constructor(arr = [], compare = null) {
    this.data = arr;
    this.size = arr.length;
    this.compare = compare;
  }
  getSize() {
    return this.size;
  }
  isEmpty() {
    return this.size === 0;
  }
  // 增加元素
  add(value) {
    this.data.push(value);
    this.size++;
    // 增加的时候把添加的元素进行 siftup
    this._siftUp(this.getSize() - 1);
  }
  // 找到优先级最高的元素
  findMax() {
    if (this.getSize() === 0)
      return;
    return this.data[0];
  }
  // 让优先级最高的元素(即队首元素)出队
```

```

extractMax() {
    // 1.保存队首元素
    let ret = this.findMax();
    // 2.让队首和队尾元素交换位置
    this._swap(0, this.getSize() - 1);
    // 3. 把队尾踢出去, size--
    this.data.pop();
    this.size--;
    // 4. 新的队首 siftDown
    this._siftDown(0);
    return ret;
}

toString() {
    console.log(this.data);
}

_swap(i, j) {
    [this.data[i], this.data[j]] = [this.data[j], this.data[i]];
}

_parent(index) {
    return Math.floor((index - 1) / 2);
}

_leftChild(index) {
    return 2 * index + 1;
}

_rightChild(index) {
    return 2 * index + 2;
}

_siftUp(k) {
    // 上浮操作, 只要子元素优先级比父节点大, 父子交换位置, 一直向上直到根节点
    while (k > 0 && this.compare(this.data[k], this.data[this._parent(k)])) {
        this._swap(k, this._parent(k));
        k = this._parent(k);
    }
}

_siftDown(k) {
    // 存在左孩子的时候
    while (this._leftChild(k) < this.size) {
        let j = this._leftChild(k);
        // 存在右孩子而且右孩子比左孩子大
        if (this._rightChild(k) < this.size &&
            this.compare(this.data[this._rightChild(k)], this.data[j])) {
            j++;
        }
        if (this.compare(this.data[k], this.data[j]))
            return;
        // 父节点比子节点小, 交换位置
        this._swap(k, j);
        // 继续下沉
        k = j;
    }
}
}

```

11.实现优先队列

有了最大堆作铺垫，实现优先队列就易如反掌，废话不多说，直接放上代码。

```
class PriorityQueue {
    // max 为优先队列的容量
    constructor(max, compare) {
        this.max = max;
        this.compare = compare;
        this.maxHeap = new MaxHeap([], compare);
    }

    getSize() {
        return this.maxHeap.getSize();
    }

    isEmpty() {
        return this.maxHeap.isEmpty();
    }

    getFront() {
        return this.maxHeap.findMax();
    }

    enqueue(e) {
        // 比当前最高的优先级的还要高，直接不处理
        if (this.getSize() === this.max) {
            if (this.compare(e, this.getFront())) return;
            this.dequeue();
        }
        return this.maxHeap.add(e);
    }

    dequeue() {
        if (this.getSize() === 0) return null;
        return this.maxHeap.extractMax();
    }
}
```

怎么样，是不是非常简单？传说中的**优先队列**也不过如此。

且慢，可能有人问：你怎么保证这个优先队列是正确的呢？

我们不妨来做一些测试：

```
let pq = new PriorityQueue(3);
pq.enqueue(1);
pq.enqueue(333);
console.log(pq.dequeue());
console.log(pq.dequeue());
pq.enqueue(3);
pq.enqueue(6);
pq.enqueue(62);
console.log(pq.dequeue());
console.log(pq.dequeue());
console.log(pq.dequeue());
```

复制代码

结果如下:

```
333
1
62
6
3
```

可见, 这个优先队列的功能初步满足了我们的预期。

12.前 K 个高频元素

给定一个非空的整数数组, 返回其中出现频率前 k 高的元素。

示例:

```
输入: nums = [1,1,1,2,2,3], k = 2
输出: [1,2]
```

说明:

- 你可以假设给定的 k 总是合理的, 且 $1 \leq k \leq$ 数组中不相同的元素的个数。
- 你的算法的时间复杂度必须优于 $O(n \log n)$, n 是数组的大小。

思路

首先要做的肯定是统计频率, 那之后如何来选取频率前 K 个元素同时又保证时间复杂度小于 $O(n \log n)$ 呢?

当然, 这是一道典型的考察优先队列的题, 利用容量为 K 的优先队列每次踢出不符合条件的值, 那么最后剩下的即为所求。整个时间复杂度成为 $O(n \log K)$, 明显是小于 $O(n \log n)$ 的。

既然是优先队列, 就涉及到如何来定义优先级的的问题。

倘若我们以高频率为高优先级, 那么队首始终是高频率的元素, 因此每次出队是踢出出现频率最高的元素, 假设优先队列容量为 K, 那照这么做, 剩下的是频率最低的 K 个元素, 显然不符合题意。

因此, 我们需要的是每次出队时踢出**频率最低的元素**, 这样最后剩下下来的就是频率最高 K 个元素。

是不是我们为了踢出**频率最低的元素**, 还要重新写一个小顶堆的实现呢?

完全不需要! 就像我刚才所说的, 合理地定义这个优先级的比较逻辑即可。接下来我们来具体实现一下。

代码实现


```

var topKFrequent = function(nums, k) {
    let map = {};
    let pq = new PriorityQueue(k, (a, b) => map[a] - map[b] < 0);
    for(let i = 0; i < nums.length; i++) {
        if(!map[nums[i]]) map[nums[i]] = 1;
        else map[nums[i]] = map[nums[i]] + 1;
    }
    let arr = Array.from(new Set(nums));
    for(let i = 0; i < arr.length; i++) {
        pq.enqueue(arr[i]);
    }
    return pq.maxHeap.data;
};

```

13.合并 K 个排序链表

合并 k 个排序链表，返回合并后的排序链表。请分析和描述算法的复杂度。

示例:

```

输入:
[
  1->4->5,
  1->3->4,
  2->6
]
输出: 1->1->2->3->4->4->5->6

```

这一题我们之前在链表实现过，殊不知，它也可以利用优先队列完美解决。

```

/**
 * @param {ListNode[]} lists
 * @return {ListNode}
 */
var mergeKLists = function(lists) {
    let dummyHead = p = new ListNode();
    // 定义优先级的函数，重要！
    let pq = new PriorityQueue(lists.length, (a, b) => a.val <= b.val);
    // 将头结点推入优先队列
    for(let i = 0; i < lists.length; i++)
        if(lists[i]) pq.enqueue(lists[i]);
    // 取出值最小的节点，如果 next 不为空，继续推入队列
    while(pq.getSize()) {
        let min = pq.dequeue();
        p.next = min;
        p = p.next;
        if(min.next) pq.enqueue(min.next);
    }
    return dummyHead.next;
};

```

14.什么是双端队列？

双端队列是一种特殊的队列，首尾都可以添加或者删除元素，是一种加强版的队列。

JS 中的数组就是一种典型的双端队列。push、pop 方法分别从**尾部**添加和删除元素，unshift、shift 方法分别从**首部**添加和删除元素。

15.滑动窗口最大值

给定一个数组 nums，有一个大小为 k 的滑动窗口从数组的最左侧移动到数组的最右侧。你只可以看到在滑动窗口内的 k 个数字。滑动窗口每次只向右移动一位。

返回滑动窗口中的最大值。

示例:

输入: nums = [1,3,-1,-3,5,3,6,7]，和 k = 3
输出: [3,3,5,5,6,7]
解释:

滑动窗口的位置	最大值
-----	-----
[1 3 -1] -3 5 3 6 7	3
1 [3 -1 -3] 5 3 6 7	3
1 3 [-1 -3 5] 3 6 7	5
1 3 -1 [-3 5 3] 6 7	5
1 3 -1 -3 [5 3 6] 7	6
1 3 -1 -3 5 [3 6 7]	7

要求: 时间复杂度应为线性。

思路

这是典型地使用双端队列求解的问题。

建立一个双端队列 window，每次 push 进来一个新的值，就将 window 中目前前面所有比它小的值都删除。利用双端队列的特性，可以从后往前遍历，遇到小的就删除之，否则停止。

这样可以保证队首始终是最大值，因此寻找最大值的时间复杂度可以降到 $O(1)$ 。由于 window 中会有越来越多的值被淘汰，因此整体的时间复杂度是线性的。

代码实现

代码非常的简洁，但是如果要写出 bug free 的代码还是有相当的难度的，希望你能自己独立实现一遍。

```
var maxSlidingWindow = function(nums, k) {  
  // 异常处理  
  if(nums.length === 0 || !k) return [];  
  let window = [], res = [];  
  for(let i = 0; i < nums.length; i++) {  
    // 先把滑动窗口之外的踢出  
    if(window[0] !== undefined && window[0] <= i - k) window.shift();  
    // 保证队首是最大的  
    while(nums[window[window.length - 1]] <= nums[i]) window.pop();  
    window.push(i);  
    if(window.length === k) res.push(nums[window[0]]);  
  }  
  return res;  
}
```

```
        window.push(i);
        if(i >= k - 1) res.push(nums[window[0]])
    }
    return res;
};
```

16. 栈实现队列

使用栈实现队列的下列操作:

push(x) -- 将一个元素放入队列的尾部。 pop() -- 从队列首部移除元素。 peek() -- 返回队列首部的元素。 empty() -- 返回队列是否为空。

示例:

```
let queue = new MyQueue();

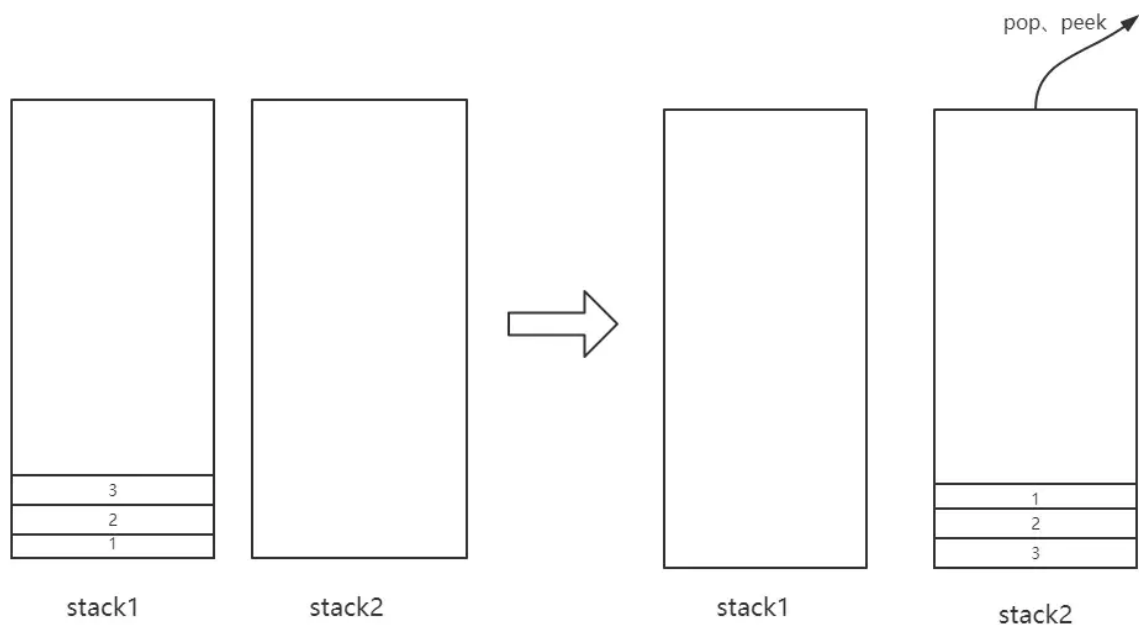
queue.push(1);
queue.push(2);
queue.peek(); // 返回 1
queue.pop(); // 返回 1
queue.empty(); // 返回 false
```

思路

既然栈是**先进后出**, 要想得到**先进先出**的效果, 我们不妨用两个栈。

当进行 push 操作时, push 到 `stack1`, 而进行 pop 和 peek 的操作时, 我们通过 `stack2`。

当然这其中有一个特殊情况, 就是 `stack2` 是空, 如何进行 pop 和 peek? 很简单, 把 `stack1` 中的元素依次 pop 并推入 `stack2` 中, 然后正常地操作 `stack2` 即可, 如下图所示:



这就就能保证先入先出的效果了。

代码实现

```
var MyQueue = function() {
  this.stack1 = [];
  this.stack2 = [];
};

MyQueue.prototype.push = function(x) {
  this.stack1.push(x);
};
// 将 stack1 的元素转移到 stack2
MyQueue.prototype.transform = function() {
  while(this.stack1.length) {
    this.stack2.push(this.stack1.pop());
  }
}

MyQueue.prototype.pop = function() {
  if(!this.stack2.length) this.transform();
  return this.stack2.pop();
};

MyQueue.prototype.peek = function() {
  if(!this.stack2.length) this.transform();
  return this.stack2[this.stack2.length - 1];
};

MyQueue.prototype.empty = function() {
  return !this.stack1.length && !this.stack2.length;
};
复制代码
```

17.队列实现栈

和上一题的效果刚好相反，用队列 **先进先出** 的方式来实现 **先进后出** 的效果。

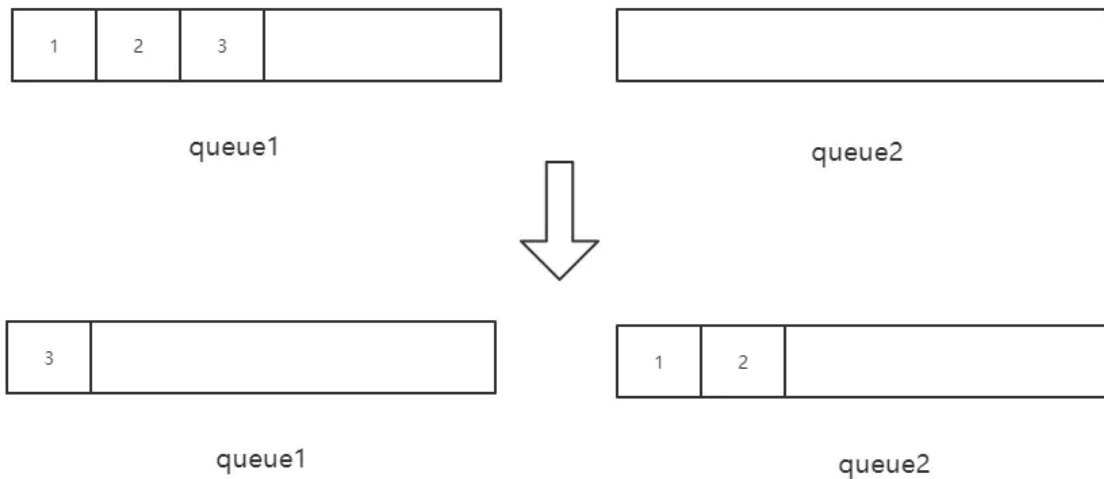
思路



queue1

以上面的队列为例，push 操作好说，直接从在队列末尾推入。但 pop 和 peek 呢？

回到我们的目标，我们的目标是拿到队尾的值，也就是 **3**。这就好办了，我们让前面的元素统统出队，只留队尾元素即可，剩下的元素让另外一个队列保存。



代码实现

实现过程中，值得注意的一点是，`queue1` 始终保存前面的元素，`queue2` 始终保存队尾元素（即栈顶元素）。

但是当 `push` 的时候有一个陷阱，就是当 `queue2` 已经有元素的时候，不能将新值 `push` 到 `queue1`，因为此时的**栈顶元素**应该更新。此时对于新的值来说，应先 `push` 到 `queue2`，然后将**旧的栈顶**从 `queue2` 出队，推入 `queue1`，这样就实现了**更新栈顶**的操作。

```
var MyStack = function() {
  this.queue1 = [];
  this.queue2 = [];
};
MyStack.prototype.push = function(x) {
  if(!this.queue2.length) this.queue1.push(x);
  else {
    // queue2 已经有值
    this.queue2.push(x);
    // 旧的栈顶移到 queue1 中
    this.queue1.push(this.queue2.shift());
  }
};
MyStack.prototype.transform = function() {
  while(this.queue1.length !== 1) {
    this.queue2.push(this.queue1.shift())
  }
  // queue2 保存了前面的元素
  // 让 queue1 和 queue2 交换
  // 现在queue1 包含前面的元素，queue2 里面就只包含队尾的元素
  let tmp = this.queue1;
  this.queue1 = this.queue2;
  this.queue2 = tmp;
}
MyStack.prototype.pop = function() {
  if(!this.queue2.length) this.transform();
  return this.queue2.shift();
};
MyStack.prototype.top = function() {
  if(!this.queue2.length) this.transform();
  return this.queue2[0];
};
```

```
MyStack.prototype.empty = function() {  
    return !this.queue1.length && !this.queue2.length;  
};
```

三.二叉树

1.前序遍历

示例:

示例:

输入: [1,null,2,3]

```
  1  
  \  
   2  
  \  
   3
```

输出: [1,2,3]

递归方式

```
/**  
 * @param {TreeNode} root  
 * @return {number[]}  
 */  
var preorderTraversal = function(root) {  
    let arr = [];  
    let traverse = (root) => {  
        if(root == null) return;  
        arr.push(root.val);  
        traverse(root.left);  
        traverse(root.right);  
    }  
    traverse(root);  
    return arr;  
};
```

非递归方式

```

var preorderTraversal = function(root) {
  if(root == null) return [];
  let stack = [], res = [];
  stack.push(root);
  while(stack.length) {
    let node = stack.pop();
    res.push(node.val);
    // 左孩子后进先出，进行先左后右的深度优先遍历
    if(node.right) stack.push(node.right);
    if(node.left) stack.push(node.left);
  }
  return res;
};

```

2.中序遍历

给定一个二叉树，返回它的中序 遍历。

示例:

```

输入: [1,null,2,3]
    1
     \
      2
     /
    3
输出: [1,3,2]

```

递归方式:

```

/**
 * @param {TreeNode} root
 * @return {number[]}
 */
var inorderTraversal = function(root) {
  let arr = [];
  let traverse = (root) => {
    if(root == null) return;
    traverse(root.left);
    arr.push(root.val);
    traverse(root.right);
  }
  traverse(root);
  return arr;
};

```

非递归方式

```

var inorderTraversal = function(root) {
  if(root == null) return [];

```

```

    let stack = [], res = [];
    let p = root;
    while(stack.length || p) {
        while(p) {
            stack.push(p);
            p = p.left;
        }
        let node = stack.pop();
        res.push(node.val);
        p = node.right;
    }
    return res;
};

```

3.后序遍历

给定一个二叉树，返回它的 后序 遍历。

示例:

输入: [1,null,2,3]

```

    1
     \
      2
     /
    3

```

输出: [3,2,1]

递归方式

```

/**
 * @param {TreeNode} root
 * @return {number[]}
 */
var postorderTraversal = function(root) {
    let arr = [];
    let traverse = (root) => {
        if(root == null) return;
        traverse(root.left);
        traverse(root.right);
        arr.push(root.val);
    }
    traverse(root);
    return arr
};

```

非递归方式

```

var postorderTraversal = function(root) {

```



```

    if(root == null) return [];
    let stack = [], res = [];
    let visited = new Set();
    let p = root;
    while(stack.length || p) {
        while(p) {
            stack.push(p);
            p = p.left;
        }
        let node = stack[stack.length - 1];
        // 如果右孩子存在，而且右孩子未被访问
        if(node.right && !visited.has(node.right)) {
            p = node.right;
            visited.add(node.right);
        } else {
            res.push(node.val);
            stack.pop();
        }
    }
    return res;
};

```

4.最大深度

给定一个二叉树，找出其最大深度。

二叉树的深度为根节点到最远叶子节点的最长路径上的节点数。

说明: 叶子节点是指没有子节点的节点。

示例: 给定二叉树 [3,9,20,null,null,15,7]:

```

    3
   / \
  9  20
   / \
  15  7

```

递归实现

实现非常简单，直接贴出代码:

```

/**
 * @param {TreeNode} root
 * @return {number}
 */
var maxDepth = function(root) {
    // 递归终止条件
    if(root == null) return 0;
    return Math.max(maxDepth(root.left) + 1, maxDepth(root.right) + 1);
};

```

非递归实现

采用层序遍历的方式，非常好理解。

```
var maxDepth = function(root) {  
    if(root == null) return 0;  
    let queue = [root];  
    let level = 0;  
    while(queue.length) {  
        let size = queue.length;  
        while(size --) {  
            let front = queue.shift();  
            if(front.left) queue.push(front.left);  
            if(front.right) queue.push(front.right);  
        }  
        // level ++ 后的值代表着现在已经处理完了几层节点  
        level ++;  
    }  
    return level;  
};
```

5.最小深度

给定一个二叉树，找出其最小深度。

最小深度是从根节点到最近叶子节点的最短路径上的节点数量。

说明: 叶子节点是指没有子节点的节点。

示例:

给定二叉树 [3,9,20,null,null,15,7]:

```
    3  
   / \  
  9  20  
 /  \  
15  7
```

返回它的最小深度 2.

递归实现

在实现的过程中，如果按照最大深度的方式来做会出现一个陷阱，即:

```
/**  
 * @param {TreeNode} root  
 * @return {number}  
 */  
var minDepth = function(root) {  
    // 递归终止条件  
    if(root == null) return 0;  
    return Math.min(minDepth(root.left) + 1, minDepth(root.right)+1);  
};
```

当 root 节点有一个孩子为空的时候，此时返回的是 1，但这是不对的，最小高度指的是**根节点到最近叶子节点**的最小路径，而不是到一个空节点的路径。

因此我们需要做如下的调整：

```
var minDepth = function(root) {
  if(root == null) return 0;
  // 左右孩子都不为空才能像刚才那样调用
  if(root.left && root.right)
    return Math.min(minDepth(root.left), minDepth(root.right)) + 1;
  // 右孩子为空了，直接忽略之
  else if(root.left)
    return minDepth(root.left) + 1;
  // 左孩子为空，忽略
  else if(root.right)
    return minDepth(root.right) + 1;
  // 两个孩子都为空，说明到达了叶子节点，返回 1
  else return 1;
};
```

这样程序便能正常工作了。

非递归实现

类似于 **最大高度** 问题，采用了层序遍历的方式，很容易理解。

```
var minDepth = function(root) {
  if(root == null) return 0;
  let queue = [root];
  let level = 0;
  while(queue.length) {
    let size = queue.length;
    while(size --) {
      let front = queue.shift();
      // 找到叶子节点
      if(!front.left && !front.right) return level + 1;
      if(front.left) queue.push(front.left);
      if(front.right) queue.push(front.right);
    }
    // level ++ 后的值代表着现在已经处理完了几层节点
    level ++;
  }
  return level;
};
```

6.对称二叉树

给定一个二叉树，检查它是否是镜像对称的。

例如，二叉树 [1,2,2,3,4,4,3] 是对称的。

```

      1
     /\
    2  2
   /\ /\
  3 4 4 3

```

但是下面这个 [1,2,2,null,3,null,3] 则不是镜像对称的:

```

      1
     /\
    2  2
     \  \
     3   3

```

递归实现

递归方式的代码是非常干练和优雅的，希望你先自己实现一遍，然后对比改进。

```

/**
 * @param {TreeNode} root
 * @return {boolean}
 */
var isSymmetric = function(root) {
  let help = (node1, node2) => {
    // 都为空
    if(!node1 && !node2) return true;
    // 一个为空一个不为空，或者两个节点值不相等
    if(!node1 || !node2 || node1.val !== node2.val) return false;
    return help(node1.left, node2.right) && help(node1.right, node2.left);
  }
  if(root == null) return true;
  return help(root.left, root.right);
};

```

非递归实现

用一个队列保存访问过的节点，每次取出两个节点，进行比较。

```

var isSymmetric = function(root) {
  if(root == null) return true;
  let queue = [root.left, root.right];
  let node1, node2;
  while(queue.length) {
    node1 = queue.shift();
    node2 = queue.shift();
    // 两节点均为空
    if(!node1 && !node2) continue;
    // 一个为空一个不为空，或者两个节点值不相等
    if(!node1 || !node2 || node1.val !== node2.val) return false;
    queue.push(node1.left);
    queue.push(node2.right);
    queue.push(node1.right);
  }
};

```

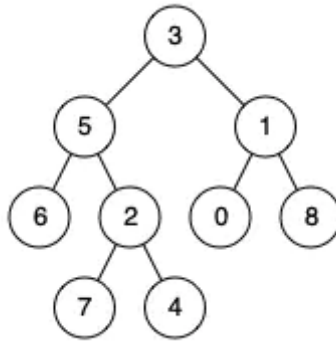
```

        queue.push(node2.left);
    }
    return true;
};

```

7. 二叉树的最近公共祖先

对于一个普通的二叉树: root = [3,5,1,6,2,0,8,null,null,7,4]



输入: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1
 输出: 3
 解释: 节点 5 和节点 1 的最近公共祖先是节点 3。

思路分析

思路一: 首先遍历一遍二叉树, 记录下每个节点的父节点。然后对于题目给的 p 节点, 根据这个记录表不断的找 p 的上层节点, 直到根, 记录下 p 的**上层节点集合**。然后对于 q 节点, 根据记录不断向上找它的上层节点, 在寻找的过程中一旦发现**这个上层节点已经包含在刚刚的集合中**, 说明发现了最近公共祖先, 直接返回。

思路二: 深度优先遍历二叉树, 如果当前节点为 p 或者 q, 直接返回这个节点, 否则查看左右孩子, 左孩子中不包含 p 或者 q 则去找右孩子, 右孩子不包含 p 或者 q 就去找左孩子, 剩下的情况就是**左右孩子中都存在 p 或者 q**, 那么此时直接返回这个节点。

祖先节点集合法

```

/**
 * @param {TreeNode} root
 * @param {TreeNode} p
 * @param {TreeNode} q
 * @return {TreeNode}
 */
var lowestCommonAncestor = function(root, p, q) {
    if(root == null || root == p || root == q) return root;
    let set = new Set();
    let map = new WeakMap();
    let queue = [];
    queue.push(root);
    // 层序遍历

```

```

while(queue.length) {
    let size = queue.length;
    while(size --) {
        let front = queue.shift();
        if(front.left) {
            queue.push(front.left);
            // 记录父亲节点
            map.set(front.left, front);
        }
        if(front.right) {
            queue.push(front.right);
            // 记录父亲节点
            map.set(front.right, front);
        }
    }
}
// 构造 p 的上层节点集合
while(p) {
    set.add(p);
    p = map.get(p);
}
while(q) {
    // 一旦发现公共节点重合，直接返回
    if(set.has(q)) return q;
    q = map.get(q);
}
};

```

可以看到整棵二叉树遍历了一遍，时间复杂度大致是 $O(n)$ ，但是由于哈希表的存在，空间复杂度比较高，接下来我们来用另一种遍历的方式，可以大大减少空间的开销。

深度优先遍历法

代码非常简洁、美观，不过更重要的是体会其中递归调用的过程，代码是自顶向下执行的，我建议大家用**自底向上**的方式来理解它，即从最左下的节点开始分析，相信你会很好的理解整个过程。

```

var lowestCommonAncestor = function(root, p, q) {
    if (root == null || root == p || root == q) return root;
    let left = lowestCommonAncestor(root.left, p, q);
    let right = lowestCommonAncestor(root.right, p, q);
    if(left == null) return right;
    else if(right == null) return left;
    return root;
};

```

8. 二叉搜索树的最近公共祖先

给定如下二叉搜索树: root = [6,2,8,0,4,7,9,null,null,3,5]

输入: root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 8
 输出: 6
 解释: 节点 2 和节点 8 的最近公共祖先是 6。

实现

二叉搜索树作为一种特殊的二叉树，当然是可以用上述的两种方式来实现的。

不过借助二叉搜索树有序的特性，我们也可以写出另外一个版本的深度优化遍历。

```
/**
 * @param {TreeNode} root
 * @param {TreeNode} p
 * @param {TreeNode} q
 * @return {TreeNode}
 */
var lowestCommonAncestor = function(root, p, q) {
    if(root == null || root == p || root == q) return root;
    // root.val 比 p 和 q 都大，找左孩子
    if(root.val > p.val && root.val > q.val)
        return lowestCommonAncestor(root.left, p, q);
    // root.val 比 p 和 q 都小，找右孩子
    if(root.val < p.val && root.val < q.val)
        return lowestCommonAncestor(root.right, p, q);
    else
        return root;
};
```

同时也可以采用非递归的方式：

```
var lowestCommonAncestor = function(root, p, q) {
    let node = root;
    while(node) {
        if(p.val > node.val && q.val > node.val)
            node = node.right;
        else if(p.val < node.val && q.val < node.val)
            node = node.left;
        else return node;
    }
};
```

是不是被二叉树精简而优雅的代码惊艳到了呢？希望你能好好体会其中遍历的过程，然后务必自己独立实现一遍，保证对这种数据结构足够熟悉，增强自己的编程内力。

9.二叉树的直径

给定一棵二叉树，你需要计算它的直径长度。一棵二叉树的直径长度是任意两个结点路径长度中的最大值。这条路径可能穿过根结点。

示例：给定二叉树

```

    1
   /\
  2  3
 /\  \
4   5
```

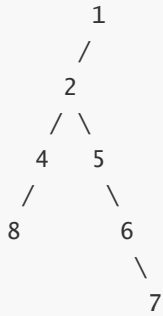
返回 3, 它的长度是路径 [4,2,1,3] 或者 [5,2,1,3]。

注意：两结点之间的路径长度是以它们之间边的数目表示。

思路

所谓的求直径, 本质上是求树中节点左右子树高度和的最大值。

注意, 这里我说的是树中节点, 并非根节点。因为会有这样一种情况:



那这个时候, 直径最大的路径是: 8 -> 4 -> 2 -> 5 -> 6 -> 7。交界的元素并不是根节点。这是这个问题特别需要注意的地方, 不然无解。

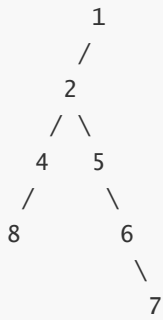
初步求解

目标已经确定了, 求树中节点左右子树高度和的最大值。开干!

```
/**
 * @param {TreeNode} root
 * @return {number}
 */
var diameterOfBinaryTree = function(root) {
  // 求最大深度
  let maxDepth = (node) => {
    if(node == null) return 0;
    return Math.max(maxDepth(node.left) + 1, maxDepth(node.right) + 1);
  }
  let help = (node) => {
    if(node == null) return 0;
    let rootSum = maxDepth(node.left) + maxDepth(node.right);
    let childSum = Math.max(help(node.left), help(node.right));
    return Math.max(rootSum, childSum);
  }
  if(root == null) return 0;
  return help(root);
};
```

这样一段代码放到 LeetCode 是可以通过, 但时间上却不让人很满意, 为什么呢?

因为在反复调用 maxDepth 的过程, 对树中的一些节点增加了很多不必要的访问。比如:



我们看什么时候访问节点 8，maxDepth(节点 2)的时候访问，maxDepth(节点 4)的时候又会访问，如果节点层级更高，重复访问的次数更加频繁，剩下的节点6、节点 7 都是同理。每一个节点访问的次数大概是 $O(\log K)$ (设当前节点在第 K 层)。那能不能把这个频率降到 $O(1)$ 呢？

答案是肯定的，接下来我们来优化这个算法。

优化解法

```
var diameterOfBinaryTree = function(root) {
  let help = (node) => {
    if(node == null) return 0;
    let left = node.left ? help(node.left) + 1: 0;
    let right = node.right ? help(node.right) + 1: 0;
    let cur = left + right;
    if(cur > max) max = cur;
    // 这个返回的操作相当关键
    return Math.max(left, right);
  }
  let max = 0;
  if(root == null) return 0;
  help(root);
  return max;
};
```

在这个过程中设置了一个 `max` 全局变量，深度优先遍历这棵树，每遍历完一个节点就更新 `max`，并通过返回值的方式 **自底向上** 把当前节点左右子树的最大高度传给父函数使用，使得每个节点只需访问一次即可。

现在提交我们优化后的代码，时间消耗明显降低。

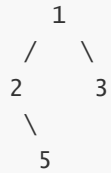
10.二叉树的所有路径

给定一个二叉树，返回所有从根节点到叶子节点的路径。

说明: 叶子节点是指没有子节点的节点。

示例:

输入：



输出：["1->2->5", "1->3"]

解释：所有根节点到叶子节点的路径为: 1->2->5, 1->3

递归解法

利用 DFS(深度优先遍历) 的方式进行遍历。

```
/**
 * @param {TreeNode} root
 * @return {string[]}
 */
var binaryTreePaths = function(root) {
    let path = [];
    let res = [];
    let dfs = (node) => {
        if(node == null) return;
        path.push(node);
        dfs(node.left);
        dfs(node.right);
        if(!node.left && !node.right)
            res.push(path.map(item => item.val).join('->'));
        // 注意每访问完一个节点记得把它从path中删除，达到回溯效果
        path.pop();
    }
    dfs(root);
    return res;
};
```

非递归解法

接下来我们通过 [非递归的后序遍历](#) 的方式来实现一下, 顺便复习一下后序遍历的实现。 ::: tip 提示 后序遍历其实也是 DFS 的一种具体实现方式。 :::

```
var binaryTreePaths = function(root) {
    if(root == null) return [];
    let stack = [];
    let p = root;
    let set = new Set();
    res = [];
    while(stack.length || p) {
        while(p) {
            stack.push(p);
            p = p.left;
        }
    }
```

```

    let node = stack[stack.length - 1];
    // 叶子节点
    if(!node.right && !node.left) {
        res.push(stack.map(item => item.val).join('->'));
    }
    // 右孩子存在，且右孩子未被访问
    if(node.right && !set.has(node.right)) {
        p = node.right;
        set.add(node.right);
    } else {
        stack.pop();
    }
}
return res;
};

```

11. 二叉树的最大路径和

给定一个非空二叉树，返回其最大路径和。

本题中，路径被定义为一条从树中任意节点出发，达到任意节点的序列。该路径至少包含一个节点，且不一定经过根节点。

示例:

输入: [-10,9,20,null,null,15,7]

```

    -10
   /  \
  9    20
 /  \
15   7

```

输出: 42

递归解

```

/**
 * @param {TreeNode} root
 * @return {number}
 */
var maxPathSum = function(root) {
    let help = (node) => {
        if(node == null) return 0;
        let left = Math.max(help(node.left), 0);
        let right = Math.max(help(node.right), 0);
        let cur = left + node.val + right;
        // 如果发现某一个节点上的路径值比max还大，则更新max
        if(cur > max) max = cur;
        // left 和 right 永远是"一根筋"，中间不会有转折
        return Math.max(left, right) + node.val;
    }
    let max = Number.MIN_SAFE_INTEGER;

```

```
    help(root);  
    return max;  
};
```

12.验证二叉搜索树

给定一个二叉树，判断其是否是一个有效的二叉搜索树。

假设一个二叉搜索树具有如下特征：

节点的左子树只包含小于当前节点的数。 节点的右子树只包含大于当前节点的数。 所有左子树和右子树自身必须也是二叉搜索树。

示例 1:

```
输入：  
    2  
   /\n  1  3  
输出：true
```

方法一: 中序遍历

通过中序遍历，保存前一个节点的值，扫描到当前节点时，和前一个节点的值比较，如果大于前一个节点，则满足条件，否则不是二叉搜索树。

```
/**  
 * @param {TreeNode} root  
 * @return {boolean}  
 */  
var isValidBST = function(root) {  
    let prev = null;  
    const help = (node) => {  
        if(node == null) return true;  
        if(!help(node.left)) return false;  
        if(prev !== null && prev >= node.val) return false;  
        // 保存当前节点，为下一个节点的遍历做准备  
        prev = node.val;  
        return help(node.right);  
    }  
    return help(root);  
};
```

方法二: 限定上下界进行DFS

二叉搜索树每一个节点的值，都有一个**上界和下界**，深度优先遍历的过程中，如果访问左孩子，则通过当前节点的值来**更新左孩子节点的上界**，同时访问右孩子，则**更新右孩子的下界**，只要出现节点值越界的情况，则不满足二叉搜索树的条件。

```

    parent
   /    \
 left   right

```

假设这是一棵巨大的二叉树的一个部分(parent、left、right都是实实在在的节点)，那么全部的节点排完序一定是这样：

...left, parent, right...

可以看到左孩子的 **最严格的上界** 是该节点，同时，右孩子的 **最严格的下界** 也是该节点。我们按照这样的规则来进行更新上下界。

递归实现：

```

var isValidBST = function(root) {
  const help = (node, max, min) => {
    if(node == null) return true;
    if(node.val >= max || node.val <= min) return false;
    // 左孩子更新上界，右孩子更新下界，相当于边界要求越来越苛刻
    return help(node.left, node.val, min)
      && help(node.right, max, node.val);
  }
  return help(root, Number.MAX_SAFE_INTEGER, Number.MIN_SAFE_INTEGER);
};

```

非递归实现：

```

var isValidBST = function(root) {
  if(root == null) return true;
  let stack = [root];
  let min = Number.MIN_SAFE_INTEGER;
  let max = Number.MAX_SAFE_INTEGER;
  root.max = max; root.min = min;
  while(stack.length) {
    let node = stack.pop();
    if(node.val <= node.min || node.val >= node.max)
      return false;
    if(node.left) {
      stack.push(node.left);
      // 更新上下界
      node.left.max = node.val;
      node.left.min = node.min;
    }
    if(node.right) {
      stack.push(node.right);
      // 更新上下界
      node.right.max = node.max;
      node.right.min = node.val;
    }
  }
  return true;
};

```

13.将有序数组转换为二叉搜索树

将一个按照升序排列的有序数组，转换为一棵高度平衡二叉搜索树。

本题中，一个高度平衡二叉树是指一个二叉树每个节点的左右两个子树的高度差的绝对值不超过 1。

示例:

给定有序数组: `[-10,-3,0,5,9]`,

一个可能的答案是: `[0,-3,9,-10,null,5]`, 它可以表示下面这个高度平衡二叉搜索树:

```

      0
     / \
    -3  9
   /   /
  -10  5
```

递归实现

```
/**
 * @param {number[]} nums
 * @return {TreeNode}
 */
var sortedArrayToBST = function(nums) {
  let help = (start, end) => {
    if(start > end) return null;
    if(start === end) return new TreeNode(nums[start]);
    let mid = Math.floor((start + end) / 2);
    // 找出中点建立节点
    let node = new TreeNode(nums[mid]);
    node.left = help(start, mid - 1);
    node.right = help(mid + 1, end);
    return node;
  }
  return help(0, nums.length - 1);
};
```

递归程序比较好理解，不断地调用 help 完成整棵树树的构建。那如何用非递归来解决呢？我觉得这是一个非常值得大家思考的问题。希望你能动手试一试，如果实在想不出来，可以参考下面我写的非递归版本。

其实思路跟递归的版本是一样的，只不过实现起来是用栈来实现 DFS 的效果。

```
/**
 * @param {number[]} nums
 * @return {TreeNode}
 */
var sortedArrayToBST = function(nums) {
  if(nums.length === 0) return null;
  let mid = Math.floor(nums.length / 2);
  let root = new TreeNode(nums[mid]);
  // 说明: 1. index 指的是当前元素在数组中的索引
  //       2. 每一个节点的值都是区间中点, 那么 start 属性就是这个区间的起点, end 为其终点
  root.index = mid; root.start = 0; root.end = nums.length - 1;
```

```

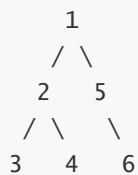
let stack = [root];
while(stack.length) {
    let node = stack.pop();
    // node 出来了，它本身包含了一个区间，[start, ..., index, ... end]
    // 下面判断[node.start, node.index - 1]之间是否还有开发的余地
    if(node.index - 1 >= node.start) {
        let leftMid = Math.floor((node.start + node.index)/2);
        let leftNode = new TreeNode(nums[leftMid]);
        node.left = leftNode;
        // 初始化新节点的区间起点、终点和索引
        leftNode.start = node.start;
        leftNode.end = node.index - 1;
        leftNode.index = leftMid;
        stack.push(leftNode);
    }
    // 中间夹着node.index，已经有元素了，这个位置不能再开发
    // 下面判断[node.index + 1, node.end]之间是否还有开发的余地
    if(node.end >= node.index + 1) {
        let rightMid = Math.floor((node.index + 1 + node.end)/2);
        let rightNode = new TreeNode(nums[rightMid]);
        node.right = rightNode;
        // 初始化新节点的区间起点、终点和索引
        rightNode.start = node.index + 1;
        rightNode.end = node.end;
        rightNode.index = rightMid;
        stack.push(rightNode);
    }
}
return root;
};

```

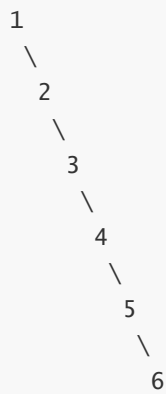
14.二叉树展开为链表

给定一个二叉(搜索)树，原地将它展开为链表。

例如，给定二叉树

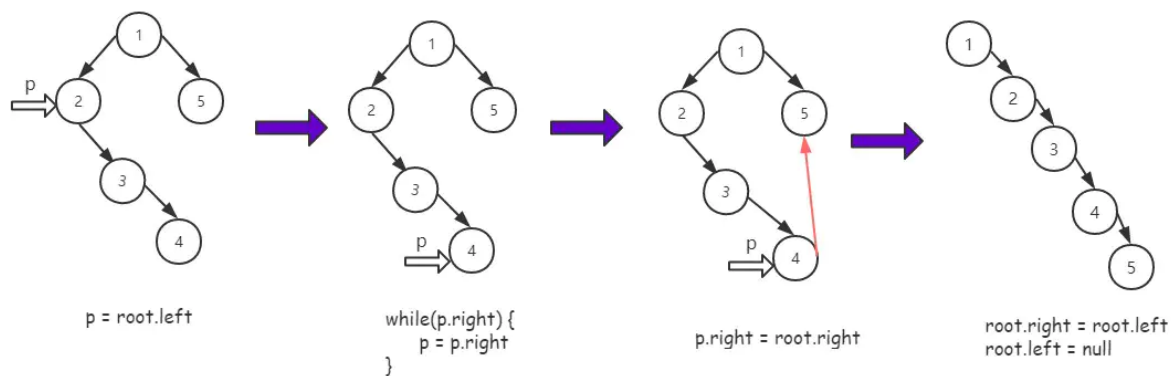


将其展开为：



递归方式

采用后序遍历，遍历完左右孩子我们要做些什么呢？用下面的图来演示一下：



```

/**
 * @param {TreeNode} root
 * @return {void} Do not return anything, modify root in-place instead.
 */
var flatten = function(root) {
  if(root == null) return;
  flatten(root.left);
  flatten(root.right);
  if(root.left) {
    let p = root.left;
    while(p.right) {
      p = p.right;
    }
    p.right = root.right;
    root.right = root.left;
    root.left = null;
  }
};

```

非递归方式

采用非递归的后序遍历方式，思路跟之前的完全一样。


```

var flatten = function(root) {
    if(root == null) return;
    let stack = [];
    let visited = new Set();
    let p = root;
    // 开始后序遍历
    while(stack.length || p) {
        while(p) {
            stack.push(p);
            p = p.left;
        }
        let node = stack[stack.length - 1];
        // 如果右孩子存在，而且右孩子未被访问
        if(node.right && !visited.has(node.right)) {
            p = node.right;
            visited.add(node.right);
        } else {
            // 以下为思路图中关键逻辑
            if(node.left) {
                let p = node.left;
                while(p.right) {
                    p = p.right;
                }
                p.right = node.right;
                node.right = node.left;
                node.left = null;
            }
            stack.pop();
        }
    }
};

```

15. 不同的二叉搜索树II

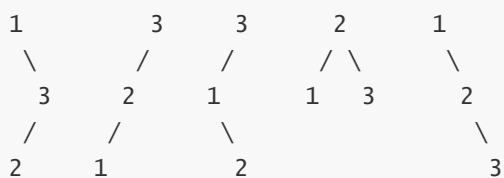
给定一个整数 n ，生成所有由 $1 \dots n$ 为节点所组成的二叉搜索树。

示例:

输入: 3
 输出:
 [
 [1,null,3,2],
 [3,2,null,1],
 [3,1,null,null,2],
 [2,1,3],
 [1,null,2,null,3]
]

解释:

以上的输出对应以下 5 种不同结构的二叉搜索树:



递归解法

递归创建子树

```
/**
 * @param {number} n
 * @return {TreeNode[]}
 */
var generateTrees = function(n) {
  let help = (start, end) => {
    if(start > end) return [null];
    if(start === end) return [new TreeNode(start)];
    let res = [];
    for(let i = start; i <= end; i++) {
      // 左孩子集
      let leftNodes = help(start, i - 1);
      // 右孩子集
      let rightNodes = help(i + 1, end);
      for(let j = 0; j < leftNodes.length; j++) {
        for(let k = 0; k < rightNodes.length; k++) {
          let root = new TreeNode(i);
          root.left = leftNodes[j];
          root.right = rightNodes[k];
          res.push(root);
        }
      }
    }
    return res;
  }
  if(n == 0) return [];
  return help(1, n);
};
```

非递归解法

```
var generateTrees = function(n) {
  let clone = (node, offset) => {
    if(node == null) return null;
    let newnode = new TreeNode(node.val + offset);
    newnode.left = clone(node.left, offset);
    newnode.right = clone(node.right, offset);
    return newnode;
  }
  if(n == 0) return [];
  let dp = [];
  dp[0] = [null];
  // i 是子问题中的节点个数，子问题：[1], [1,2], [1,2,3]...逐步递增，直到[1,2,3...,n]
  for(let i = 1; i <= n; i++) {
    dp[i] = [];
    for(let j = 1; j <= i; j++) {
      // 左子树集
      for(let leftNode of dp[j - 1]) {
        // 右子树集
```

```
        for(let rightNode of dp[i - j]) {
            let node = new TreeNode(j);
            // 左子树结构共享
            node.left = leftNode;
            // 右子树无法共享，但可以借用节点个数相同的树，每个节点增加一个偏移量
            node.right = clone(rightNode, j);
            dp[i].push(node);
        }
    }
}
return dp[n];
};
```